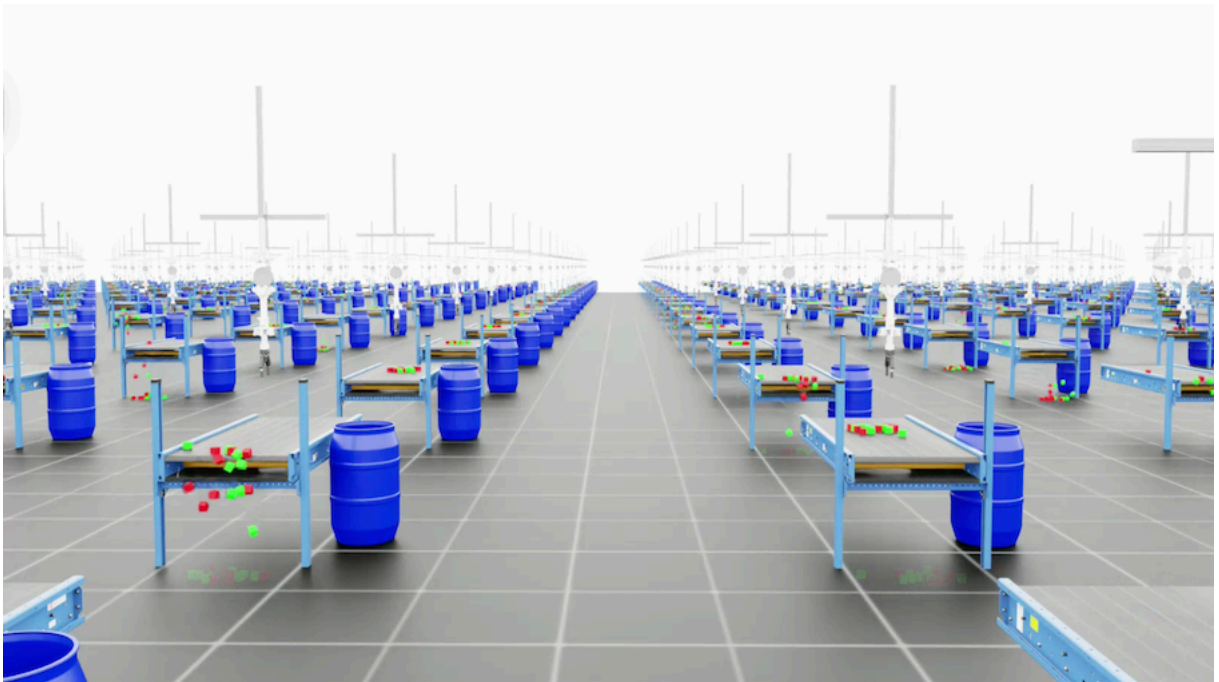


Simulative Construction and Verification of a Tendon-Driven Robot for Conveyor-Based Pick-and-Place Tasks Using Isaac Lab

Project Thesis in the program Computational Engineering M. Sc.

**Friedrich-Alexander-Universität Erlangen-Nürnberg
Lehrstuhl für Fertigungsautomatisierung und Produktionssystematik
Prof. Dr.-Ing. J. Franke**



Author: Georg Meyer

Student ID: 22791103

Supervisors: Prof. Dr.-Ing. J. Franke
Dipl.-Ing. M. Landgraf

Deadline: 01.07.2026

Duration: x months

Erklärung

Ich versichere, dass ich die vorliegende Arbeit ohne fremde Hilfe und ohne Benutzung anderer als der angegebenen Quellen angefertigt habe und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen hat und von dieser als Teil einer Prüfungsleistung angenommen wurde. Alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, sind als solche gekennzeichnet.

Erlangen, den 15.04.2026

Georg Meyer

Table of Contents

Erklärung	3
List of Figures	4
List of Tables	6
List of Abbreviations	7
List of Symbols	8
1 Introduction	1
1.1 Motivation and Context	1
1.2 Problem Statement and Objectives	1
1.3 Thesis Outline	2
2 Background	3
2.1 Reinforcement Learning for Robotic Manipulation	3
2.1.1 Learning objectives and algorithm families	4
2.1.2 Exploration and data collection in manipulation	5
2.1.3 Simulation-to-reality considerations	5
2.1.4 Proximal Policy Optimization (PPO)	5
2.2 Tendon-Driven Mechanisms and Linkage Kinematics	6
2.2.1 Tendon-Driven Actuation Principles	6
2.2.2 Planar Four-Bar Linkages and the Antiparallelogram	8
2.2.3 Cable-Driven Parallel Mechanisms	9
2.2.4 Jacobian-Transpose Force Mapping	11
2.2.5 Tensegrity Principles in Robot Design	12
2.3 Workspace Analysis and Dexterity Metrics	14
2.3.1 Forward Kinematics and Sampling Motivation	14
2.3.2 Monte Carlo Workspace Sampling and Voxel Aggregation	14
2.3.3 Dexterity Metrics	15
2.3.4 Implications for Task-Space Design in Learning-Based Manipulation	15
2.4 Physics Simulation	16
2.4.1 Rigid-Body Dynamics and Articulations	16
2.4.2 Contact and Friction Models	17
2.4.3 Simulation Parameters	17
2.4.4 GPU-Accelerated Simulation for Parallel RL Environments	18
2.5 NVIDIA Isaac Sim and IsaacLab	18
2.5.1 Isaac Sim in the Omniverse Stack	18
2.5.2 IsaacLab as a Robot Learning Framework	19
2.5.3 Comparison to Alternative Simulators	20
2.6 Robot and Scene Description Formats	20
2.6.1 USD as Scene Graph	20
2.6.2 URDF as Robot Model Format	21
2.6.3 Format Pipeline and Tendon Workarounds	21
3 State of the Art and Research Gap	23
3.1 GPU-Accelerated Reinforcement Learning for Manipulation	23
3.2 Tendon- and Cable-Driven Robot Simulation	24

3.3	Tensegrity-Inspired and Linkage-Based Robot Design	24
3.4	Identified Research Gap	25
3.5	Success Criteria	25
4	Methodology	27
4.1	Overview of Approach	27
4.2	Robot Design	28
4.2.1	Kinematic Chain and Degrees of Freedom	28
4.2.2	Antiparallelogram Elbow Mechanism	31
4.2.3	Cable-Driven Wrist Mechanism	32
4.2.4	Drive System and Tendon Routing	33
4.3	Simulation Model Construction	35
4.3.1	Mesh Processing Pipeline	35
4.3.2	Articulation and Physics Configuration	36
4.3.3	Disc Approximation Variant	36
4.3.4	Physical Antiparallelogram Linkage Variant	38
4.3.5	Five-DoF Manipulator Assembly	39
4.4	Tendon Actuation in Simulation	40
4.4.1	Jacobian-Transpose Torque Mapping	41
4.4.2	Physical Body-Force Tendon Model	42
4.4.3	Actuation Mode Comparison	43
4.4.4	Motor Specifications and Simulation Parameters	44
4.5	Model Validation	44
4.5.1	Step-Response Testing Protocol	44
4.5.2	PD Gain Tuning	46
4.5.3	Monte Carlo Workspace Analysis	46
4.6	Reinforcement Learning Task Formulation	47
4.6.1	Simulation Environment	47
4.6.2	Reach Task	47
4.6.3	Cube Place Task	49
4.6.4	Cube Sort Task (Outlook)	50
4.7	Training Infrastructure	50
4.7.1	PPO Configuration	50
4.7.2	Simulation Parameters	51
4.7.3	Software and Hardware	51
5	Results	52
5.1	Model Validation Results	52
5.1.1	Step-Response Analysis	52
5.1.2	Workspace Analysis	56
5.2	Reach Task Results	58
5.2.1	Training Convergence	58
5.2.2	Final Performance	59
5.2.3	Cross-Variant Comparison	59
5.3	Cube Place Results	60
5.3.1	Training Convergence	60
5.3.2	Final Performance	60

5.3.3 Curriculum Effect	61
5.4 Cube Sort Preliminary Results	62
5.5 Summary of Key Results	62
6 Discussion	64
6.1 Evaluation of Outcomes	64
6.2 Comparison with Related Work	64
6.3 Limitations of the Study	64
6.4 Practical Implications	64
6.5 Recommendations and Lessons Learned	64
7 Conclusion	65
7.1 Conclusion	65
7.2 Outlook	65
Bibliography	66
A Appendix: Simulation and Control Parameters	76
A.1 Joint Drive Parameters	76
A.2 Step-Response PID Parameters	76
B Appendix: PPO Hyperparameters	77
C Appendix: Cube Place Reward Terms	78
D Appendix: Reach Task — Complete Training Metrics	80
D.1 Reward Overview	80
D.2 Task Reward Components	80
D.3 Regularization and Training Diagnostics	81
E Appendix: Cube Place Task — Complete Training Metrics	84
E.1 Reward Overview	84
E.2 Task Metrics	84
E.3 Task Reward Components — Manipulation Pipeline	85
E.4 Task Reward Components — Placement and Target	86
E.5 Regularization and Training Diagnostics	87
F Appendix: Cube Sort Task — Preliminary Training Metrics	90
F.1 Reward Overview	90
F.2 Task Metrics	90
F.3 Reward Components	91
F.4 Training Diagnostics	92
G Appendix: Training Monitor	94

List of Figures

Figure 1: RL agent-environment interaction loop	3
Figure 2: Antagonistic tendon actuation at a revolute joint	6
Figure 3: Feasible torque–stiffness region of an antagonistic tendon pair	7
Figure 4: Parallelogram vs. antiparallelogram four-bar linkage topologies	9
Figure 5: Cable-driven parallel mechanism concept with three cables	10
Figure 6: Tensegrity class-1 and class-2 structure classification	12
Figure 7: Isaac Sim physics simulation stack	16
Figure 8: Isaac Sim and IsaacLab software stack	18
Figure 9: Overview of the experimental pipeline	27
Figure 10: Kinematic topology of the tensegrity manipulator	28
Figure 11: Robot model variants rendered in Isaac Sim	29
Figure 12: Five-pose overview of the antiparallelogram elbow linkage	31
Figure 13: 3D overview of the cable-driven wrist mechanism with workspace cone	33
Figure 14: Mesh processing pipeline from CAD to Isaac Lab asset	35
Figure 15: Joint visualization of the two arm variants	36
Figure 16: Antiparallelogram vs. disc approximation kinematic comparison	37
Figure 17: Collision geometry comparison of arm variants	39
Figure 18: Prismatic base joints and Robotiq 2F-140 gripper	40
Figure 19: Tendon actuation pipeline data flow	41
Figure 20: Sequential reward pipeline for the cube place task	49
Figure 21: Step-response time traces for elbow and wrist joints	52
Figure 22: NRMSE comparison across actuation modes	54
Figure 23: Settling time comparison	54
Figure 24: PD damping sweep for elbow joint	55
Figure 25: Workspace quality metrics (density, manipulability, condition)	56

Figure 26: Reach task training convergence	58
Figure 27: Cross-variant reach task comparison	59
Figure 28: Cube place training convergence	60
Figure 29: Curriculum stages effect on training	61
Figure 30: Preliminary cube sort training results	62
Figure 31: Reach task reward overview	80
Figure 32: Reach task reward components	80
Figure 33: Reach task diagnostics	81
Figure 34: Reach task additional diagnostics	82
Figure 35: Cube place reward overview	84
Figure 36: Cube place task metrics	84
Figure 37: Cube place manipulation rewards	85
Figure 38: Cube place placement rewards	86
Figure 39: Cube place regularization terms	87
Figure 40: Cube place training diagnostics	88
Figure 41: Cube sort reward overview	90
Figure 42: Cube sort task metrics	90
Figure 43: Cube sort reward components	91
Figure 44: Cube sort training diagnostics	92
Figure 45: Terminal monitor — overview	94
Figure 46: Terminal monitor — reward charts	95
Figure 47: Terminal monitor — policy diagnostics	96
Figure 48: Web-based training monitor dashboard	97

List of Tables

Table 1: Key rigid-body simulation parameters	17
Table 2: Qualitative simulator comparison	20
Table 3: USD and URDF roles in simulation	21
Table 4: Link dimensions and masses of the tendon-driven arm	30
Table 5: Actuated joints of the tensegrity manipulator	34
Table 6: Robot USD variant matrix	39
Table 7: Comparison of the three actuation modes	43
Table 8: Motor and actuation parameters	44
Table 9: Step-response test matrix	45
Table 10: Shared scene elements	47
Table 11: Reach task reward terms	48
Table 12: Cube place task reward terms	49
Table 13: PPO hyperparameter summary	50
Table 14: Simulation parameters per task	51
Table 15: NRMSE comparison across actuation modes	52
Table 16: Physical-linkage tendon step-response metrics	53
Table 17: Base joint step-response metrics	55
Table 18: Workspace analysis statistics	56
Table 19: Reach task final performance metrics	59
Table 20: Cube place final performance metrics	60
Table 21: Tensegrity joint drive parameters	76
Table 22: Step-response PID gains	76
Table 23: PPO hyperparameter comparison	77
Table 24: Cube place reward specification	78

List of Abbreviations

API	Application Programming Interface
CAD	Computer-Aided Design
CDPM	Cable-Driven Parallel Mechanism
CPU	Central Processing Unit
DoF	Degree of Freedom
FK	Forward Kinematics
GAE	Generalised Advantage Estimation
GPU	Graphics Processing Unit
ICR	Instantaneous Center of Rotation
MDP	Markov Decision Process
NRMSE	Normalized Root-Mean-Square Error
PD	Proportional-Derivative
PGS	Projected Gauss–Seidel
PID	Proportional-Integral-Derivative
PPO	Proximal Policy Optimization
RL	Reinforcement Learning
TGS	Temporal Gauss–Seidel
TUI	Text-based User Interface
URDF	Unified Robot Description Format
USD	Universal Scene Description
XML	Extensible Markup Language
XPBD	Extended Position-Based Dynamics

List of Symbols

$\mathcal{A}$	Action space
$\hat{A}_t$	Advantage estimator
$J(\theta)$	Expected discounted return
$\mathbf{J}(\mathbf{q})$	Geometric Jacobian
$\mathbf{J}^\top$	Tendon Jacobian transpose
$\mathcal{S}$	State space
$\mathbf{T}$	Tendon tension vector
ε	Clipping parameter (PPO)
γ	Discount factor
h	Simulation time step
κ_{inv}	Inverse condition number
π_θ	Policy (parameterized by theta)
$r(s_t, a_t)$	Reward function
$\boldsymbol{\tau}$	Joint torque vector
θ	Policy parameters
$w(\mathbf{q})$	Yoshikawa manipulability

1 Introduction

1.1 Motivation and Context

Tendon-driven robots are articulated mechanisms in which joint motion is generated by tensile elements that transmit forces from remotely placed actuators to the joints. This design reduces distal mass and inertia, enables compact joint packaging, and introduces intrinsic mechanical compliance that can be beneficial for contact-rich manipulation tasks [2][47][81]. At the same time, the cable-based transmission introduces modeling complexity: tendon routing geometry, elasticity, friction, and hysteresis effects create a non-trivial mapping between actuator commands and joint motion that is difficult to capture with classical analytical models alone [29][42].

Reinforcement Learning (RL) has emerged as a promising paradigm for learning robot control policies directly from interaction data, reducing the reliance on explicit analytical models and hand-crafted control strategies [32][95]. In the context of robotic manipulation, RL has demonstrated the ability to discover effective policies for tasks involving continuous, high-dimensional control, contact-rich dynamics, and sparse feedback [32]. Modern physics simulators with Graphics Processing Unit (GPU)-accelerated parallel execution have become central enablers of this approach: by running thousands of environment replicas simultaneously, they provide the sample throughput required for stable policy learning while avoiding wear, risk, and slow data collection on physical hardware [36].

NVIDIA Isaac Sim and the associated IsaacLab framework provide an integrated toolchain for this simulation-driven development. Isaac Sim offers high-fidelity physics simulation based on PhysX 5 with articulated rigid-body dynamics, contact modeling, and GPU-accelerated pipelines within a Universal Scene Description (USD)-based scene representation [61]. IsaacLab builds on top of Isaac Sim and provides a unified, modular interface for designing robotics tasks as Markov Decision Processes (MDPs) and training policies at scale [68]. Together, they enable a workflow in which robot models are constructed, validated, and used for policy training within a single, reproducible environment.

This thesis is the first part of a two-part project conducted at the Institute for Factory Automation and Production Systems (FAPS) at Friedrich-Alexander-Universität Erlangen-Nürnberg. The broader project goal is to develop a multi-agent robotic system for automated textile sorting from mixed waste streams, where a tendon-driven robot picks clothing items from a conveyor and a second robot arm stretches, inspects, and sorts them by condition. The present work establishes the simulation foundation and validates the tendon-driven robot model through basic manipulation tasks (reaching and placing rigid objects), providing the verified platform on which subsequent work will build.

1.2 Problem Statement and Objectives

Deploying a custom tendon-driven manipulator for conveyor-based pick-and-place tasks requires a faithful simulation model that accurately represents the robot's kinematics, actuation characteristics, and workspace geometry. Without such a validated model, RL policies trained

in simulation may learn to exploit simulator-specific artifacts rather than developing behaviors that transfer to the physical system [72][86].

The specific robot considered in this thesis is a 5-Degree of Freedom (DoF) manipulator consisting of a 2-DoF prismatic positioning stage (providing linear motion in the y - and z -directions), a 3-DoF tendon-driven arm (one proximal bending joint and a 2-DoF cable-driven wrist), and a Robotiq 2F-140 parallel gripper as the end-effector. The design builds on the tendon-driven robot concept described by Klein [30]. Integrating this robot into a simulation-based RL pipeline requires addressing several interconnected challenges: constructing a modular simulation scene, faithfully representing the tendon-driven actuation mechanism, validating the simulated robot against established benchmarks, and formulating RL tasks that progressively build manipulation competence from basic skills to task-level behavior.

A further question addressed in subsequent work (master thesis) is whether the compliant tendon-driven wrist mechanism can be transferred to a standard industrial robot arm and whether such a transfer yields measurable benefits in manipulation performance.

The thesis pursues the following objectives:

1. **Validated tendon-driven robot model in simulation.** Construct and validate a modular simulation model of the 5-DoF tendon-driven manipulator in Isaac Sim that faithfully represents kinematics, actuation dynamics, and workspace geometry, such that it can serve as a reliable platform for RL-based policy training.
2. **Progressive RL-based manipulation skill acquisition.** Develop and evaluate RL policies for manipulation tasks of increasing complexity comparing Proportional-Derivative (PD)-driven and tendon-driven actuation variants to quantify how actuation modeling influences learning performance.
3. **Foundation for deformable-object manipulation.** Establish a documented, reproducible simulation infrastructure that forms the basis for subsequent work on deformable cloth handling, multi-agent textile sorting, and sim-to-real transfer in the master thesis.

1.3 Thesis Outline

This thesis is structured as follows. Section 2 introduces the fundamentals required for this work, covering RL for robotic manipulation with an emphasis on Proximal Policy Optimization (PPO) [88], rigid-body physics simulation in PhysX 5, the NVIDIA Isaac Sim and IsaacLab architecture [68], and tendon-driven robot kinematics and modeling. Section 3 reviews the state of the art in GPU-accelerated RL for manipulation and tendon-driven robot simulation, and identifies the specific research gap addressed by this thesis.

Section 4 describes the methodological approach, including the simulation scene design, the modular robot model assembly, the tendon actuation implementation, the validation procedures (step-response analysis and workspace characterization), and the MDP formulations for the reach and cube place tasks using `skrl` PPO [88][89]. Section 5 presents the quantitative and qualitative results: robot validation metrics and learned policy performance for both actuation variants across both tasks.

Section 6 discusses the implications and limitations of the approach, including simulation fidelity considerations and the effect of tendon-driven versus PD-driven actuation on policy

learning. Section 7 concludes the thesis and outlines the path toward the master thesis, where the validated simulation platform will be extended with deformable cloth simulation, multi-agent coordination, and sim-to-real transfer preparation.

2 Background

2.1 Reinforcement Learning for Robotic Manipulation

Robotic manipulation requires generating control commands that achieve a goal while dealing with uncertainty, partial observability, contacts, and high-dimensional configuration spaces. Classical approaches rely on analytical models, trajectory optimisation, and carefully engineered feedback control. However, accurate models of frictional contact, deformable objects, or cluttered interactions are often difficult to obtain, and manually designing robust policies can be time-consuming. RL provides an alternative: it enables learning control policies from interaction data by optimising a task objective, thereby reducing reliance on explicit modelling and hand-crafted strategies. General RL background and formalism are treated in standard texts such as Sutton and Barto [95], and robotics-specific perspectives (including practical constraints and common failure modes) are discussed in Kober *et al.* [32].

MDP viewpoint and partial observability

A manipulation task can be modelled as a MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $P(s_{t+1} | s_t, a_t)$ the transition dynamics, $r(s_t, a_t)$ the reward, and $\gamma \in [0, 1)$ the discount factor. A policy $\pi_\theta(a | s)$ (parameterized by θ) induces trajectories $(s_0, a_0, r_0, \dots, s_T)$ and is trained to maximize the expected discounted return $J(\theta) = \mathbb{E}_{\pi_\theta} [\sum_{t=0}^T \gamma^t r_t]$. [95]

In real manipulation systems, the full state s_t is typically not directly accessible. Instead, control is based on observations o_t (e.g., proprioception, gripper state, pose estimates, perception features). This partial observability motivates careful observation design and, where appropriate, augmenting policies with memory (e.g., recurrent networks), or stacking observation histories to approximate the Markov property in practice. [32] Figure 1 illustrates this interaction loop with the manipulation-specific action, observation, and reward structure used in this thesis.

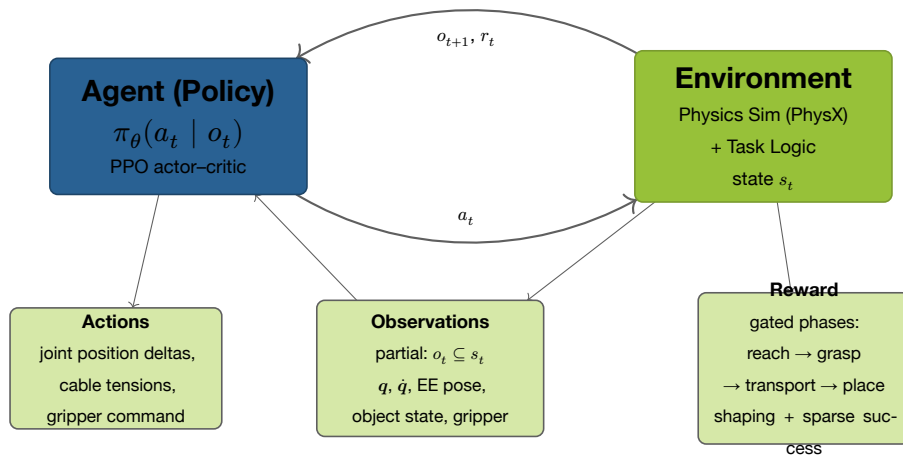


Figure 1: The reinforcement learning interaction loop: the agent selects action a_t from policy π , the environment transitions and returns observation o_{t+1} and reward r_t .

Key challenges in manipulation

Three aspects commonly make RL for manipulation demanding:

- **continuous, high-dimensional control:** Actions may correspond to joint torques, velocities, or Cartesian increments for many degrees of freedom.
- **contact-rich and nonlinear dynamics:** Small modelling errors in friction or contact impulses can lead to qualitatively different outcomes.
- **sparse or delayed feedback:** Success may only be measurable after long sequences of intermediate steps.

Further practical constraints include safety, limited real-world interaction budgets, and the need for robustness across object variations and perception noise. These characteristics influence algorithm choice, reward design, termination conditions, and the role of simulation during training. [32]

Reward design and task specification

In RL, the reward function encodes the task objective. For manipulation, rewards are often composed of shaping terms (e.g., distance-to-target, alignment, grasp quality proxies) and sparse success signals (e.g., “object lifted above height”). Well-shaped rewards can accelerate learning but risk biasing the learned behavior toward unintended shortcuts. Conversely, sparse rewards better reflect the true objective but can make exploration difficult. Robotics RL workflows therefore often iterate on reward definitions and termination criteria to balance learnability and task fidelity. [32][95]

Sequential reward gating for multi-phase tasks

Multi-step manipulation tasks such as pick-and-place naturally decompose into sequential phases (reach, grasp, transport, place), each requiring distinct behaviors. Ng *et al.* [48] proved that adding a *potential-based* shaping reward $F(s, a, s') = \gamma\Phi(s') - \Phi(s)$ preserves all optimal policies of the original MDP, which provides theoretical justification for adding phase-dependent intermediate rewards. Devlin and Kudenko [8] extended this result to *dynamic* potentials that depend on time-varying state augmentations (e.g., persistent latch flags), showing that optimality is preserved even when the potential changes during an episode. In practice, Popov *et al.* [76] demonstrated the effectiveness of *composite shaping rewards* for Lego brick stacking, where each reward component is gated by a task-phase predicate (e.g., transport reward activates only after a “grasped” condition is met). This gated activation pattern is equivalent to a two-state *reward machine* in the formalism of Icarte *et al.* [99], where finite-state automata define per-phase reward functions driven by propositional events. Benchmark environments such as robosuite [108] implement this principle through explicit `staged_rewards()` methods with sequentially gated reach, grasp, lift, and hover components.

2.1.1 Learning objectives and algorithm families

For continuous control, modern deep RL methods commonly use *policy gradient* and *actor-critic* approaches. Policy gradient methods directly optimise the policy parameters by ascending an estimate of $\nabla_{\theta} J(\theta)$, which naturally supports continuous actions through differentiable policy parameterizations. Actor-critic methods introduce a learned critic (value function) that provides lower-variance learning signals for the actor, often improving stability and learning speed in practice. A central quantity is the advantage function $A^{\pi}(s_t, a_t)$, which measures how much better an action is compared to the policy’s baseline behaviour at a

state. In practice, advantage estimates $\hat{A}_t$ are computed from value predictions and collected rollouts, and then used to weight policy updates. [95]

Bias–variance trade-off in advantage estimation

A practical challenge is balancing low-variance learning signals with accurate credit assignment over time. Generalised Advantage Estimation (GAE) provides a widely used mechanism to interpolate between Monte Carlo returns and bootstrapped temporal-difference estimates by exponentially weighting temporal-difference residuals. This improves learning stability in many continuous-control applications and is commonly paired with actor–critic training. [87]

2.1.2 Exploration and data collection in manipulation

A defining challenge in RL is the *exploration–exploitation trade-off*: the agent must explore sufficiently to discover high-reward behaviours while exploiting learned knowledge to improve performance [95]. In manipulation, exploration is complicated by long horizons, sparse success signals, and irreversible or unsafe states (in real systems) [32]. Common practical measures include:

- **Stochastic policies and action sampling**, which induce exploratory behavior while still allowing gradient-based optimization [95]. In manipulation, *hybrid action spaces* combining continuous arm control with simplified (binary or low-dimensional) gripper commands are common, as they reduce the exploration burden on the gripper dimension while retaining continuous dexterity for arm motions [28][108].
- **Reward shaping and curricula**, where training starts from simplified conditions (e.g., easier initial object poses) and progressively increases difficulty to maintain a learnable signal [32].
- **Episode resets and termination design**, which affect state visitation and can substantially change the data distribution seen by the learner [32].

These aspects tie the learning algorithm closely to environment design: even a robust optimiser can fail if exploration is insufficient or if the reward/termination specification induces pathological behavior [32][95].

2.1.3 Simulation-to-reality considerations

RL policies are frequently trained in simulation to avoid wear, risk, and slow data collection on real robots [32]. However, transferring policies to the real world is challenging due to the *reality gap*: discrepancies in geometry, contact parameters, actuator dynamics, sensor noise, and unmodeled effects can degrade performance. Common strategies include **domain randomization**, where visual and physical parameters are randomized during training so that the policy learns behaviors that are robust across a range of plausible conditions [97], and **dynamics randomization**, which targets uncertainties in physical parameters (e.g., mass, friction, motor gains, delays) to improve transfer for control policies [73].

2.1.4 Proximal Policy Optimization (PPO)

Among actor–critic methods, PPO is a popular choice due to its robustness and simplicity. PPO performs iterative policy improvement by collecting trajectories with the current policy and updating the policy with a surrogate objective that discourages overly large steps in policy space. Define the importance ratio $r_t(\theta) = \pi_\theta \frac{a_t | s_t}{\pi_{\theta_{\text{old}}}}(a_t | s_t)$. PPO maximizes the clipped objective

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E} \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t) \right] \quad (1)$$

where $\hat{A}_t$ is an advantage estimate (commonly computed with GAE). Intuitively, clipping limits the incentive to change the policy too aggressively, which helps prevent performance collapse from unstable updates. This is a practical consideration in contact-rich manipulation where learning can be sensitive to abrupt policy changes. [87] [32][88]

From a robotics perspective, PPO has several properties that make it attractive in manipulation pipelines:

- **Stable learning dynamics:** The clipped objective yields conservative and typically reliable updates [88].
- **Compatibility with continuous actions:** PPO naturally supports Gaussian policies over continuous controls [95].
- **Practical training loop:** The rollout-and-update structure integrates well with simulator-based training and supports systematic iteration on reward and termination design [32].

These considerations motivate PPO as a strong baseline for manipulation, especially when training stability and predictable iteration cycles are prioritized.

2.2 Tendon-Driven Mechanisms and Linkage Kinematics

This section introduces the mechanical and kinematic foundations of the robot used in this thesis. The manipulator combines a tendon-driven actuation paradigm with a linkage-based elbow joint and a cable-driven wrist, all motivated by the structural principles of tensegrity. Understanding these concepts is necessary for interpreting the robot model (Section 4.2), the tendon actuation modes (Section 4.4), and the simulation choices made in the methodology.

2.2.1 Tendon-Driven Actuation Principles

In tendon-driven robots, joint motion is produced by tensile elements (tendons or cables) that transmit forces from remotely placed actuators to the joints through pulleys, guides, or routing channels. The actuators can therefore be positioned proximally, keeping the distal structure lightweight and reducing its inertia. This actuation paradigm is widely used in cable-driven mechanisms, robotic hands, and bio-inspired designs, particularly where low distal inertia, compact joint packaging, or intrinsic compliance is desired. [47][81]

Antagonistic arrangements

Because cables can only pull, a single tendon cannot produce bidirectional joint torque. Joints are therefore actuated by *antagonistic* tendon pairs (or groups), as shown in Figure 2. Differential tension produces a net joint torque, while simultaneous co-contraction modulates apparent joint stiffness without changing the joint angle [31][43]. The resulting feasible torque–stiffness space is illustrated in Figure 3. For an n -DoF system, Lee and Tsai [101] established that $n + 1$ cables are the minimum required for full force closure (bidirectional torque on all joints with positive cable tensions), while additional cables beyond $n + 1$ provide redundancy that can be exploited for stiffness modulation and tension optimization [31][101].

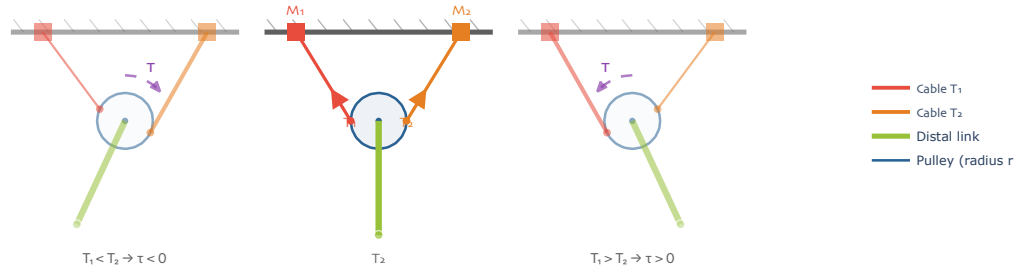


Figure 2: Antagonistic tendon actuation at a single revolute joint shown in three configurations. Two cables T_1 (red) and T_2 (orange) are routed from proximal motors M_1 and M_2 on the fixed frame through a pulley of radius r at the joint centre. Both cables attach to opposite sides of the pulley rim, with the colored markers indicating the attachment points. In the neutral configuration (centre), equal tensions produce zero net torque. Differential tension produces a net torque $\tau = r(T_1 - T_2)$: when $T_1 < T_2$ (left) the joint deflects to negative angles, and when $T_1 > T_2$ (right) to positive angles. Simultaneous co-contraction ($T_1 + T_2$) modulates apparent joint stiffness κ independently of position, as shown quantitatively in Figure 3.

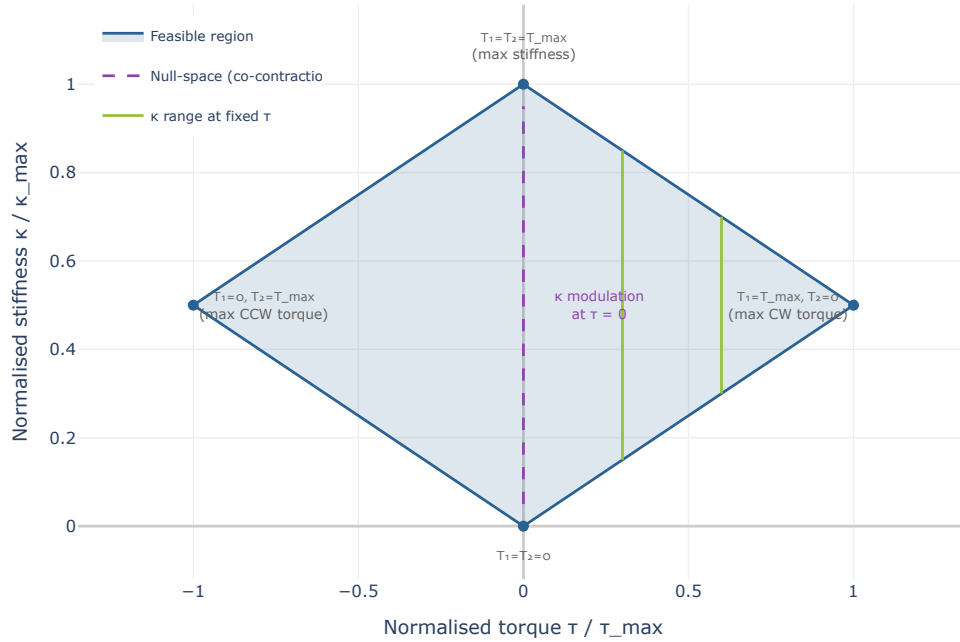


Figure 3: Feasible torque–stiffness space for an antagonistic tendon pair with tensions bounded by $0 \leq T_i \leq T_{\max}$. The diamond-shaped region shows all achievable combinations of normalised net torque $\tau/\tau_{\max}$ and normalised apparent stiffness $\kappa/\kappa_{\max}$. The vertices correspond to the extremal tension combinations: maximum co-contraction at the top ($T_1 = T_2 = T_{\max}$, maximum stiffness), zero tension at the bottom, and maximum unilateral torque on the sides. The dashed purple line indicates the null-space direction (co-contraction at zero torque), along which stiffness can be modulated without affecting the joint torque. The green lines illustrate the available stiffness range at two fixed torque values, demonstrating that stiffness modulation decreases as the demanded torque increases.

Transmission non-idealities

The mechanical transmission through cables introduces modeling complexity beyond direct-drive systems. Tendon stretch, cable–pulley compliance, and frictional effects cause configuration-dependent motion errors, phase lags, and hysteresis loops under load. These effects break the assumption of a unique, memoryless mapping between actuator displacement and end-effector pose. As a result, tendon-driven systems are often controlled using local joint sensing to close the loop in joint space, or model-based compensation to improve the actuator-to-joint mapping. [29][42]

2.2.2 Planar Four-Bar Linkages and the Antiparallelogram

The elbow joint of the tensegrity manipulator uses an antiparallelogram four-bar linkage. This subsection introduces the relevant linkage theory. The specific kinematic closure equations for the elbow are derived in Section 4.2.2 of the methodology.

Four-bar linkage fundamentals

A planar four-bar linkage consists of four rigid bars connected by four revolute joints, forming a single closed loop with one degree of freedom [40][102]. One bar is conventionally designated as the *frame* (grounded link). The remaining links are the *input crank*, *coupler*, and *output rocker*. The Grashof condition classifies four-bar mechanisms by the range of motion of each link: if the sum of the shortest and longest link lengths does not exceed the sum of the other two, at least one link can rotate fully, yielding crank–rocker, double-crank, or double-rocker configurations [102]. The configuration is fully determined by a single input angle through the *loop-closure equation*, which can be expressed in complex-number form as

$$Z_1 e^{i\theta} + Z_2 e^{i\beta} = Z_3 e^{i\varphi} + Z_4 \quad (2)$$

where Z_i denote the complex link vectors, θ is the input angle, φ the output angle, and β the coupler angle [40]. The *transmission angle* (the angle between the coupler and the output link) characterizes force transmission quality. Values near 90° maximize force transfer, while values approaching 0° or 180° indicate near-singular configurations [22][102].

Antiparallelogram topology

The antiparallelogram is a special case of the four-bar linkage in which the two side links (cranks) are of equal length and *cross* each other, while the frame and coupler share a common length [10][40]. This crossed topology produces qualitatively different kinematics from the more common parallelogram arrangement. Figure 4 contrasts both topologies and visualises the swept workspace and Instantaneous Center of Rotation (ICR) behaviour.

Centrode geometry and ICR migration

The instantaneous center of rotation (ICR) of the coupler relative to the frame traces a curve called the *fixed centrode*. For a standard revolute joint, the ICR is fixed at the joint axis. For the antiparallelogram, Dijksman [10] showed that the fixed centrode is an *ellipse*, meaning the ICR migrates continuously during motion. Stachel [93] derived the ellipse semi-axes as β and $\sqrt{\beta^2 - \alpha^2}$ (where α and β are determined by the link lengths), and Bryant and Sangwin [5] provide a geometric derivation of this elliptical trace. This migrating ICR produces a *nonlinear, configuration-dependent transmission ratio*. The effective lever arm between the tendon attachment point and the output rotation axis changes continuously with the elbow angle, complicating the force–torque relationship.

Implications for tendon actuation

The variable transmission ratio has direct consequences for tendon-driven actuation. As the elbow moves away from the zero configuration, the effective moment arm of the tendon attachment changes, meaning the same cable tension produces different joint torques at different configurations [14]. A common engineering simplification is the *disc approximation*, which replaces the crossed linkage with a simple revolute joint having a constant lever arm. This approximation is valid for small angular deflections but introduces increasing error at larger angles, where the migrating ICR significantly alters the transmission geometry.

Loop-closure enforcement

The antiparallelogram can bifurcate into a parallelogram configuration at an instability point where the two side links are collinear. In physical mechanisms, cables or mechanical stops prevent this mode-switching by constraining the linkage to the crossed configuration [14].

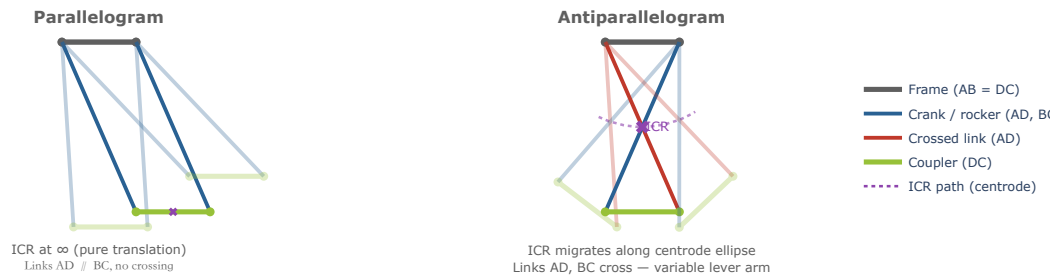


Figure 4: Comparison of the two four-bar linkage topologies relevant to tendon-driven manipulators. Left: the standard parallelogram, where both side links (blue) remain parallel throughout motion and the instantaneous centre of rotation (ICR, purple cross) lies at infinity, producing pure translational coupler motion. Right: the antiparallelogram, where the side links cross each other. The ICR migrates along a centrode ellipse (purple dashed), creating a configuration-dependent lever arm. This nonlinear transmission ratio distinguishes the antiparallelogram from a simple revolute joint and requires either the physical linkage model or a disc approximation for torque computation. Multiple ghost poses at different input angles illustrate the swept workspace of each topology.

2.2.3 Cable-Driven Parallel Mechanisms

The wrist mechanism of the tensegrity manipulator is a cable-driven parallel mechanism where three active cables, arranged symmetrically around a universal joint, control two rotational degrees of freedom. This subsection introduces the general framework. The specific structure matrix for the thesis robot's wrist is derived in Section 4.2.3 of the methodology.

Concept

A cable-driven parallel mechanism (Cable-Driven Parallel Mechanism (CDPM)) consists of a platform (end-effector) connected to a fixed base frame by m cables, where platform motion is controlled by adjusting cable lengths [77]. Unlike rigid-link parallel robots, cables can only transmit tensile forces, which fundamentally constrains the feasible wrench (forces and/or torques) space [77][104]. Figure 5 illustrates this arrangement for the three-cable, two-DoF configuration used in the wrist. The field originated with Landsberger's 1984 wire-driven par-

allel robot and has been substantially developed by Verhoeven [104], Pott [77], and Gosselin among others.

Structure matrix formalism

The static relationship between cable tensions and the platform wrench is described by the *structure matrix* $A^\top$. Following the notation established by Verhoeven [104] and formalized by Bruckmann *et al.* [4], the equilibrium condition is

$$w = A^\top t, \quad (3)$$

where $w \in \mathbb{R}^n$ is the desired platform wrench, $t \in \mathbb{R}^m$ is the cable tension vector, and $A^\top \in \mathbb{R}^{n \times m}$ is the structure matrix whose columns are determined by the cable direction vectors and attachment geometry [4][78]. The term “structure matrix” is due to Verhoeven [104], while Gosselin uses the equivalent term “wrench matrix” [16]. The structure matrix is the transpose of the Jacobian that maps platform velocities to cable length rates, establishing a kineto-static duality [7][45].

Redundant actuation and positive tensions

Since cables can only pull ($t \geq 0$), a minimum of $n + 1$ cables is required for an n -DoF platform to achieve *wrench closure* (the ability to generate arbitrary wrenches at a given pose) [16]. The *wrench-feasible workspace* is the more practical criterion, defined as the set of poses where all required wrenches can be sustained with tensions within prescribed bounds $T_{\min} \leq T_i \leq T_{\max}$, accounting for both minimum tension (preventing cable slack) and actuator saturation [3]. For a system with $m > n + 1$ cables, the null space of $A^\top$ provides freedom to adjust the tension distribution without affecting the net wrench. Pott *et al.* [78] developed a closed-form algorithm for computing feasible tension distributions in this redundant case, decomposing the solution as $t = t_{\text{particular}} + N\lambda$, where N spans the null space and λ is chosen to satisfy the tension bounds.

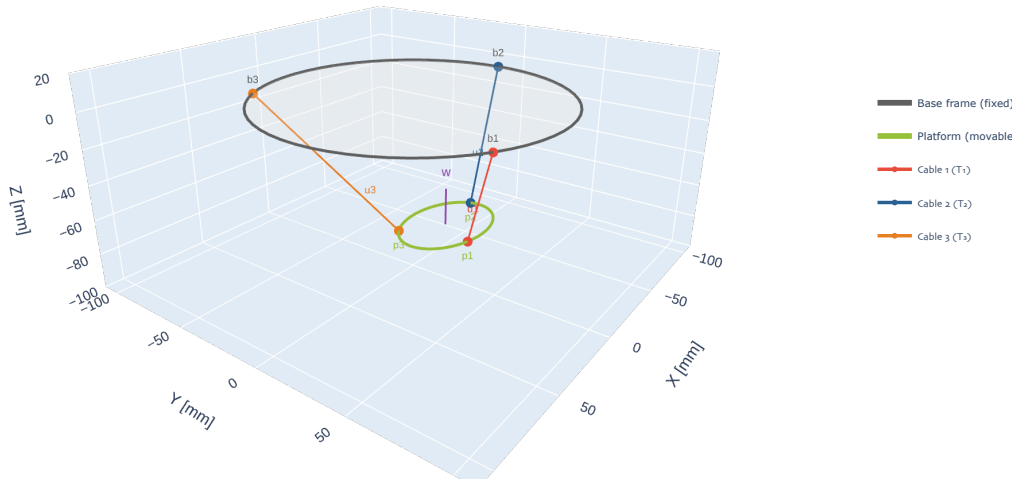


Figure 5: Three-dimensional schematic of a cable-driven parallel mechanism with $m = 3$ cables and $n = 2$ rotational degrees of freedom, illustrating the general CDPM concept underlying the wrist mechanism. Three cables (red, blue, orange) connect base frame anchors b_i at 120° intervals on the fixed ring to platform anchors p_i on the tilted movable ring. The platform is shown in a tilted configuration to visualise the cable arrangement. Each cable's unit direction vector u_i and the resultant platform wrench w are indicated. The static equilibrium is governed by the structure matrix: $w = A^T t$, where the one degree of redundancy ($m - n = 1$) enables tension optimisation under the positive-tension constraint $t \geq 0$.

2.2.4 Jacobian-Transpose Force Mapping

The fundamental relationship between cable tensions and joint torques in tendon-driven rigid-link robots is the Jacobian-transpose mapping. This formalism underlies all three actuation modes implemented in the methodology (Section 4.4).

Kineto-static duality

For a tendon-driven mechanism with joint coordinates $q \in \mathbb{R}^n$ and cable lengths $l \in \mathbb{R}^m$, the differential kinematics of the transmission are described by

$$\dot{l} = R(q)\dot{q}, \quad (4)$$

where $R(q) \in \mathbb{R}^{m \times n}$ is the transmission Jacobian (also called the routing matrix) [45]. By virtual work, the joint torques τ produced by cable tensions T are [7][45][100]

$$\tau = R^T(q)T. \quad (5)$$

This is the *Jacobian-transpose mapping*, establishing that the static force map is the transpose of the kinematic velocity map [7]. The formulation was introduced for tendon-driven hands by Salisbury and Craig [85] and generalized by Tsai and Lee [101].

Constant versus configuration-dependent Jacobian

A key distinction arises from the tendon routing geometry. Tsai and Lee [101] showed that when tendons wrap around circular pulleys of constant radius, the transmission Jacobian R is *constant* (i.e., independent of q): the entries are simply the pulley radii with appropriate signs.

This is valid because circular pulleys maintain a fixed moment arm regardless of joint angle. When tendon routing involves non-circular geometry, such as the antiparallelogram linkage's migrating ICR, the effective moment arm is configuration-dependent, and $\mathbf{R}(\mathbf{q})$ must be recomputed at each configuration [14][45]. A common engineering approximation linearizes $\mathbf{R}$ at the zero configuration ($\mathbf{q} = 0$) and treats the resulting constant matrix as valid over the operating range. This approximation is exact for circular-pulley routing and increasingly inaccurate as the actual routing geometry departs from the linearization point.

Positive-tension constraint

Since cables can only produce tensile forces, the feasible joint torque space is limited to the image of the positive orthant $\mathbf{T} \geq 0$ under $\mathbf{R}^\top$. For a system with m cables and n joints, the $n + 1$ rule states that at least $n + 1$ cables are necessary to generate torques in all directions [85][101]. Additional cables provide a null space that can be used for co-contraction (stiffness modulation) without changing the net torque [31].

2.2.5 Tensegrity Principles in Robot Design

The tensegrity concept provides the structural design rationale for the robot used in this thesis, whose name (tensegrity manipulator) reflects this heritage.

Definition and origins

The term Tensegrity is a combination of *tensile integrity* introduced by Buckminster Fuller. Tensegrity describes structures in which isolated compression members (struts, bars) are held in stable equilibrium by a continuous network of tension members (cables, springs). Skelton and de Oliveira provide the engineering definition and introduce the *class-k* taxonomy, illustrated in Figure 6. In a class- k tensegrity, up to k rigid bodies may be in contact through joints (class-1 means no struts touch). The concept originated with Kenneth Snelson's *X-Piece* sculpture (1948) and Fuller's subsequent patent work, and has since been developed into a mature engineering discipline spanning architecture, structural engineering, and robotics. [92][94]

Key properties

Tensegrity structures offer several properties attractive for robotic applications:

- **Inherent compliance and impact absorption:** The tensile network distributes and absorbs impact forces passively, providing safety benefits faster than any control loop can react [2][105].
- **High strength-to-weight ratio:** Compression members carry only axial loads (no bending), enabling lightweight design [92].
- **Controllable stiffness:** By adjusting the pretension in cables, stiffness can be modulated without dedicated variable-stiffness hardware [31][43].
- **Remote actuation:** Motors can be placed at the base, with force transmitted through cables to distal joints, minimizing moving mass [12][14].

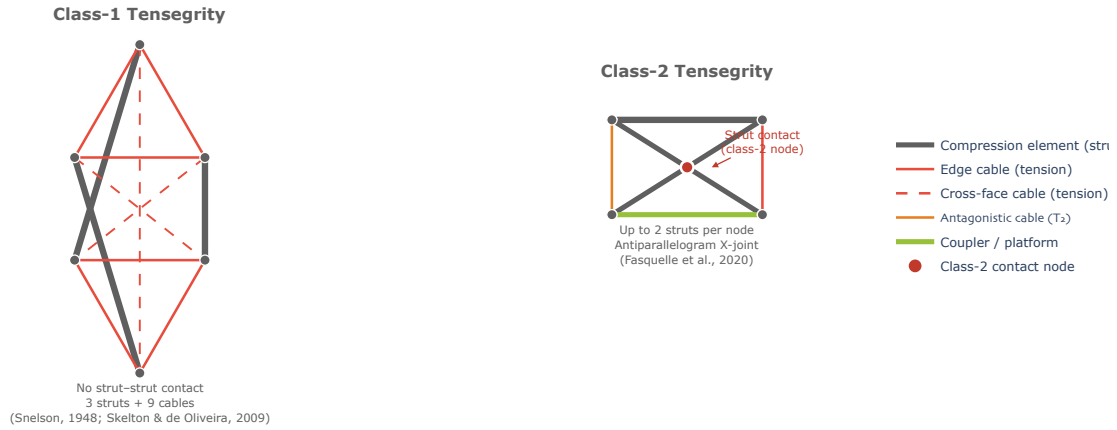


Figure 6: Tensegrity structure classification according to Skelton and de Oliveira. Left: a class-1 tensegrity prism (3 struts, 9 cables) where no compression elements are in direct contact — struts float within the continuous tension network of edge cables (solid red) and cross-face cables (dashed red). Right: a class-2 tensegrity module, where up to two struts may share a node (red dot). This topology corresponds to the antiparallelogram X-joint used in the elbow mechanism of the thesis robot, where the crossed struts form a planar linkage connected by cables and a coupler platform (green). The legend distinguishes compression elements (struts, grey), edge and cross-face tension elements (cables, red), antagonistic cables (orange), and the coupler link (green).

Tensegrity in robotics

Shah *et al.* position tensegrity as bridging rigid and soft robotics. The rigid compression elements provide structural load capacity while the elastic tension network provides compliance and deformability. Existing tensegrity robots span locomotion platforms (rolling and hopping), continuum manipulators (stacked-module assemblies), and serial manipulators with tensegrity-inspired joints. [90]

For manipulation specifically, Lessard *et al.* [33] demonstrated a bio-inspired tensegrity arm with structurally compliant joints, while Fasquelle *et al.* [12] proposed a 3-DoF manipulator using antiparallelogram X-joints — where each joint is a class-2 tensegrity mechanism with bars loaded only axially.

Tensegrity-inspired design in the thesis robot

The FAPS tensegrity manipulator [30] embodies the tensegrity principle through its cable-driven actuation: the rigid links (upper arm and forearm) act as compression elements, while the cables transmit all actuation forces as tension. The antiparallelogram elbow is a class-2 tensegrity mechanism whose crossed links carry compressive loads while cables maintain the kinematic constraint. The cable-driven wrist similarly distributes actuation through a tension network, with three cables suspending the wrist platform in a configuration directly analogous to a small-scale cable-driven parallel robot. Walter *et al.* demonstrated that this wrist architecture achieves up to 83% impact force isolation in hardware experiments, confirming the compliance benefits predicted by tensegrity theory for the wrist module. [105]

Compliance and safety context

Bicchi and Tonietti [2] established that mechanical compliance is essential for safe human–robot interaction, as passive compliance reacts faster than any control loop during fast impacts. The series elastic actuator concept introduces deliberate compliance in series with the actuator, trading bandwidth for shock tolerance and force control accuracy [79]. Variable impedance actuators formalize adjustable compliance through hardware mechanisms, but require complex dual-motor or nonlinear-spring designs [103]. Tensegrity provides an alternative path. The structural compliance inherent in the tension network delivers passive safety benefits analogous to series elastic or variable-stiffness actuators, without dedicated VSA hardware. Combined with the manipulator’s low distal inertia (motors at the base, lightweight carbon-fiber forearm), this aligns with Haddadin *et al.*’s finding that reducing robot weight is the primary requirement for safe collaborative operation [18].

2.3 Workspace Analysis and Dexterity Metrics

The workspace of a serial manipulator is the set of all end-effector poses that are reachable under joint limits and kinematic constraints. Workspace analysis is used to verify that a robot can reach task-relevant regions and to identify configurations that should be avoided due to singularities or poor motion quality. [11][82]

2.3.1 Forward Kinematics and Sampling Motivation

Forward Kinematics (FK) is defined as the mapping from a joint configuration $\mathbf{q}$ to an end-effector pose, i.e., position and orientation of a designated tool frame. For a general serial chain, this mapping can be written as

$$\mathbf{T}_{EE} = f(\mathbf{q}), \quad (6)$$

where $\mathbf{T}_{EE} \in SE(3)$ denotes a homogeneous transformation of the end-effector frame. In many practical systems, analytical derivation of workspace boundary surfaces is unsolvable due to high DoF, joint coupling, or non-standard kinematics. In such cases, the workspace is frequently approximated numerically by sampling joint space and evaluating FK repeatedly. The Monte Carlo approach is widely used because it is simple to implement, directly incorporates joint limits, and does not require explicit boundary derivation. [82]

2.3.2 Monte Carlo Workspace Sampling and Voxel Aggregation

In the Monte Carlo approach, configurations are sampled from an established distribution over the defined joint ranges. In the simplest implementation, each joint q_i is sampled independently and uniformly within its limit interval. For each sample $\mathbf{q}^{(k)}$, the corresponding end-effector pose is computed by FK. The resulting set of poses is then aggregated into a discrete spatial representation, e.g., Cartesian voxels, to estimate reachability statistics per region. This computation is conceptually aligned with established robotics software workflows that generate reachable workspace point sets paired with joint configurations [37].

When voxel aggregation is used, the workspace is partitioned into a regular grid of bins (voxels) in position and optionally orientation space. Each sample contributes to the occupancy count of the voxel containing the sampled pose. This discretization supports additional metrics beyond mere reachability, including density-based indicators of how many joint-space configurations map to a given region. [11]

2.3.3 Dexterity Metrics

Yoshikawa manipulability

Dexterity is commonly evaluated using Jacobian-based indices. Let $J(\mathbf{q})$ denote the geometric Jacobian that maps joint velocities to end-effector twist. The Yoshikawa manipulability measure is defined as

$$w(\mathbf{q}) = \sqrt{\det(J(\mathbf{q})J(\mathbf{q})^\top)}. \quad (7)$$

This scalar is proportional to the volume of the manipulability ellipsoid and becomes zero at kinematic singularities where the Jacobian loses rank. Higher values indicate that small joint motions can produce motion in a wider range of Cartesian directions, which is desirable for tasks requiring dexterous repositioning and local adjustments. [107]

Inverse condition number as an isotropy index

A complementary measure is derived from the Jacobian condition number. Salisbury and Craig introduced the use of Jacobian conditioning to identify well-conditioned (isotropic) operating points and to quantify error amplification through the kinematic mapping. Let $\sigma_{\min}$ and $\sigma_{\max}$ denote the minimum and maximum singular values of $J(\mathbf{q})$. The inverse condition number is defined as

$$\kappa_{\text{inv}}(\mathbf{q}) = \sigma_{\min} \frac{J(\mathbf{q})}{\sigma_{\max}} (J(\mathbf{q})). \quad (8)$$

This index lies in $[0, 1]$, approaches zero near singular configurations, and equals one at perfectly isotropic configurations (equal singular values), where motion/force transmission properties are directionally uniform. [85]

Reachability density

Reachability density is defined here as the number of sampled configurations whose end-effector poses fall within a given voxel. Under uniform joint sampling, this count is proportional to the joint-space volume that maps into the voxel region. Thus, higher density indicates that more distinct configurations can realize poses in that region. This concept is closely related to workspace density formulations that assign each voxel the number of reachable points (or a normalized probability density) and interpret higher density as higher positional/orientational reach richness and, in discretely actuated systems, higher achievable pose resolution. [11]

2.3.4 Implications for Task-Space Design in Learning-Based Manipulation

For manipulation tasks in cluttered scenes and for RL-based control, the workspace and dexterity metrics directly inform environment layout and action-space design. Task objects (e.g., pick targets, placement zones) are preferably positioned in regions with high reachability density and consistently high manipulability to reduce the probability that the robot is driven into near-singular configurations during exploration. Similarly, action bounds for end-effector motion primitives can be restricted to dexterous regions to improve learning stability and to reduce large corrective motions caused by kinematic ill-conditioning. [11][85][107]

When tensegrity compliance is present, these considerations become more important because poor kinematic conditioning can amplify transmission uncertainties into larger Cartesian errors, which in turn increases undesired contact forces during object interaction [2][42].

2.4 Physics Simulation

Physics-based simulation in Isaac Sim 5.1 is implemented as a layered stack that separates scene and asset description, binding of these descriptions to a concrete backend, and numerical solvers that evolve the physical state over time. In the Omniverse ecosystem, physics properties are authored in USD through the `UsdPhysics` schema (engine-agnostic) and `PhysXSchema` (backend-specific), allowing assets and environments to be defined declaratively in the scene graph. [62][74]

During simulation, the `omni.physx` runtime parses the USD stage, constructs the corresponding physics scene, advances the simulation, and writes back poses and state to USD [62]. Isaac Sim 5.1 uses NVIDIA PhysX 5 as its primary physics backend, providing rigid-body dynamics, articulations, contacts, and particle-based phenomena, with optional GPU-accelerated pipelines for throughput-critical workloads such as parallel RL training [54][63][64]. Figure 7 summarises this layered architecture.

Sim-to-real transfer is fundamentally limited by discrepancies between simulated and real dynamics (the *reality gap*), where unmodelled effects, incorrect parameters, and solver artefacts can lead to policies that exploit simulator-specific regularities [27][86]. In RL, the simulator acts as a data-generating process. Thus, modelling and solver choices directly shape the transition distribution used for learning, influencing both policy quality and generalization [72][86]. This implies that solver selection, time-step choice, contact modelling, and collision geometry approximations must be treated as part of the learning system design [56][72].

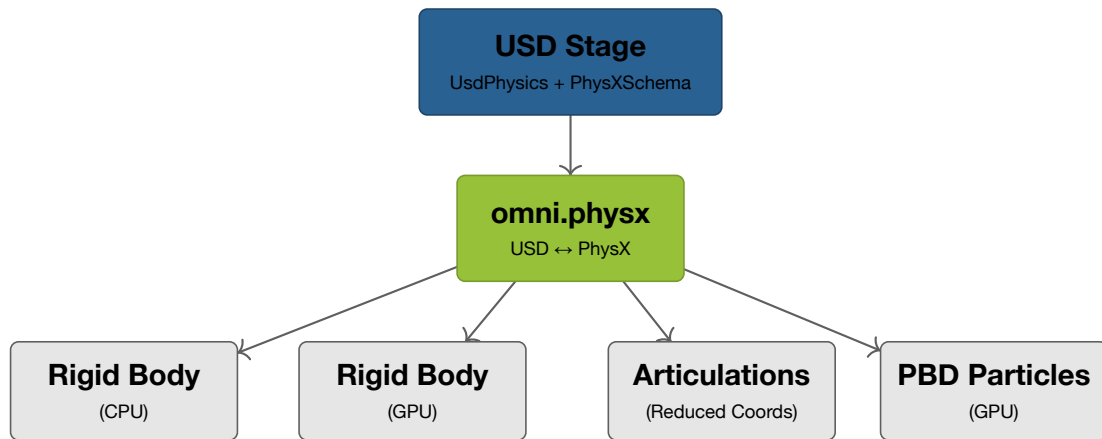


Figure 7: Conceptual physics simulation stack in Isaac Sim 5.1: physics is authored in USD via `UsdPhysics` and `PhysXSchema`, interpreted by `omni.physx`, and simulated using PhysX 5 with optional GPU-accelerated pipelines.

2.4.1 Rigid-Body Dynamics and Articulations

Rigid-body simulation assumes that each object moves as a rigid transform (no deformation), with state represented by position/orientation and linear/angular velocity. PhysX represents the simulated world as actors (static, kinematic, dynamic) with attached collision shapes and

material properties. Dynamics are advanced under external loads, contact constraints, and joint constraints. [49][54]

Articulated bodies

Robotic manipulators are modelled as articulated mechanisms. Rigid links connected by joints that constrain relative motion and optionally apply actuation. PhysX provides an *articulation* abstraction simulated in *reduced coordinates*, where the configuration is parameterized by the root pose and joint coordinates rather than the world pose of each link. This formulation improves numerical fidelity for mechanisms with large mass ratios and long kinematic chains compared to independently simulating rigid bodies connected by generic constraints. [55]

Articulation tendons

PhysX exposes *articulation tendons* as additional constraint structures within an articulation. *Fixed tendons* couple joint coordinates through constant gearing ratios, while *spatial tendons* define spring-damper and limit constraints between attachment points across links, producing configuration-dependent forces. These constructs enable cable-like actuation or joint coupling to be represented at the physics level without external force computation. [55][59]

See Section 4.4 for their application in the methodology.

2.4.2 Contact and Friction Models

Contact response is formulated as a constrained dynamics problem. Contacts prevent inter-penetration and transfer or dissipate energy through restitution and friction [50]. Friction and restitution are specified via materials attached to shapes, with friction parameterized by static and dynamic coefficients [49][51]. Because contact-rich manipulation is common in robotic grasping, parameter choices for friction and restitution are directly coupled to learning outcomes and sim-to-real robustness [72][86].

Constraint solving

PhysX supports a classic PGS-style solver and the newer TGS solver, with TGS recommended for improved convergence and joint drive accuracy [49]. Both operate iteratively over constraint manifolds (contacts, joints, friction), trading accuracy for performance by limiting iteration counts. For stiff constraint systems, replacing multiple solver iterations at a long time step by multiple smaller substeps can yield superior stability at comparable computational cost [35][56].

2.4.3 Simulation Parameters

Parameter	Location	Primary effect
Time step h	Physics scene	Accuracy vs. throughput; contact/joint stability [57]
Solver iterations	Actor / island	Constraint convergence; joint error; contact quality [49]
contactOffset/restOffset	Shape	Contact prediction and resting separation [52]
Friction coefficients	Material	Sliding/sticking behavior; grasp stability [51]

Table 1: Selected rigid-body simulation parameters relevant to robot learning in Isaac Sim / PhysX.

Table 1 summarizes key simulation parameters that influence the fidelity and stability of rigid-body dynamics in PhysX. The time step h determines how frequently dynamics are integrated and how often the policy interacts with the environment. Overly large h can lead to missed collisions and unstable joint behavior, whereas smaller h improves accuracy at increased compute cost [56][57].

Collision detection cost depends on the collider representation. Primitive shapes (spheres, boxes, capsules) provide the best performance and robustness, while convex meshes provide a practical middle ground for complex objects [50][58]. Collider selection trades geometric fidelity against computational cost and solver stability. This tradeoff must be balanced against the throughput requirements of RL training.

2.4.4 GPU-Accelerated Simulation for Parallel RL Environments

TODO: Recheck this section for accuracy and correct citations

PhysX provides a GPU rigid-body pipeline that accelerates broad-phase, narrow-phase, contact generation, and constraint solving on CUDA-capable hardware. Large performance gains arise when many rigid bodies are active, which aligns with RL workloads where hundreds to thousands of environment replicas are simulated in parallel. The key advantage is that physics stepping and policy learning can share device memory, avoiding Central Processing Unit (CPU)–GPU transfer bottlenecks. This design has been demonstrated to yield order-of-magnitude throughput improvements in GPU-based robot learning platforms. [36][64]

2.5 NVIDIA Isaac Sim and IsaacLab

NVIDIA Isaac Sim (Version 5.1.0) is used as the primary simulation environment in this thesis because it integrates robotics-oriented scene authoring, high-fidelity multi-physics simulation, and sensor-realistic rendering within a single Omniverse-based application [61]. The associated IsaacLab framework structures the learning workflow, providing a modular interface for designing robotics tasks as MDPs and training policies at scale using GPU-accelerated parallel simulation [68].

2.5.1 Isaac Sim in the Omniverse Stack

Isaac Sim is a reference application on top of NVIDIA Omniverse, designed for developing, simulating, and testing AI-driven robots in physically based virtual environment. A central architectural decision is the use of USD as the scene representation. Robotics assets and environments are expressed as a hierarchical scene graph, serving as the common data layer across physics, rendering, and robotics tooling. Isaac Sim is built on Omniverse Kit, a modular framework based on lightweight plugins and an extension mechanism. Capabilities such as physics interfaces, sensor simulation, and robotics middleware bridges are exposed as extensions that can be enabled, configured, and scripted from Python — supporting both interactive development and headless execution in automated pipelines. [61] The resulting software stack is shown in Figure 8.

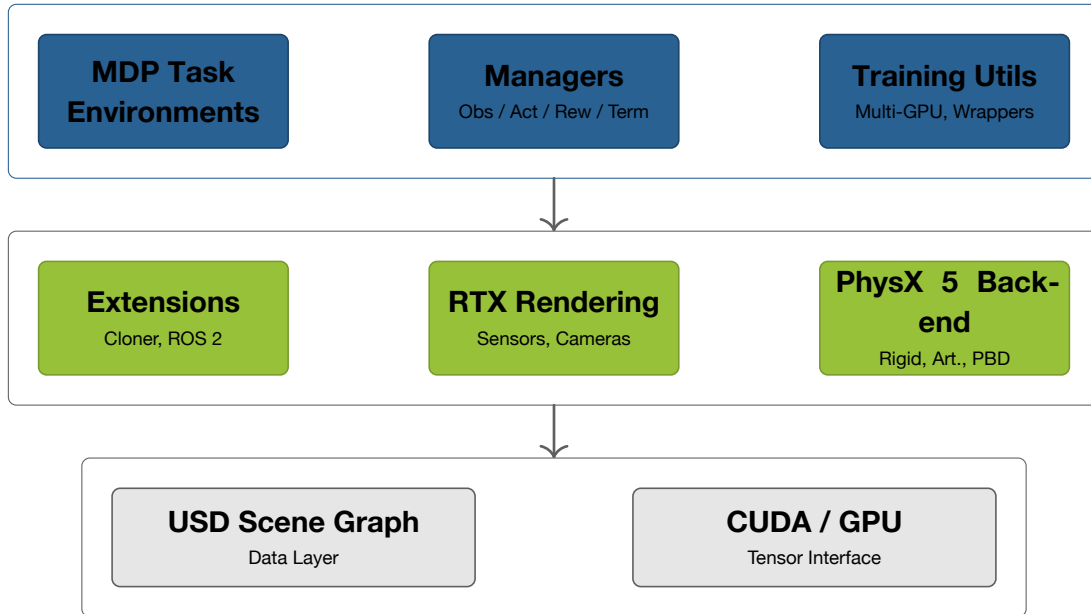


Figure 8: Schematic view of the software stack used in this thesis. Isaac Sim is built on Omniverse Kit with USD as the scene data layer and PhysX 5 as the physics backend. IsaacLab provides task abstractions (manager-based MDP environments) and training utilities on top.

The most relevant capabilities for this thesis are:

- **PhysX 5-based multi-physics:** GPU-accelerated rigid-body and articulated dynamics for contact-rich robotics simulation (Section 2.4) [53][61].
- **Environment cloning:** The `isaacsim.core.cloner` extension efficiently replicates prims and environments with optional inter-clone collision filtering, enabling the throughput scaling required for RL [60].
- **USD-centric asset pipeline:** Structured scene graphs with physics and rendering metadata support scalable instancing and non-destructive composition of complex environments [61].

2.5.2 IsaacLab as a Robot Learning Framework

IsaacLab is the successor to Isaac Gym and extends GPU-native simulation into large-scale robot learning while remaining integrated with Isaac Sim for high-fidelity physics [68]. It consolidates the GPU-native paradigm of Isaac Gym [36] and the modular environment design of the Orbit framework [41] into a unified training runtime.

Manager-based MDP workflow

IsaacLab structures simulation tasks as MDPs through a manager-based architecture: a base environment is composed from reusable managers (action, observation, event), then extended to an RL task environment by adding reward, termination, command, and curriculum components [23]. This decomposition separates environment implementation from task specification: task-specific behavior is encoded in configuration objects rather than by modifying core environment classes, enabling consistent reuse across tasks and reducing duplication across experimental variants [23].

The `ManagerBasedRLEnv` interface returns MDP-relevant signals while remaining compatible with standard environment creation patterns (e.g., via `gymnasium.make`), supporting integration with multiple RL libraries without rewriting the simulator interface layer [23].

GPU-native training

The key performance advantage is that physics stepping and policy learning share device memory, avoiding CPU–GPU transfer bottlenecks. Isaac Gym demonstrated that this GPU-resident design can yield order-of-magnitude improvements in end-to-end training throughput compared to CPU-based simulation [36]. IsaacLab extends this paradigm with documented multi-GPU and multi-node training support, enabling scale-out execution for RL workloads when training is bottlenecked by rollout collection [24][68].

2.5.3 Comparison to Alternative Simulators

Platform	Primary strength	Thesis fit
Isaac Sim + IsaacLab	Integrated Omniverse stack: PhysX, RTX, RL tooling [61][68]	High
MuJoCo	Efficient dynamics for model-based control [98]	Medium
PyBullet/Bullet	Accessible real-time physics [6][80]	Medium

Table 2: Qualitative comparison of simulation platforms with respect to thesis requirements. Isaac Sim is selected for its integrated physics, rendering, and scalable RL training in a single runtime.

MuJoCo [98] is widely adopted in continuous-control research for efficient per-step dynamics but does not natively provide sensor-realistic rendering or a scalable parallel training framework within the same architecture. PyBullet [80] offers rapid prototyping through Python-accessible simulation but requires assembly of separate components for rendering, training infrastructure, and middleware integration. Isaac Sim 5.1 with IsaacLab is selected because it provides an integrated USD-based scene representation, PhysX-backed multi-physics, and a principled MDP-based task design workflow with GPU-native training support in a single runtime [61][68].

2.6 Robot and Scene Description Formats

Robotic learning workflows in simulation require at least two layers of model description: a *robot model* capturing kinematic structure and physical properties, and a *scene model* capturing environment layout, lighting, and simulator-specific physics configuration. In the Omniverse ecosystem, the scene is represented as a composed USD stage, while robots are typically authored in robotics-oriented formats such as Unified Robot Description Format (URDF) and subsequently imported into USD for simulation.

2.6.1 USD as Scene Graph

USD is a system for encoding scalable, hierarchically organized data with the primary purpose of interchange and augmentation across digital content creation applications [75]. Beyond

being a file encoding, USD composes a single stage from scene description distributed across many layers and assets, supporting sparse overrides via composition operators (references, variants) [75]. This design makes USD the foundation of Omniverse-based simulation: worlds are assembled from reusable assets (robots, fixtures, task objects), while experiment-specific changes (randomization parameters, material settings) are authored as non-destructive overrides.

Physics semantics are provided by *UsdPhysics*, which defines schemas for rigid bodies, joints, constraints, and joint drives on the scene graph [74]. Simulator-specific extensions (e.g., PhysX solver parameters, tendon constraints) are exposed through additional *PhysXSchema* additions [65].

2.6.2 URDF as Robot Model Format

URDF is an Extensible Markup Language (XML)-based format in the ROS ecosystem for specifying robot geometry and organization [69]. A URDF model describes the robot as rigid links connected through joints, augmented with collision geometry, inertial parameters, and joint limits [70]. This focus on kinematic structure makes URDF the standard interchange format for robotic manipulators.

Structural limitations for tendon mechanisms

Standard URDF supports only serial kinematic chains; closed-loop dependencies are handled through simulator-specific mechanisms [21]. The *mimic joint* concept allows linear coupling between joints (e.g., for belt-and-pulley systems), but tendon routing with wrapping, varying moment arms, elastic stretch, and multi-joint coupling is not representable with standard URDF alone [21]. More complex tendon actuation must be introduced through simulator-specific extensions or higher-level actuation models.

2.6.3 Format Pipeline and Tendon Workarounds

Format	Primary scope	Role in simulation
USD	Scene graph with composition	World assembly, physics/rendering schemas [74][75]
URDF	Robot kinematics + rigid-body data	Robot model interchange; converted into USD [66][69]

Table 3: Roles of USD and URDF in the simulation pipeline used in this thesis.

In this thesis, the tendon-driven arm is authored as a modified URDF and imported into Isaac Sim using the provided URDF-to-USD importer [66]. Isaac Lab recommends converting external formats into USD and performing subsequent editing in the USD representation [25]. Once converted, the robot is integrated with environment assets on a composed USD stage, with physics semantics attached through *UsdPhysics* and *PhysXSchema* [65][74].

Tendon modeling workarounds

Since standard URDF cannot express tendon routing and PhysX articulation tendons are listed among known limitations in Omniverse Physics [67], tendon actuation is approximated through joint drives and coupling simplifications rather than explicit tendon constraints (Section 4.4 details the three actuation modes implemented in the methodology). Mimic-joint coupling is used where tendon effects can be approximated as linear joint relationships, and external force computation is used where configuration-dependent tendon geometry must be accounted for [21][67].

3 State of the Art and Research Gap

This chapter reviews prior work along three dimensions that converge in the present thesis: (i) Reinforcement Learning-based manipulation in GPU-accelerated simulators, (ii) simulation and control of tendon- and cable-driven robots, (iii) tensegrity-inspired and linkage-based robot design. The review focuses on contributions that directly inform the methodological choices made in this thesis and concludes by identifying the specific gap that the present work addresses.

3.1 GPU-Accelerated Reinforcement Learning for Manipulation

Isaac Gym and large-scale parallel training

Makoviychuk *et al.* demonstrated that running both PhysX simulation and neural-network training entirely on GPU eliminates CPU–GPU data transfer bottlenecks and yields two to three orders of magnitude training speedup over CPU-based simulators [36]. In their Franka cube stacking benchmark, 16{384 parallel agents trained with PPO converged in under 25 minutes on a single A100 GPU, and a Shadow Hand cube rotation task achieved 20 consecutive successful rotations in approximately one hour. These results established Isaac Gym as the foundational platform for GPU-accelerated RL and underpin the Isaac Sim/IsaacLab ecosystem used in the present work [36].

Contact-rich manipulation and assembly

Narang *et al.* introduced *Factory*, a suite of Isaac Gym environments for contact-rich assembly tasks that achieves a reported 20{000-fold speedup over prior state-of-the-art contact simulation through signed-distance-function collisions, contact reduction from approximately one million naive contacts to approximately 10{000, and a Gauss–Seidel solver [46]. Building on *Factory*, Tang *et al.* (*IndustReal*) transferred peg-insertion and gear-assembly policies from simulation to a real Franka Panda, reporting 83–99% success rates over 600 physical trials with realistic part clearances of $\leq 0.5\text{--}0.6$ mm. *IndustReal* introduced a simulation-aware policy update mechanism that detects unreliable simulator states and a sampling-based curriculum for progressive task difficulty—techniques that are directly adoptable for conveyor-based pick-and-place policy design [96].

Frameworks: Orbit, Isaac Lab, and skrl

Mittal *et al.* presented *Orbit*, a modular design powered by NVIDIA Isaac Sim with GPU-based parallelization, wrapping four RL libraries (rl_games, RSL-RL, Stable-Baselines3, skrl) and providing 20+ benchmark tasks [41]. *Orbit* evolved into *Isaac Lab* in March 2024, achieving up to 1.6 million frames per second for state-based manipulation tasks and adding mimic joint support, closed-loop kinematic chains, and a deformable-object solver [68]. The *skrl* library provides native first-class support for Isaac Lab environments with competitive performance to *rl_games* on standard benchmarks while offering greater modularity through seven stand-alone components [89]. In the present thesis, Isaac Lab serves as the development framework and *skrl* provides the PPO training backend.

3.2 Tendon- and Cable-Driven Robot Simulation

Rigid-link tendon-driven manipulation

Or *et al.* presented *Robostrich*, a 24-cable, 15-DoF rigid-link tendon-driven manipulator trained with curriculum-based RL in MuJoCo, demonstrating that progressive target difficulty outperforms flat training for underactuated tendon arms [71]. Guist *et al.* (PAMY2) applied model-based RL to a 4-DoF pneumatic-tendon arm, achieving safe and accurate high-speed motions through a learned dynamics model embedded in a trajectory optimizer [17]. Both works use MuJoCo’s native tendon elements and general actuators, which support tendon contraction-to-joint-actuation coupling including routing sites and moment arms [17][71]. Shahid *et al.* modeled a bio-inspired tendon arm in MuJoCo with curriculum-based SAC, confirming that curriculum training outperforms vanilla SAC for reaching tasks on underactuated manipulators [91]. Saito and Morimoto compared a 9-tendon continuum arm with a 7-DoF rigid manipulator, finding that continuum arms excel in uncertainty-rich tasks while rigid arms are superior for precision tasks, with sim-to-real validation on a real pneumatic continuum robot [84].

Cable-driven parallel robot simulation with RL

Dhakate *et al.* (CaRoSaC) used Unity3D with Obi Rope for flexible cable simulation via Extended Position-Based Dynamics (XPBD), validated against a real industrial 4-cable-suspended parallel robot. The TD3-based RL controller outperformed classical kinematic solvers in trajectory tracking, especially near workspace boundaries where force-distribution assumptions break down [9]. Garrido Campos *et al.* validated a cable-driven parallel robot simulation against a hardware prototype, achieving a peak position divergence of only 0.27% across various speed scenarios [15].

Tendon-driven modeling benchmarks

Lilge and Burgner-Kahrs comprehensively benchmarked four tendon-driven continuum robot modeling approaches—constant curvature, pseudo-rigid-body, and Cosserat rod formulations—under varying loads. A key finding is that tendon force expressions differ across the literature and yield divergent results under external loading, establishing accuracy-versus-computation-time trade-offs that inform which level of fidelity a physics engine should reproduce [34].

Gap observation: no tendon-driven RL in Isaac Sim

All rigid-link tendon RL work reviewed above uses MuJoCo (Robostrich, PAMY2, Shahid) or Unity (CaRoSaC) as the simulation backend. No published work, as of March 2026, trains RL policies for tendon-driven rigid-link robots within the Isaac Sim / IsaacLab ecosystem, despite its GPU-native training advantages.

3.3 Tensegrity-Inspired and Linkage-Based Robot Design

Tensegrity manipulators

Shah *et al.* [90] survey the application of tensegrity to robotics, categorizing designs into locomotion platforms, continuum arms, and rigid-link serial mechanisms. For manipulation specifically, Lessard *et al.* [33] demonstrated a bio-inspired tensegrity arm with structurally compliant joints. Furet and Wenger [14] proposed a planar tensegrity 2-X manipulator using antiparallelogram joints and derived the kinetostatic analysis including coupled cable tensions.

Fasquelle *et al.* [12][13] extended this to a physical 3-DoF prototype, performing system identification and closed-loop position control. Muralidharan and Wenger [43] compared antagonistically actuated X-joints with R-joints regarding workspace coverage, stiffness modulation, and singularity-free design.

Antiparallelogram joints in robot mechanisms

Four-bar linkages and specifically the antiparallelogram appear in several robotic joint designs. Hamon and Aoustin [19] used crossed four-bar linkages for the knees of a planar bipedal robot, exploiting the nonlinear torque transmission. Yoon *et al.* [106] applied spatial antiparallelogram linkages to a 2-DoF wrist mechanism for the DLR robot. Muralidharan *et al.* [44] provided a comprehensive workspace comparison between 2-X and 2-R tensegrity manipulators, establishing that the X-joint variant achieves substantially larger orientation range for the same link lengths.

Tensegrity RL (locomotion only)

RL has been applied to tensegrity robots exclusively in the locomotion domain. Zhang *et al.* applied model-based RL (MDGPS) to the SUPERball tensegrity locomotive, and subsequent work used GNN-based SAC for tensegrity locomotion with varying morphologies. No published work applies RL to tensegrity *manipulation* or serial arm control.

FAPS tensegrity manipulator

Walter *et al.* [105] introduced the cable-driven tensegrity manipulator used in this thesis and demonstrated 83% impact force isolation in hardware experiments. Klein [30] established the initial workspace analysis and Gazebo simulation model for this robot but did not employ learning-based control.

3.4 Identified Research Gap

The literature reviewed above reveals three distinct bodies of work that have not been combined. First, GPU-accelerated RL for manipulation has advanced rapidly within the Isaac Gym / Isaac Sim ecosystem, with demonstrated sim-to-real transfer for contact-rich tasks involving conventional rigid-body robots [1][20][36][46][96]. Second, tendon- and cable-driven robot simulation for RL has been explored predominantly in MuJoCo [17][71][84][91] or in specialized engines [9], but not in Isaac Sim. Third, tensegrity-inspired manipulators with antiparallelogram joints have been studied from a mechanism-design perspective [12][14][43], and RL for tensegrity robots has been limited to locomotion, with no work addressing RL-based manipulation with tensegrity-inspired serial arms.

No published work, as of March 2026, combines antiparallelogram-jointed tensegrity arm design with tendon actuation modeling in the Isaac Sim / IsaacLab ecosystem and RL-based policy training. Furthermore, the FAPS-internal predecessor work by Klein [30] established workspace analysis and simulation for this cable-driven manipulator but did not employ learning-based control. The present thesis addresses this gap by constructing a validated tendon-driven tensegrity robot model in Isaac Sim, formulating progressive RL tasks within Isaac Lab, and training manipulation policies using `skrl` PPO [88][89].

3.5 Success Criteria

The following criteria define successful completion of the thesis objectives:

1. The simulated robot model reproduces the kinematic workspace of the physical design by Klein [30], validated through Monte Carlo workspace analysis and step-response testing.
2. RL policies trained with PPO converge to functional reach and cube-place behavior for at least one actuation variant (PD or tendon-driven).
3. Comparative evaluation of PD-driven and tendon-driven actuation variants provides quantitative metrics (success rate, convergence speed, final reward) enabling informed selection.
4. The simulation infrastructure is documented and reproducible, providing a foundation for the subsequent master thesis.

4 Methodology

This chapter describes the complete workflow for constructing, validating, and using a tendon-driven tensegrity manipulator in simulation for RL-based manipulation tasks. The presentation follows the temporal sequence of the project. First, the robot design is introduced (Section 4.2). Next, the process for translating the Computer-Aided Design (CAD) geometry into a physics-ready Isaac Sim simulation model is detailed (Section 4.3). The tendon actuation layer that maps cable tensions to joint torques in simulation is then derived (Section 4.4). Validation of the resulting model against reference data is presented (Section 4.5). Finally, the formulation of RL tasks built on top of this model is described (Section 4.6), together with the training infrastructure (Section 4.7).

All implementation is carried out in NVIDIA Isaac Sim 5.1.0 with the IsaacLab framework, using the `skrl` library for PPO-based policy training [88][89]. The source code is organized as an external IsaacLab extension (`tensegrity_pick`) that provides registered Gymnasium environments for each task and robot variant.

4.1 Overview of Approach

The experimental methodology follows three stages, illustrated in Figure 9:

1. **Model construction and validation.** A physically representative simulation model of the tendon-driven tensegrity manipulator is constructed from CAD data and hardware specifications. Two distinct elbow model variants are implemented: a simplified disc approximation and the full antiparallelogram four-bar linkage. Each variant is validated through step-response testing and workspace analysis against reference data from Klein [30].
2. **Progressive task formulation.** Two RL tasks of increasing complexity are defined on the validated model. A *reach* task trains the agent to track end-effector poses, verifying that each actuation mode supports effective policy learning. A *cube place* task extends reach to a sequential pick-transport-place problem with the Robotiq 2F-140 gripper, requiring both arm coordination and gripper control.
3. **Cross-variant evaluation.** Each task is trained under three actuation modes (PD-driven, tendon-driven, and physical tendon-driven), producing a systematic comparison of learning performance, motion quality, and task success across different levels of physical fidelity.

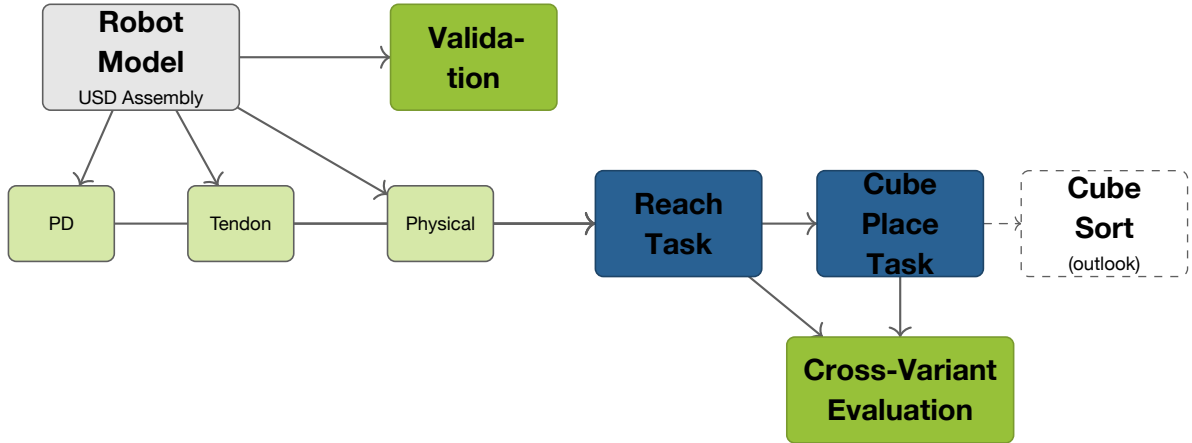


Figure 9: Overview of the experimental pipeline. Stage 1 constructs and validates the robot model with three actuation modes. Stage 2 formulates progressive RL tasks. Stage 3 evaluates task performance across all robot-actuation variants.

4.2 Robot Design

This section describes the composite manipulator system used throughout this work. Its design brings together three distinct subsystems of different origin: a tendon-driven tensegrity arm built as physical hardware at FAPS, an industrial parallel-jaw gripper added for its versatility, and a prismatic positioning base created purely in simulation. Understanding each component’s design rationale is essential for interpreting the simulation model and the choices made during task development.

4.2.1 Kinematic Chain and Degrees of Freedom

The tensegrity manipulator is a ceiling-mounted, 5-DoF serial kinematic chain. It consists of three functional subsystems arranged in series: a 2-DoF prismatic positioning base, a 3-DoF tendon-driven arm, and a Robotiq 2F-140 parallel-jaw gripper. Figure 10 shows the kinematic topology, and Figure 11 shows the fully assembled manipulator variants rendered in simulation.

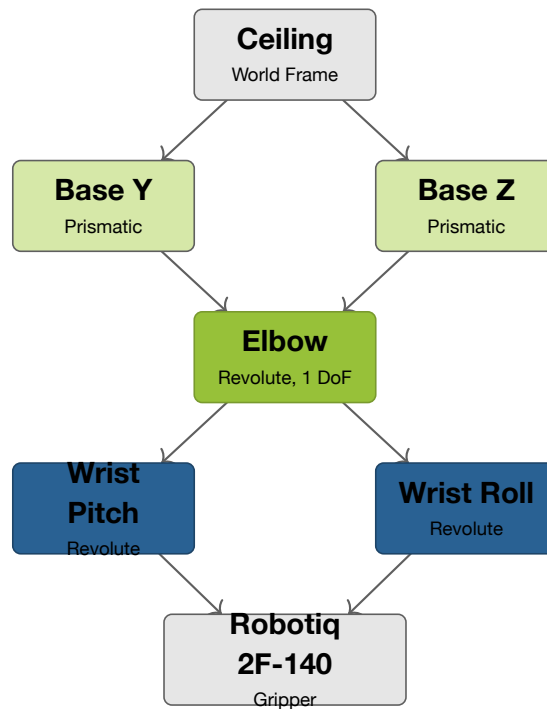


Figure 10: Kinematic topology of the 5-DoF tensegrity manipulator. The ceiling-mounted base provides two prismatic DoF. The tendon-driven arm contributes three revolute DoF (elbow, wrist pitch, wrist roll). The Robotiq 2F-140 gripper attaches at the wrist output flange.

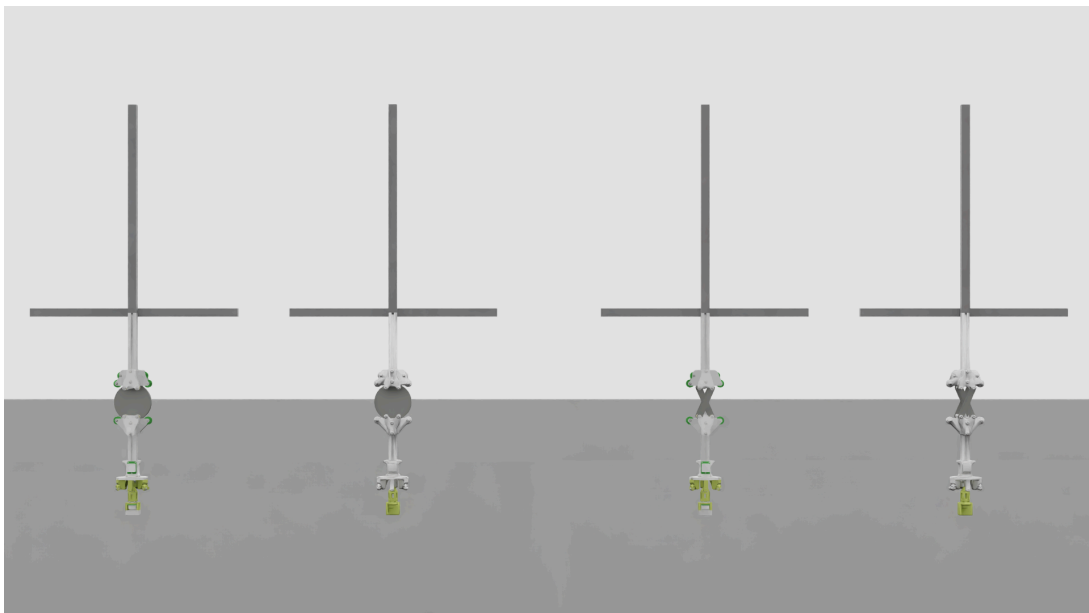


Figure 11: Robot model variants rendered in Isaac Sim. Additional to the Elbow disk approximated model, established by Klein [30], a model mimicking the actual four-bar antiparallelogram mechanism was added. Each variant is available with full-resolution and low-resolution visual meshes.

Tendon-driven arm. The arm comprises an upper arm (root link), a forearm, and a two-axis wrist, connected by three revolute joints. All three joints are actuated exclusively through cable-driven transmissions; no direct motor-to-joint coupling exists. The elbow joint provides $\pm 70^\circ$ of rotation in the sagittal plane. The wrist mechanism provides $\pm 50^\circ$ about each of two perpendicular axes (pitch and roll), enabling the end-effector to orient relative to the forearm axis [30].

Prismatic base. The two prismatic joints translate the arm mounting plate along the horizontal Y -axis and the vertical Z -axis. The base is a purely simulative addition. It was created using the Isaac Sim Robot Wizard (BETA) and is oversimplified to allow rapid prototyping of different base configurations in the future. Its Y -axis provides linear motion in the same direction as the elbow swing, extending the arm's horizontal reach along the cross-section of the conveyor belt. The Z -axis provides vertical motion of the arm toward the conveyor. A additional prismatic joint along the X -axis was purposely omitted, as the conveyor provides transport motion in that direction. The wrist's two rotational degrees of freedom compensate for residual positioning errors introduced by conveyor tolerances. The base provides a planar positioning range of ± 0.5 m in Y and -0.5 m to 0.0 m in Z (downward from the ceiling).

Gripper. The Robotiq 2F-140 is an adaptive parallel-jaw gripper with a 140 mm stroke, adjustable grip force from 10 N to 125 N, and a payload capacity of approximately 2.5 kg [83]. It was selected for its versatility and adaptability to a wide range of object sizes and shapes, which is critical for the envisioned trash-sorting application where the robot must handle everything from rigid cubes to deformable cloth. The gripper is modeled as a self-contained articulated asset loaded from the Isaac Sim built-in asset library. A single actuated joint drives both fingers symmetrically via an internal linkage.

Modular assembly. The robot's USD description, Isaac Lab configuration, and task structure are designed so that the base, arm, and gripper are independently exchangeable. This modularity makes it straightforward to explore alternative base or gripper combinations — for example, replacing the prismatic base with an articulated arm mount or substituting a suction gripper — without modifying the training pipeline.

Table 4 summarizes the principal link dimensions and masses of the tendon-driven arm, extracted from the CAD assembly and verified against Klein [30].

Link	Length (mm)	Mass (kg)	Description
Root link	150	2.6737	Upper arm, elbow motor housing
Forearm	300	0.8891	Carbon-fiber tube with cable guides
Wrist plate	55	0.0653	2-DoF cable-driven joint platform
EE link	10	0.001	End-effector mounting flange

Table 4: Link dimensions and masses of the tendon-driven arm as specified in the CAD model [30]. The root link contains the primary elbow actuation mechanism. The forearm serves as the structural conduit for the wrist tendons.

4.2.2 Antiparallelogram Elbow Mechanism

The elbow joint uses a tensegrity-inspired antiparallelogram four-bar linkage as its primary actuation mechanism. This crossed linkage topology provides inherent compliance and impact isolation, a key design goal for safe human–robot interaction described by Walter et al. [105].

Linkage geometry. The antiparallelogram is a planar four-bar mechanism in which the two side links (length $l_e = 150$ mm) cross each other, while the frame and coupler bars share a common length $k_e = 60$ mm [30]. Figure 12 illustrates the topology. The joints labeled A and B are fixed to the frame (root link); joints C and D are attached to the coupler (forearm) and trace the output motion. The four revolute joints of this mechanism are the minimum representation required to capture the kinematic constraint.

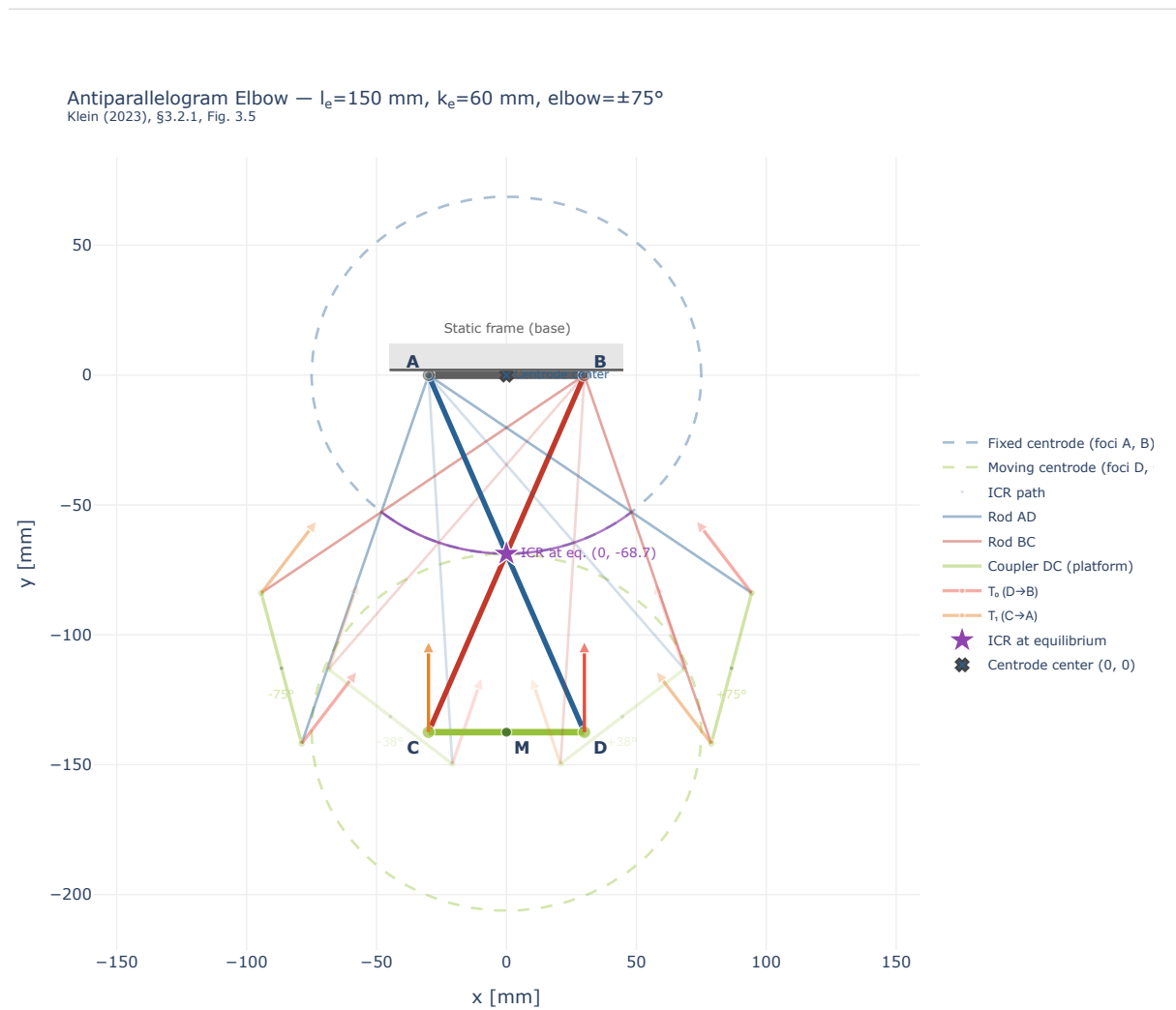


Figure 12: Five-pose overview of the antiparallelogram four-bar linkage forming the elbow mechanism, generated from the kinematic model. The linkage sweeps from -75° to $+75^\circ$ elbow angle. Joints A and B are grounded on the root link (frame, length k_e). Joints C and D are connected by the coupler (length k_e). The crossed side links each have length l_e . Centrode ellipses, instantaneous center of rotation (ICR), and tendon force directions are shown.

Kinematic closure condition. The configuration of a four-bar linkage is fully determined by a single input angle. Following the analytical framework of McCarthy and Soh [40], the loop-closure equation for the antiparallelogram relates the input crank angle θ (measured at joint A) to the output angle φ (measured at joint B) through a set of algebraic constraints derived from the polygon-closure condition:

$$\mathbf{Z}_1 e^{i\theta} + \mathbf{Z}_2 e^{i\beta} = \mathbf{Z}_3 e^{i\varphi} + \mathbf{Z}_4 \quad (9)$$

where $\mathbf{Z}_i$ denote the complex link vectors and β is the coupler angle. For the specific case of the antiparallelogram ($|\mathbf{Z}_1| = |\mathbf{Z}_3| = l_e$, $|\mathbf{Z}_2| = |\mathbf{Z}_4| = k_e$), this reduces to a tangent-half-angle substitution yielding a closed-form expression for $\varphi(\theta)$ [40].

Centrode analysis. The instantaneous center of rotation of the coupler relative to the frame traces a curve known as the fixed centrode. For the antiparallelogram, Dijksman [10] showed that the centrodes are ellipses, reflecting the fact that the mechanism's instant center migrates continuously during motion rather than remaining fixed. This migration produces a nonlinear transmission ratio: the effective moment arm of the tendons with respect to the elbow output angle changes as a function of the elbow configuration. The disc approximation model (described in Section 4.3.3) replaces this configuration-dependent behavior with a constant lever arm, while the physical linkage model (Section 4.3.4) preserves it.

Tendon actuation of the elbow. Two antagonistic tendons are attached at the midpoints of the side links and route to spool drums on Maxon EC60 motors. When one tendon is tensioned, the crossed links rotate and the coupler (forearm) swings about the effective instant center. The elbow torque is therefore a function of both the applied cable tension and the instantaneous geometry of the linkage. For the disc approximation variant, this relationship is linearized using a constant lever arm $r_e = 72.5 \text{ mm}$ (the perpendicular distance from tendon attachment to the revolute axis at zero configuration). For the physical variant, the torque depends on the actual force application geometry at the current linkage configuration.

4.2.3 Cable-Driven Wrist Mechanism

The wrist provides two rotational degrees of freedom (pitch and roll) and is based on the cable-driven parallel mechanism described by Nemoto et al. [47]. It connects the forearm tube to the end-effector plate through a passive universal joint, and motion is controlled entirely by three active cables.

Topology. The wrist consists of a base ring (attached to the forearm), a platform ring (attached to the end-effector), and six cables arranged symmetrically. Three cables are *passive* (structural, constraining the platform) and three are *active* (driven by motors via tendons routed through the forearm) [47]. The active cables are spaced at 120° intervals around the circumference and pass through guide holes in the base ring at a radius r_w from the wrist center.

Structure matrix. The mapping from cable tensions to platform wrenches is described by a structure matrix $\mathbf{A} \in \mathbb{R}^{2 \times 3}$ that captures the geometric arrangement of the three active cables. For small angular deflections, the cable direction vectors project onto the two rotational axes of the wrist, yielding the approximate relationship:

$$\boldsymbol{\tau}_w = \mathbf{A} \mathbf{T}_w \quad (10)$$

where $\boldsymbol{\tau}_w = [\tau_{\text{pitch}}, \tau_{\text{roll}}]^\top$ and $\mathbf{T}_w = [T_1, T_2, T_3]^\top$ denotes the three active cable tensions. The entries of $\mathbf{A}$ are determined by the angular positions of the cable attachment points and the

wrist radius. At the zero configuration with cables at 0° , 120° , and 240° , the structure matrix takes the form:

$$\mathbf{A} = r_w \begin{pmatrix} \cos(0^\circ) & \cos(120^\circ) & \cos(240^\circ) \\ \sin(0^\circ) & \sin(120^\circ) & \sin(240^\circ) \end{pmatrix} = r_w \begin{pmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{pmatrix} \quad (11)$$

This arrangement is redundantly actuated (three cables, two DoF), ensuring that the tension distribution can always satisfy the unilateral cable constraint $T_w \geq 0$ within the wrist's operating range [78][104].

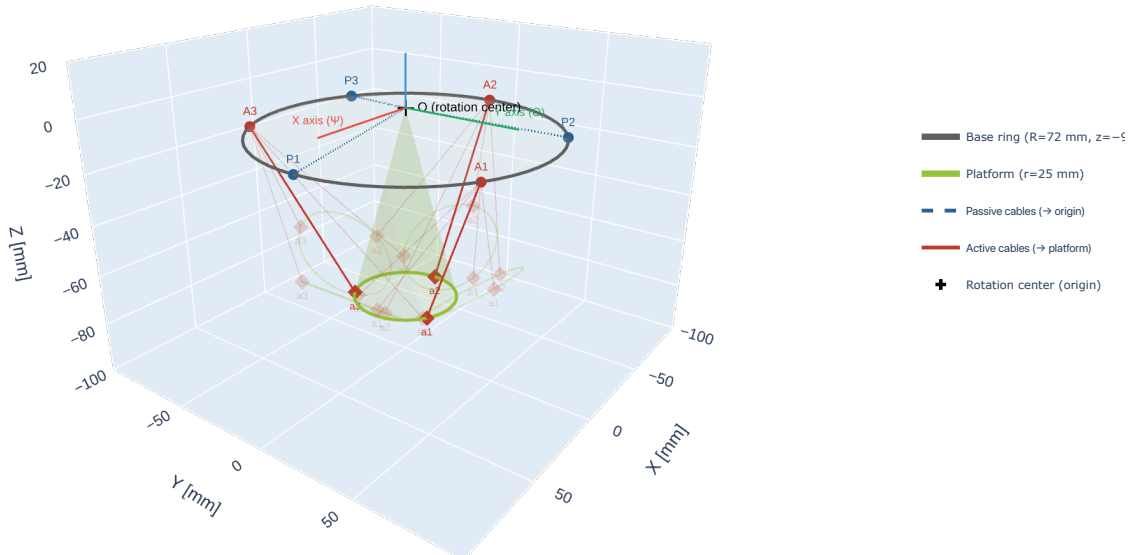


Figure 13: Three-dimensional overview of the cable-driven 2-DoF wrist mechanism, generated from the kinematic model. The base ring (radius $R = 72.5$ mm) is shown with six cable attachment points: three active cables (red) and three passive cables (blue). The platform ring (radius $r_w = 20$ mm) tilts about two perpendicular axes. Ghost poses illustrate the workspace envelope. The semi-transparent green cone visualises the reachable orientation space of the platform normal vector, representing the combined angular limits of the two wrist joints.

4.2.4 Drive System and Tendon Routing

The five tendons (two for elbow, three for wrist) are driven by five identical Maxon EC60 flat brushless DC motors (150 W, 24 V nominal), each controlled by a Maxon EPOS4 70/15 positioning controller [38][39]. The motors are mounted in the root link housing, and cables are routed through the structural tubes to their respective attachment points.

Spool and pulley system

Each motor winds a cable onto a spool with radius $r_s = 5$ mm. For the elbow, a 3:1 pulley reduction between the motor spool and the tendon attachment amplifies the mechanical advantage, yielding an effective elbow lever arm of:

$$r_{\text{eff,elbow}} = 3 \times r_s = 15 \text{ mm} \quad (12)$$

The wrist tendons are direct-drive (no additional reduction), so the effective wrist lever arm equals the wrist platform radius [30].

Maximum tensions

The maximum motor torque ($\tau_{\text{motor,max}}$) divided by the spool radius determines the maximum cable tension per motor. With the 3:1 elbow pulley, the maximum elbow tendon tension is three times the single-motor maximum:

$$T_{\text{max,elbow}} = 3 \cdot \frac{\tau_{\text{motor,max}}}{r_s} \quad (13)$$

The wrist tendons are direct-drive and carry at most $T_{\text{max,wrist}} = \frac{\tau_{\text{motor,max}}}{r_s}$ per cable. From the motor's power supply limit (PSU peak torque 0.79 Nm), the theoretical peak elbow force is 474 N. In simulation, a unified saturation limit of $T_{\text{max}} = 500 \text{ N}$ per tendon is used for all five tendons. While this accurately bounds the elbow, it is significantly higher than the wrist's continuous capability of 80 N.

Effort limits

The corresponding joint effort limits used by the physics solver cap the maximum torque producible through the tendon arrangement. Because the elbow tendons act antagonistically, maximum torque is generated when one pulls at T_{max} while the other is slack:

$$\tau_{\text{max,elbow}} = 1 \cdot T_{\text{max}} \cdot r_{\text{eff,elbow}} = 1 \times 500 \times 0.0725 = 36.25 \text{ Nm} \quad (14)$$

For the wrist, the maximum single-joint effort depends on the tendon geometry ($r_w = 20 \text{ mm}$) and the active tension limit. To protect against unphysical torques caused by the unified 500 N cable saturation, the configured joint effort limits clamp the wrist explicitly. The limits are set to 36.25 Nm for the elbow and 3.0 Nm for each wrist joint, providing a safeguard against unrealistic tension combinations [30].

Table 5 summarizes the joint parameters of the assembled 5-DoF manipulator.

Joint	Type & Axis	Range of Motion	Effort limit (Nm)
base_y_joint	Prismatic (Y)	$\pm 0.5 \text{ m}$	200
base_z_joint	Prismatic (Z)	$-0.5 \text{ to } 0.0 \text{ m}$	200
elbow_joint	Revolute (pitch)	$\pm 70^\circ$	36.25
wrist_y_joint	Revolute (pitch)	$\pm 50^\circ$	3.0
wrist_x_joint	Revolute (roll)	$\pm 50^\circ$	3.0
finger_joint	Revolute (gripper)	$0 \text{ to } 0.7854 \text{ rad}$	200

Table 5: Actuated joints of the integrated 5-DoF tensegrity manipulator. Joint ranges correspond to the practical workspace reported by Klein [30]. Effort limits are derived from the motor and tendon specifications.

4.3 Simulation Model Construction

Translating the CAD assembly of the tensegrity manipulator into a physics-ready Isaac Sim asset requires a multi-stage pipeline that processes raw geometry, defines articulation topology, assigns physics properties, and assembles the final robot. This section describes each stage in detail.

4.3.1 Mesh Processing Pipeline

The original CAD assembly is authored in Autodesk Fusion 360 and exported in FBX format. All subsequent processing is performed in Blender to prepare the meshes for physics simulation. Figure 14 illustrates the complete pipeline from CAD to Isaac Lab asset.

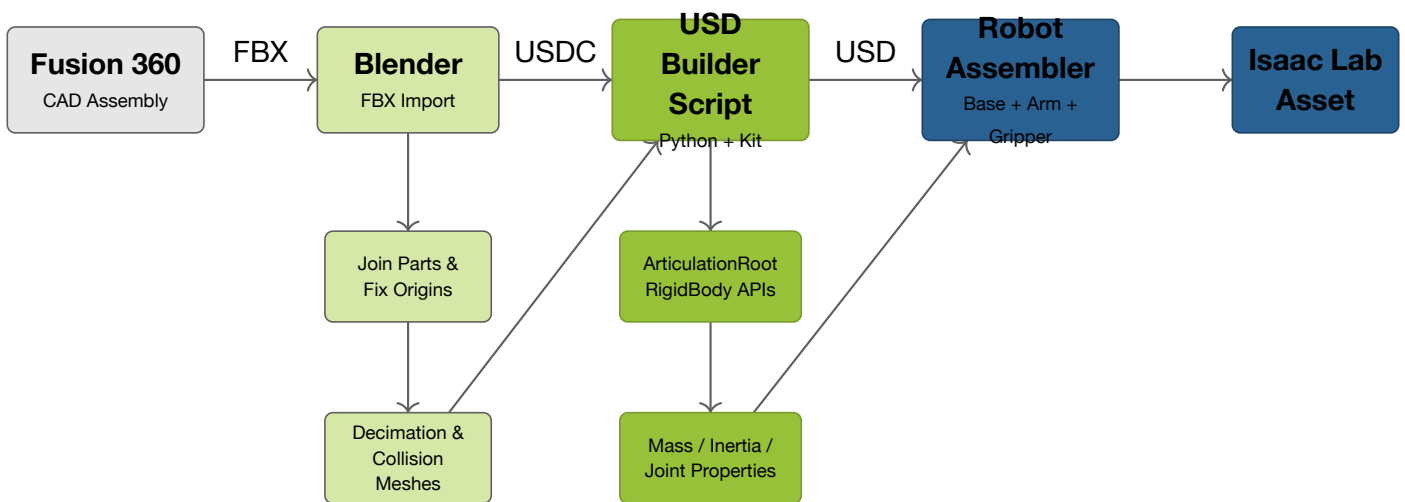


Figure 14: Mesh processing pipeline from CAD to Isaac Lab asset. Fusion 360 exports the assembly as FBX. Blender performs geometry preparation (joining, origin correction, decimation, and collision mesh extraction). The USD Builder Script assigns articulation, rigid-body, and collision APIs. The Robot Assembler combines base, arm, and gripper into a complete manipulator.

The Blender processing stage performs three principal operations:

- **Part joining and origin correction:** Individual part meshes from the CAD export are joined per rigid link (e.g., all screws, tubes, and brackets belonging to the forearm are merged into a single mesh object). Each link's origin is set to its center of mass as specified in the CAD data, ensuring that inertial properties align with the mesh geometry.
- **Mesh decimation:** For each visual mesh, a low-polygon variant is generated using Blender's *Decimate* modifier (ratio ≈ 0.1 – 0.3 depending on mesh complexity). The full-resolution meshes are retained for rendering, while the decimated variants are used for collision detection in GPU-accelerated simulation where mesh complexity directly affects solver throughput.
- **Collision mesh extraction:** Separate collision meshes are produced per link. For the disc approximation variant, convex hull decompositions of the full link geometry are used. For the physical antiparallelogram variant, the individual linkage bars require tighter collision

representations to avoid self-intersection during four-bar motion; here, oriented bounding boxes or tight convex hulls per bar segment are generated.

All mesh outputs are exported in OBJ or glTF format and organized in a per-link directory structure expected by the USD builder script.

4.3.2 Articulation and Physics Configuration

A Python-based USD builder script, executed within the NVIDIA Omniverse Kit runtime, constructs the complete articulated asset from the prepared meshes. The script performs the following per link:

1. **Prim hierarchy creation:** Each link is represented as a USD Xform prim under the articulation root. Visual and collision meshes are imported as child prims of their respective link.
2. **RigidBodyAPI application:** The `UsdPhysics.RigidBodyAPI` is applied to each link prim, enabling rigid-body simulation. Mass properties (mass, center of mass, and inertia tensor) are assigned from the CAD specifications listed in Table 4.
3. **CollisionAPI application:** Collision shapes are assigned by applying `UsdPhysics.MeshCollisionAPI` to the collision mesh prims. The approximation type is set to `convexHull` for standard links and to `convexDecomposition` for geometrically complex links (e.g., the wrist plate with cable guide holes).
4. **Joint definition:** Joints are created using `UsdPhysics.RevoluteJoint` or `UsdPhysics.PrismaticJoint` prims between parent and child links. Each joint prim specifies the axis of rotation or translation, position limits, effort limits (from Table 5), and velocity limits.
5. **ArticulationRootAPI:** The top-level prim receives `UsdPhysics.ArticulationRootAPI`, marking the tree root for the PhysX reduced-coordinate solver. Solver position and velocity iteration counts are tunable per asset.

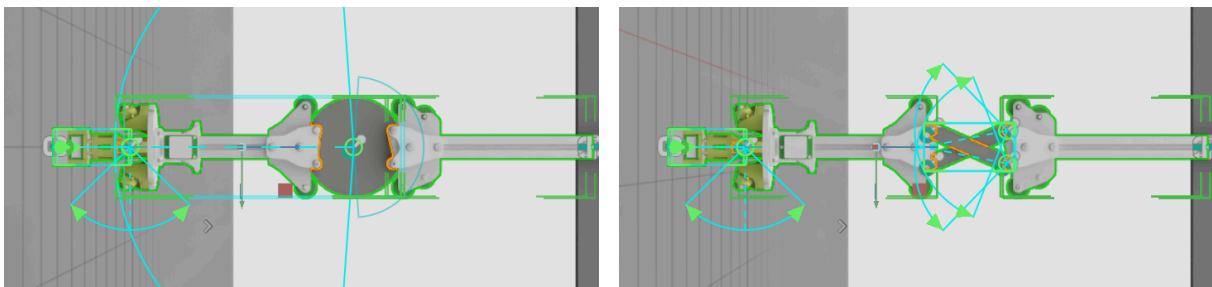


Figure 15: Joint visualization of the two arm variants in Isaac Sim. *Left:* Disc approximation variant with a single revolute elbow joint. *Right:* Physical antiparallelogram variant with four revolute joints forming the crossed linkage. Both share identical wrist joints.

4.3.3 Disc Approximation Variant

The disc approximation simplifies the antiparallelogram elbow mechanism into a single revolute joint. The four-bar linkage is replaced by a direct rotation of the forearm about a fixed

pivot coinciding with the center of the linkage’s frame bar. This is geometrically equivalent to assuming a circular (disc-shaped) centrode — hence the name — where the instant center remains fixed [102].

The resulting kinematic chain contains exactly five joints (two prismatic, three revolute) and five rigid links. The collision geometry consists of convex hull approximations of the full link meshes (Figure 17, left). This variant is computationally efficient, with low collision complexity and a minimal joint count, making it suitable for large-scale parallel RL training (4,096+ environments).

The key simplification is that the effective tendon lever arm for the elbow is treated as constant ($r_e = 72.5$ mm) regardless of joint angle. As discussed in Section 4.2.2, the physical mechanism exhibits a configuration-dependent lever arm due to centrode migration. The disc approximation can therefore not reproduce the elbow’s nonlinear torque-angle characteristic at extreme flexion/extension angles.

Figure 16 quantifies the kinematic deviation. The left panel overlays the forearm-tip trajectories of the physical antiparallelogram and the disc approximation across the full $\pm 75^\circ$ range; the right panel shows the positional error as a function of elbow angle. Because the antiparallelogram’s migrating instantaneous center of rotation shifts the coupler outward at large deflections, the disc arc always falls *inside* the physical trajectory. The maximum deviation remains below 10 mm across the operational range ($\pm 70^\circ$), confirming that the approximation is acceptable for the simulation tasks. Importantly, the disc model *underestimates* the elbow’s effective reach, which means workspace analyses and task designs based on this variant are conservative: the physical robot can reach at least as far.

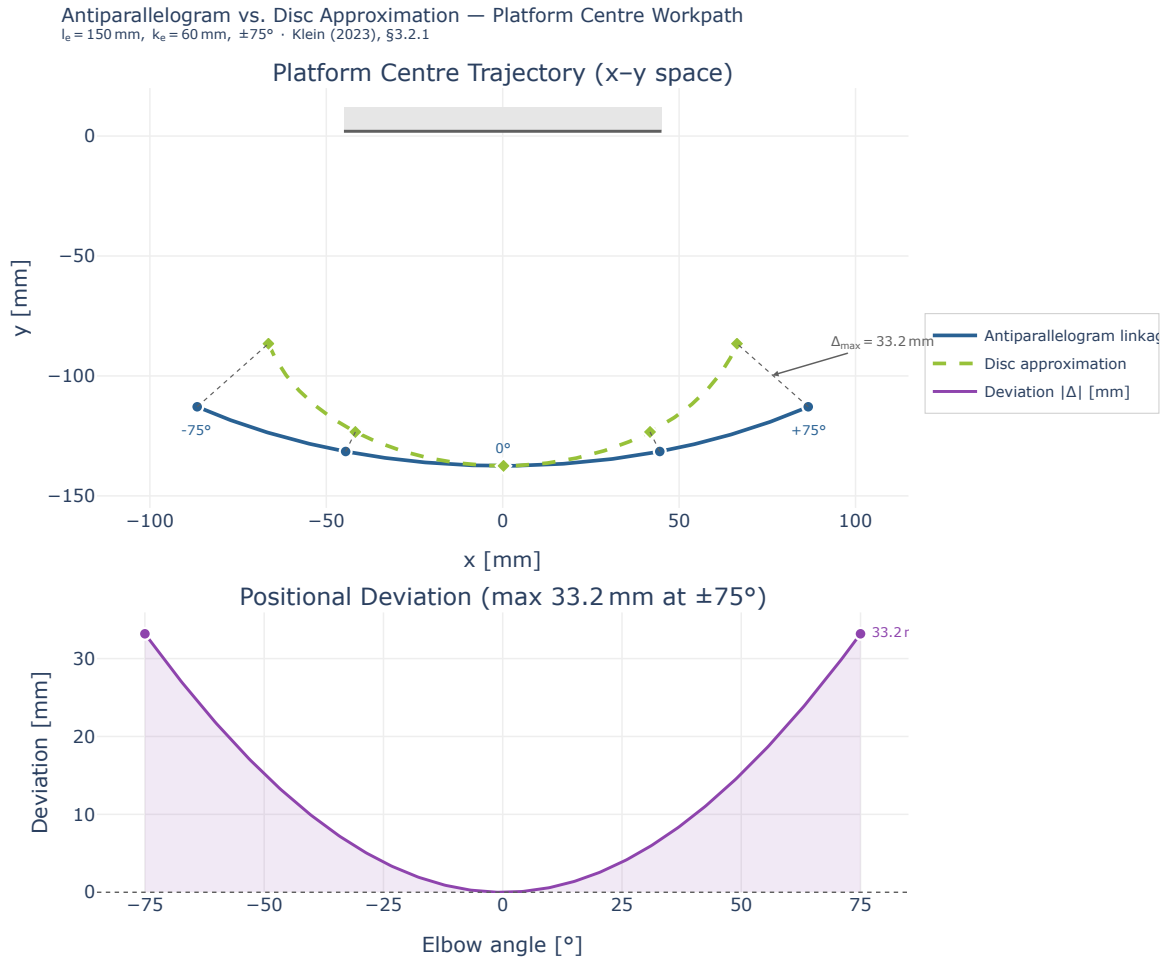


Figure 16: Kinematic comparison of the antiparallelogram mechanism and its disc approximation. *Left:* Forearm-tip trajectories for both models across $\pm 75^\circ$. *Right:* Positional deviation as a function of elbow angle. The disc approximation consistently underestimates the physical reach, making workspace designs based on it conservative.

4.3.4 Physical Antiparallelogram Linkage Variant

The physical variant replaces the single revolute elbow joint with the full four-bar antiparallelogram mechanism. The implementation adds four revolute joints and two additional rigid links (the two side bars of the crossed linkage) to the kinematic tree. The resulting articulation contains seven revolute joints (four for the linkage, two for the wrist, one for the gripper) plus the two prismatic base joints.

Loop-closure constraint

The antiparallelogram is a closed kinematic chain, but the PhysX articulation solver operates on tree-structured (open-chain) topologies. To enforce the loop closure, the fourth bar of the linkage is connected to the frame through a `PhysxSchema::PhysxJointAPI` with the `excludeFromArticulation` flag set to `true`. This instructs the solver to treat this joint as a *constraint* joint rather than as part of the reduced-coordinate articulation, effectively closing the kinematic loop at the constraint-solver level [64].

Link masses

The two side bars of the linkage are lightweight; each has a mass of 0.05 kg, derived from the aluminum bar geometry in the CAD model. The upper and lower linkage pivot blocks add 0.03 kg each. These masses are small compared to the root link (2.67 kg) and forearm (0.89 kg) but are included for accurate dynamic simulation.



Figure 17: Collision geometry comparison. *Left:* Disc approximation variant with convex-hull colliders per link. *Right:* Physical antiparallelogram variant with additional capsule colliders for the individual linkage bars and separate collision shapes for the upper and lower pivot blocks.

4.3.5 Five-DoF Manipulator Assembly

The final robot is assembled from three independently constructed USD assets using Isaac Sim's Robot Assembler tool:

1. **2-DoF articulated base** — two prismatic joints on rigid link geometry, providing Y/Z planar positioning.
2. **3-DoF tendon-driven arm** — the elbow and wrist articulation in either disc or physical variant.
3. **Robotiq 2F-140 gripper** — loaded from the Isaac Sim built-in asset library and attached at the wrist output flange.

The Robot Assembler creates fixed joints at the attachment interfaces, producing a single articulation tree. Four base variants emerge from the two $\times$ two combination matrix of elbow model and mesh resolution (Table 6).

Variant	Elbow model	Mesh resolution	Total joints
Approx. full-res	Disc (1 revolute)	Full	5 + gripper
Approx. low-res	Disc (1 revolute)	Low-poly	5 + gripper
Physical full-res	Antiparallelogram (4 revolute)	Full	8 + gripper
Physical low-res	Antiparallelogram (4 revolute)	Low-poly	8 + gripper

Table 6: Robot USD variant matrix. Two elbow models and two mesh resolutions produce four arm assets. Each is assembled with the prismatic base and Robotiq gripper to form a complete manipulator.

Figure 18 shows the base joint axes and the Robotiq 2F-140 gripper as rendered in Isaac Sim.

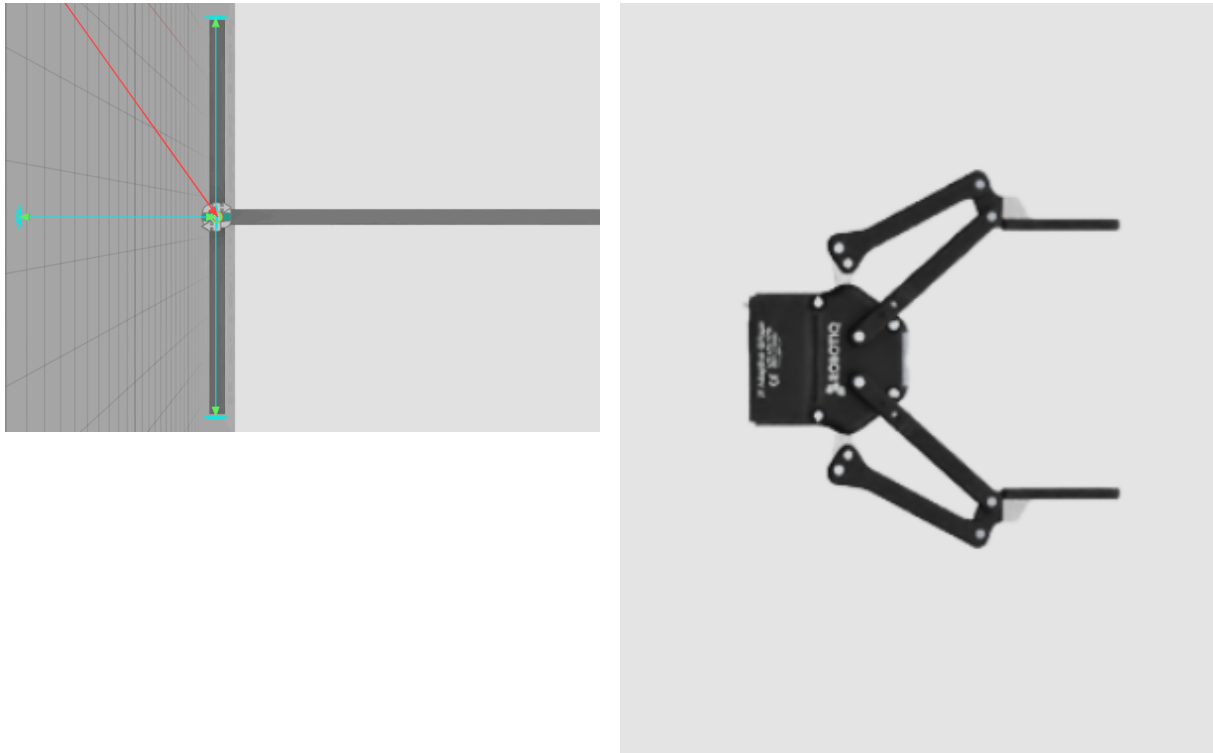


Figure 18: *Left:* Prismatic base joint axes (Y horizontal, Z vertical) as defined in the simulation. *Right:* Robotiq 2F-140 gripper with visual mesh and finger linkage.

4.4 Tendon Actuation in Simulation

A central contribution of this work is the implementation of tendon-driven actuation within the Isaac Lab framework. The physical robot's five cables must be mapped to joint torques in a way that is compatible with PhysX's reduced-coordinate articulation solver, differentiable

for gradient-based RL training, and GPU-parallelizable across thousands of environment instances. This section derives the actuation model and describes three implementation variants of increasing physical fidelity.

4.4.1 Jacobian-Transpose Torque Mapping

The fundamental relationship between cable tensions and joint torques in a tendon-driven mechanism is given by the Jacobian-transpose mapping [104]:

$$\boldsymbol{\tau} = \mathbf{J}^\top(\mathbf{q})\mathbf{T} \quad (15)$$

where $\boldsymbol{\tau} \in \mathbb{R}^n$ is the vector of joint torques, $\mathbf{T} \in \mathbb{R}^m$ is the vector of cable tensions ($m = 5$, $n = 3$), and $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{m \times n}$ is the tendon Jacobian that maps infinitesimal joint displacements to cable length changes. In general, $\mathbf{J}$ is configuration-dependent since the geometric relationship between cables and joints changes with joint angles.

For the constant-Jacobian actuation mode, $\mathbf{J}^\top$ is evaluated once at the zero configuration ($\mathbf{q} = \mathbf{0}$) and held fixed. The five tendons decompose into two subsystems:

Elbow subsystem (2 tendons, 1 joint)

The two antagonistic elbow cables attach symmetrically at the midpoints of the antiparallelogram side links. At zero configuration, each cable has a perpendicular lever arm $r_e = 72.5$ mm with respect to the effective elbow rotation axis. The net elbow torque is:

$$\tau_{\text{elbow}} = r_e(T_1 - T_2) \quad (16)$$

where T_1 and T_2 are the tensions in the two antagonistic cables. The sign convention is such that T_1 produces positive (flexion) torque and T_2 produces negative (extension) torque.

Wrist subsystem (3 tendons, 2 joints)

The three wrist cables generate torques depending on their geometric attachment to the wrist platform ($r_w = 20$ mm). Adjusting for the specific phase offsets ($0^\circ, 120^\circ, 240^\circ$) around the end-effector coordinate frame, the exact mapping implemented in the simulation is:

$$\begin{pmatrix} \tau_{\text{pitch}} \\ \tau_{\text{roll}} \end{pmatrix} = \begin{pmatrix} -r_w \frac{\sqrt{3}}{2} & 0 & r_w \frac{\sqrt{3}}{2} \\ \frac{r_w}{2} & -r_w & \frac{r_w}{2} \end{pmatrix} \begin{pmatrix} T_3 \\ T_4 \\ T_5 \end{pmatrix} \quad (17)$$

Combining both subsystems, the full constant $\mathbf{J}^\top \in \mathbb{R}^{3 \times 5}$ matrix matches the zero-configuration simulation property exactly:

$$\mathbf{J}^\top = \begin{pmatrix} r_e & -r_e & 0 & 0 & 0 \\ 0 & 0 & -r_w \frac{\sqrt{3}}{2} & 0 & r_w \frac{\sqrt{3}}{2} \\ 0 & 0 & \frac{r_w}{2} & -r_w & \frac{r_w}{2} \end{pmatrix} = \begin{pmatrix} 0.0725 & -0.0725 & 0 & 0 & 0 \\ 0 & 0 & -0.0173 & 0 & 0.0173 \\ 0 & 0 & 0.010 & -0.020 & 0.010 \end{pmatrix} \quad (18)$$

Figure 19 illustrates the complete data flow from policy output to physics-engine joint torques.

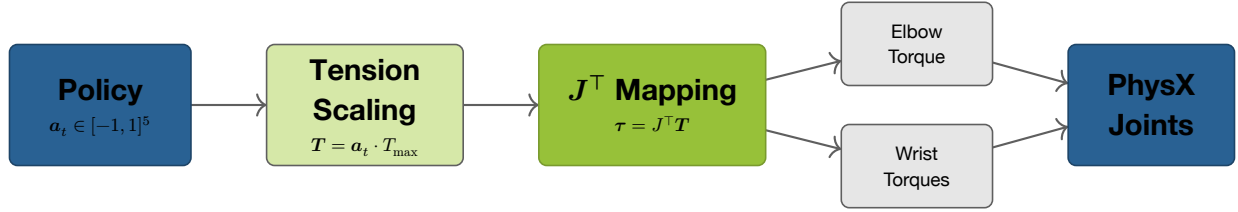


Figure 19: Data flow of the tendon actuation pipeline. The RL policy outputs five normalized actions $a_t \in [-1, 1]^5$. These are scaled to cable tensions T by the maximum tension $T_{\max}$. The J^T mapping converts tensions to elbow and wrist torques, which are applied to the PhysX articulation joints.

Architectural Justification vs. Built-in Tendons

Isaac Sim/PhysX 5 provides native “spatial tendon” primitives designed for muscle-like actuation. However, they were deliberately not used in this architecture for two primary reasons. First, spatial tendons frequently induce numerical instabilities and explosions when simulating extremely stiff linkages like the tensegrity arm mechanism and carbon-fiber structure. Second, PhysX excludes native tendon tension signals from its standard `joint_force_report` Application Programming Interface (API). This makes it mechanically impossible to extract or penalize individual cable tensions during standard RL step loops; the custom `ActionTerm` approach provides direct, observable control over the force space, perfectly suiting standard RL reward formulations.

4.4.2 Physical Body-Force Tendon Model

The physical tendon mode increases fidelity by modeling the elbow tendons as body forces applied at the actual cable attachment points on the antiparallelogram linkage bars, rather than abstracting them as scalar torques through a constant Jacobian. This produces a configuration-dependent torque that naturally captures the nonlinear transmission characteristics of the four-bar mechanism.

Elbow body forces

Each of the two elbow tendons is defined by an attachment point on the root link and a corresponding point on the forearm side of the linkage. At each simulation step, the force direction is computed as the unit vector from attachment to anchor, and the force magnitude is set to the cable tension output by the policy. The resulting force is applied through PhysX’s body-force API directly on the rigid-body link prim. The net torque about the effective elbow axis depends on the lever arm at the *current* linkage configuration rather than the zero-configuration constant.

Wrist torques

The wrist retains the constant J^T mapping from Equation 17 because the cable-driven wrist’s deviation from constant behavior over its $\pm 50^\circ$ range is small. The principal effect is captured by the structure matrix [47].

Implementation

The physical tendon model is implemented as a custom `ActionTerm` class in the IsaacLab extension. During each environment step, the term:

1. Reads the five cable-tension actions from the policy (clipped to $[0, T_{\max}]$).

2. For tendons 1–2 (elbow): retrieves current link poses from the simulation, computes force vectors between attachment points, and applies body forces via the PhysX `apply_body_force` API.
3. For tendons 3–5 (wrist): computes joint torques via Equation 17 and applies them as joint efforts using IsaacLab’s `set_joint_effort_target` API.
4. Applies PD position control to the two prismatic base joints (which are not tendon-driven).

4.4.3 Actuation Mode Comparison

Three actuation modes are implemented for systematic comparison. Table 7 summarizes their characteristics; each mode represents a different trade-off between computational cost and physical fidelity.

Mode	Elbow actuation	Wrist actuation	Elbow model
PD-driven	ImplicitActuator, position targets	ImplicitActuator	Disc approx.
Tendon-driven	Constant $J^\top$ ($r_e = 72.5$ mm)	Constant $J^\top$	Disc approx.
Physical tendon	Body-force at linkage attachment pts.	Constant $J^\top$	Antiparallelogram

Table 7: Comparison of the three actuation modes. The PD-driven mode serves as the baseline. The tendon-driven mode introduces the cable-to-joint coupling. The physical tendon mode adds configuration-dependent elbow forces and the four-bar linkage constraint.

In the **PD-driven mode**, all five joints (base Y , base Z , elbow, wrist pitch, wrist roll) use Isaac Sim’s built-in `ImplicitActuator` with position targets. The policy outputs joint position deltas Δq that are added to the current positions, and the simulator applies torques via:

$$\tau_i = K_{p,i}(q_i^* - q_i) - K_{d,i}\dot{q}_i \quad (19)$$

This mode ignores the tendon transmission entirely and treats each joint as independently actuated.

In the **tendon-driven mode**, the two base joints remain PD-controlled while the three arm joints are driven through the constant $J^\top$ mapping (Equation 18). The policy outputs seven values: two base position deltas and five cable tensions. This mode tests whether an RL policy can learn to coordinate antagonistic tendons to achieve precise joint-space or task-space targets.

In the **physical tendon mode**, the elbow model is the full antiparallelogram linkage variant (Section 4.3.4), and elbow forces are applied as body forces at the physical cable attachment points (Section 4.4.2). The wrist and base are actuated identically to the tendon-driven mode. This mode provides the highest physical fidelity and the greatest challenge for learning due to the nonlinear, configuration-dependent elbow transmission.

4.4.4 Motor Specifications and Simulation Parameters

Table 8 lists the motor and cable parameters used in simulation, derived from the Maxon EC60 datasheet and the spool geometry described in Section 4.2.4.

Parameter	Value	Source
Motor type	Maxon EC60 flat BLDC	[38]
Nominal voltage	24 V	[38]
Max. continuous torque	0.401 Nm	[38]
Spool radius r_s	5 mm	[30]
Elbow pulley ratio	3:1	[30]
Max. cable tension $T_{\max}$	500 N	Derived
Elbow effort limit	36.25 Nm	Equation 14
Wrist effort limit (per axis)	3.0 Nm	Derived
Base effort limit (per axis)	200 Nm	Configured

Table 8: Motor and actuation parameters used in the simulation model. All values are derived from the Maxon EC60 specifications and the mechanical design of the tendon routing system.

4.5 Model Validation

The simulated robot model is validated through step-response testing and workspace analysis, following the methodology established by Klein [30] for the same manipulator in the Gazebo simulator. This section describes the testing protocols and evaluation metrics. Quantitative results are presented in Section 5.

4.5.1 Step-Response Testing Protocol

Step-response tests characterize the dynamic behavior of each arm joint under controlled conditions. The protocol is designed to mirror Klein’s experimental procedure [30] to enable direct comparison between Isaac Sim, Gazebo, and hardware measurements.

Test procedure

Each arm joint receives a step position command while all other joints are held at their zero position. The joint position response is recorded at 120 Hz (matching the physics simulation rate of $\frac{1}{120}$ s substeps at decimation 2). Each test consists of four phases: a 0.5 s warm-up at zero (not recorded), a 0.5 s baseline recording at zero, a 2.5 s hold phase during which the joint tracks the step command, and a 1.0 s return-to-zero phase. The test is performed for three step amplitudes per joint.

Controller configuration

A Proportional-Integral-Derivative (PID) position controller computes the torque command for each arm joint during tendon-driven testing. The control law is:

$$\tau = K_p(q^* - q) + K_i \int_0^t (q^* - q)dt' - K_d\dot{q} + \tau_g \quad (20)$$

where q^* is the target position, q is the measured position, and τ_g is a feedforward gravity compensation term. The derivative term acts on the measured velocity ($-K_d\dot{q}$) rather than on the error derivative, which naturally filters PhysX velocity noise and avoids chattering on the wrist joints. The gravity compensation for the elbow joint is computed as:

$$\tau_g = m_e \cdot g \cdot l_c \cdot \sin(\theta_{\text{elbow}}) \quad (21)$$

with $m_e = 2.49$ kg (disc, forearm, and end-effector links combined), $g = 9.81\text{m/s}^2$, $l_c = 0.26$ m (combined center of mass from elbow axis), and θ_{elbow} the current elbow angle. For the wrist joints, gravity compensation uses $m_e = 0.40$ kg and $l_c = 0.068$ m.

The PID gains used in Isaac Sim are scaled approximately $133 \times$ from Klein's original Gazebo gains ($K_p = 0.3$, $K_i = 0.03$, $K_d = 0.02$). This scaling compensates for the different effective rotational inertia representation between the two simulators (Gazebo uses ODE/DART with generalized-coordinate dynamics; Isaac Sim uses PhysX with reduced-coordinate articulations that exhibit different numerical conditioning, yielding an effective rotational inertia of approximately $0.103 \text{ kg} \cdot \text{m}^2$ for the elbow). Table 9 summarizes the test parameters. Note that in the validation scripts, the wrist saturation bound was experimentally set to 600 N to guarantee instantaneous tracking, whereas the final robot model configuration safely restricts individual wrist limits to 500 N and 3.0 Nm.

Joint	Step amplitudes	PID gains	Saturation	Min. tension
elbow_joint	20°, 30°, 40°	$K_p = 50, K_i = 4, K_d = 2$	400 N	6 N
wrist_y_joint	10°, 20°, 30°	$K_p = 10, K_i = 1.5, K_d = 0.6$	600 N	5 N
wrist_x_joint	10°, 20°, 30°	$K_p = 10, K_i = 1.5, K_d = 0.6$	600 N	5 N

Table 9: Step-response test matrix for the tendon-driven arm. PID gains are scaled from Klein's originals [30] to compensate for simulator-specific inertia representation. The minimum tension maintains cable pre-tension on inactive tendons to prevent slack.

The **physical-linkage variant** uses moderately increased elbow gains ($K_p = 75$, $K_i = 6$, $K_d = 3$, saturation 500 N) to account for the higher effective inertia of the four-bar linkage mechanism. Wrist gains remain identical to the disc-approximation variant.

Performance metrics

Four standard metrics quantify the step-response behavior:

- **Rise time t_r** (10%–90%): The time for the response to transition from 10% to 90% of the step amplitude. It characterizes the actuator speed under the given controller gains.
- **Settling time t_s** ($\pm 2\%$ band): The time after which the response remains within $\pm 2\%$ of the steady-state value. It captures the combined effect of damping, inertia, and integrator windup.
- **Overshoot M_p** : The maximum percentage deviation above the target, indicating the degree of underdamping.
- **Normalized Root-Mean-Square Error (NRMSE)**: Defined as $\text{NRMSE} = 100 \times \|q - q^*\|_2 / (q_{\max} - q_{\min})$ over the hold phase, providing a single scalar comparison metric. The NRMSE values are compared against Klein’s reported Gazebo-vs-hardware NRMSE [30] to assess whether the Isaac Sim model achieves comparable or better agreement.

4.5.2 PD Gain Tuning

In addition to the explicit PID tendon controller, the arm joints are also tested with Isaac Sim’s built-in ImplicitActuator PD drives. The PD gains were determined through a systematic 5-phase tuning procedure:

1. **Step-response baseline** — all base (2 joints $\times$ 3 amplitudes) and arm (3 joints $\times$ 3 amplitudes) steps at the current gains.
2. **Arm damping sweep** — stiffness $K = 400$ fixed; damping D swept through $\{120, 60, 30, 25, 20, 15\}$. The optimal damping is selected by minimizing the average settling time across all arm joints.
3. **Cross-coupling check** — each arm joint steps ± 0.30 rad; base drift is monitored with a threshold of 0.01 rad/m to ensure arm motion does not destabilize the base.
4. **Gripper grasp test** — close fingers on a test cube, lift, hold, and verify grip is maintained under the tuned arm dynamics.
5. **Summary comparison** — current vs. recommended gains are presented side-by-side.

The analytical critical damping for the elbow ($J_{\text{eff}} \approx 0.22 \text{ kg} \cdot \text{m}^2$) is $D_{\text{crit}} \approx 18.7 \text{ N} \cdot \text{m} \cdot \text{s}/\text{rad}$. The initial configuration used $D = 120$ ($\zeta \approx 6.4$, massively overdamped); the damping sweep confirmed $D = 20$ as the optimal value ($\zeta \approx 1.07$, near-critically damped), providing the fastest settling without significant overshoot. The base prismatic joints use $K = 8000$, $D = 800$, tuned analogously via a damping sweep over $\{1600, 1200, 800, 600, 400, 200\}$.

4.5.3 Monte Carlo Workspace Analysis

The reachable workspace and its quality are characterized using Monte Carlo forward-kinematics sampling, following the methodology of Rastegar and Fardanesh [82], with the three per-voxel quality metrics defined in Section 2.3: reachability density, Yoshikawa manipulability, and inverse condition number.

GPU-parallelized sampling in Isaac Sim

The sampling procedure exploits Isaac Sim’s GPU-parallelized scene cloning to evaluate forward kinematics at scale. A total of $N = 2000000$ joint configurations are sampled uniformly within the joint limits from Table 5, distributed across 4,096 parallel simulation environments running on a single GPU. For each sample, Isaac Sim’s articulation API computes the forward kinematics and the 6×5 geometric Jacobian at the end-effector body in a single batched tensor operation. A gripper-tip offset of $(0, 0, -0.225) \text{ m}$ along the end-effector body’s local Z axis accounts for the Robotiq 2F-140 fingertip position. The resulting point cloud is partitioned

into a regular 3D voxel grid (cell size $\Delta = 0.02$ m), and per-voxel metrics are averaged over all sample configurations mapping to that voxel, producing a volumetric quality map of the workspace.

Model selection

Only the disc-approximation arm variant is used for workspace analysis. As demonstrated in Figure 16, the disc model underestimates the physical elbow’s reach because its fixed pivot cannot reproduce the outward shift of the antiparallelogram’s migrating instantaneous center of rotation. The disc-based workspace therefore represents a conservative lower bound on the physical robot’s reachable volume.

4.6 Reinforcement Learning Task Formulation

Two progressive RL tasks are defined on the validated robot model. The *reach* task tests fundamental motion control across actuation modes. The *cube place* task extends reach to a full manipulation sequence including grasping, transport, and release. Both tasks share a common simulation environment and use the manager-based MDP workflow provided by IsaacLab [26].

4.6.1 Simulation Environment

All tasks share a common base scene configuration (`ProjBaseSceneCfg`) that provides the physical environment for conveyor-based manipulation. Table 10 summarizes the scene elements.

Element	Details
Ground plane	Flat collision surface at $z = 0$
Dome light	Uniform scene illumination
Dual conveyor belt	4 m total length, belt surface at 0.80 m height
Target drum	Plastic drum at (0.15, 0.85, 0.0) m, diameter ≈ 0.55 m
Environment spacing	5.0 m between parallel environment replicas

Table 10: Shared scene elements inherited by all tasks. The conveyor and drum are positioned to represent an industrial pick-and-place station.

The robot is ceiling-mounted at a configurable position above the conveyor belt. This mounting configuration places the end-effector workspace directly above the belt surface, enabling top-down grasping motions representative of industrial sorting stations.

4.6.2 Reach Task

The reach task trains the robot to track arbitrary end-effector poses (position and orientation). It serves two purposes: validating that each actuation mode supports effective RL-based control, and providing baseline learning curves for cross-variant comparison.

Target generation

Targets are generated by a custom forward-kinematics-sampled pose command generator: random joint configurations within the joint limits from Table 5 are evaluated through forward kinematics, guaranteeing that every target is kinematically reachable. Targets are resampled every 4.0 s. Early resampling occurs when the position error drops below 0.05 m, preventing the policy from stalling on already-reached targets.

Observations

The policy observes: relative joint positions with additive Gaussian noise ($\sigma = 0.01$), relative joint velocities ($\sigma = 0.01$), the 7-D target pose (3-D position + 4-D quaternion), and the previous action vector. The total observation dimension varies with actuation mode: 22 (PD), 24 (tendon), 28 (physical tendon).

Actions

The action parameterization differs by actuation mode:

- **PD:** 5-D joint position deltas (scale 0.5), processed by `ImplicitActuator`.
- **Tendon:** 2 base position deltas + 5 cable tensions ($T_{\max} = 500$ N).
- **Physical tendon:** 2 base position-to-limit actions + 5 physical tendon effort commands.

Rewards

Table 11 summarizes the reward function. The design uses a combination of dense tracking penalties and a fine-grained proximity bonus to shape smooth approach behavior.

Term	Weight	Function
position_tracking	-0.2	L2 position error $\ \mathbf{p} - \mathbf{p}^*\ _2$
position_tracking_fine	+0.1	$1 - \tanh\left(\frac{\ \mathbf{p} - \mathbf{p}^*\ _2}{0.1}\right)$
orientation_tracking	-0.1	Quaternion angular error
action_rate	$-10^{-4} \rightarrow -0.005$	L2 action delta (curriculum)
joint_vel	$-10^{-4} \rightarrow -0.001$	L2 joint velocity (curriculum)

Table 11: Reach task reward terms. Arrows indicate curriculum-scheduled weight ramping. Position and orientation tracking are dense; regularization terms increase over training to discourage jerky motions.

Terminations

Episodes terminate at the 12.0 s timeout (360 steps at 30 Hz control frequency) or upon joint velocity divergence (> 100 rad/s, indicating simulation instability).

Curriculum

The regularization weights for `action_rate` and `joint_vel` ramp linearly from their initial (low) values to their final (higher) values over 4,500 environment steps. This permits aggressive early exploration before penalizing high-frequency motion.

Variants

Five robot/actuation configurations are trained under identical PPO hyperparameters: Tenseg-ity PD, Tenseg-ity Tendon, Tenseg-ity Physical Tendon, UR10e PD, and Kinova Gen3 PD. The

latter two (from the Isaac Sim asset library) provide industrial-robot baselines; their full analysis is deferred to the master thesis.

4.6.3 Cube Place Task

The cube place task extends reach to a full pick-transport-place sequence using the Robotiq 2F-140 gripper. The robot must grasp a green cube from the conveyor belt, transport it to the target drum, and release it inside.

Success criterion

An episode is successful when the green cube’s center enters the cylindrical drum volume (radius = 0.2735 m, height = 0.30 m). Episodes run the full 5 s without early success termination — the per-step success reward (weight = 100) accumulates, causing successful policies to maximize the number of time steps the cube spends inside the drum.

Observations

The policy receives 47 (PD) or 49 (tendon) observations: joint positions and velocities, end-effector pose and linear velocity, the green cube’s position relative to the gripper’s grasp center, fingertip-to-cube distance vectors, gripper closure and joint torque, cube linear and angular velocities, the cube’s position relative to the drum, and the previous action vector.

Actions

The action parameterization is:

- **PD:** 6-D — 2 base position deltas, 3 arm joint deltas, 1 binary gripper.
- **Tendon:** 8-D — 2 base position deltas, 5 cable tensions ($T_{\max} = 500$ N), 1 binary gripper.

Reward functions

The reward implements a sequential pipeline guiding the agent through approach, grasp, lift, transport, and release. A persistent `was_grasped` latch gates the transport and release rewards: once the gripper has successfully grasped and lifted the cube (verified by closure ratio, proximity, and low cube velocity), the latch activates and remains set for the remainder of the episode. This prevents reward hacking where the agent collects transport rewards without maintaining a stable grasp. Figure 20 illustrates the phased reward structure.



Figure 20: Sequential reward pipeline for the cube place task. The `was_grasped` latch persists once activated and gates all transport and release rewards, ensuring that the policy must actually maintain a grasp to collect downstream rewards.

Term	Weight	Function	Gate
reaching_object	+1	$1 - \tanh\left(\frac{d}{0.1}\right)$	—
grasping	+5	$\text{closure} \times \text{proximity}$	—
lifting_object	+8	Binary: above belt, within gripper proximity	—
height_bonus	+8	$\frac{\min(\Delta z, 0.30)}{0.30}$	—
goal_tracking	+50	$1 - \tanh\left(\frac{d_{xy}}{1.0}\right)$	was_grasped
goal_tracking_fine	+5	$1 - \tanh\left(\frac{d_{xy}}{0.20}\right)$	was_grasped
release	+10	$(1 - \text{closure}) \times \text{in}_{xy} \times \text{above}_{\text{drum}}$	was_grasped
green_in_target	+100	Cube center inside drum	was_grasped
red_in_target	−12	Red cube in drum	Red active

Table 12: Cube place task reward terms. Weights are tuned to create a clear incentive gradient from approach to release. Transport rewards are gated by the was_grasped latch to prevent reward hacking.

Terminations

Episodes run the full 5.0 s (250 steps at 25 Hz). Truncation occurs on joint velocity divergence or belt collision (end-effector penetration > 0.20 m below belt surface).

Curriculum

Two curriculum events are scheduled: (1) a red distractor cube is introduced at 100,000 environment steps, testing color-selective grasping behavior; (2) action-rate and joint-velocity regularization weights increase at 200,000 steps, encouraging smoother motions once the basic task is learned.

4.6.4 Cube Sort Task (Outlook)

The cube sort task extends cube place to a multi-object scenario with eight green and eight red cubes on a moving conveyor belt. The robot must selectively pick green cubes and place them in the drum while ignoring red cubes. This task introduces object discrimination, dynamic planning, and multi-grasp sequencing. Full formulation and training results are deferred to the master thesis.

4.7 Training Infrastructure

This section describes the PPO configuration, simulation parameters, and hardware setup used for all experiments.

4.7.1 PPO Configuration

All tasks use PPO [88] from the `skrl` library [89] with GAE [87] for advantage estimation. Table 13 lists the principal hyperparameters. The complete configuration is provided in Section B.

Parameter	Value	Notes
Policy / value network	[64, 64] / [256, 128, 64] ELU	Reach / Cube Place
Rollout horizon	24 / 48 steps	Reach / Cube Place
Learning epochs	5	Per rollout batch
Mini-batches	4 / 8	Reach / Cube Place
Learning rate	10^{-3} / 10^{-4}	KL-adaptive scheduler
Discount γ	0.99	—
GAE λ	0.95	—
Training environments	4,096	Parallel GPU instances
Evaluation environments	50	Separate evaluation pass

Table 13: PPO hyperparameter summary. All robot/actuation variants within a given task share identical hyperparameters to ensure a fair comparison.

4.7.2 Simulation Parameters

Parameter	Reach task	Cube place task
Physics timestep	$\frac{1}{60}$ s	$\frac{1}{50}$ s
Decimation factor	2	2
Effective control frequency	30 Hz	25 Hz
Episode length	12.0 s (360 steps)	5.0 s (250 steps)
Parallel environments	4,096 (train) / 50 (eval)	4,096 (train) / 50 (eval)

Table 14: Simulation parameters per task. The physics timestep and episode length are adapted to the complexity of each task. The decimation factor holds the number of physics substeps per control step.

4.7.3 Software and Hardware

All experiments are conducted on a single workstation equipped with an NVIDIA RTX A6000 GPU (48 GB VRAM), running Isaac Sim 5.1.0, IsaacLab (main branch, March 2026 snapshot), and `skrl` 1.3. The robot model is stored as USD assets in the repository, and tasks are registered as Gymnasium environments within the `tensegrity_pick` IsaacLab extension. Training is managed through `skrl`'s sequential trainer, with TensorBoard logging for reward curves, episode metrics, and custom diagnostic signals. A custom monitoring application allows the live tracking of all relevant data in a Text-based User Interface (TUI) or as a web application.

5 Results

This chapter presents the quantitative and qualitative results of the model validation and RL experiments. All results are produced by the automated training and plotting pipeline described in Section 4.

5.1 Model Validation Results

5.1.1 Step-Response Analysis

Table 15 compares the NRMSE values obtained by the Isaac Sim PD and tendon-driven models against Klein’s Gazebo-vs-real-robot reference values [30]. Both Isaac Sim models achieve comparable or better NRMSE across all joints and amplitudes. The PD model (using the tuned `ImplicitActuator` with $K = 400$, $D = 20$) exhibits the lowest NRMSE due to its direct joint-level control, while the tendon model introduces the physically realistic cable-mediated dynamics at a modest accuracy cost.

Joint	Amplitude	Klein (2023)	PD (Isaac)	Tendon (Isaac)
elbow_joint	20°	39.1 %	9.5 %	12.9 %
elbow_joint	30°	11.7 %	9.5 %	15.0 %
elbow_joint	40°	12.1 %	9.5 %	17.0 %
wrist_y_joint	10°	19.2 %	8.3 %	9.7 %
wrist_y_joint	20°	11.6 %	8.3 %	12.4 %
wrist_y_joint	30°	12.5 %	8.3 %	14.8 %
wrist_x_joint	10°	25.5 %	8.3 %	9.7 %
wrist_x_joint	20°	9.9 %	8.3 %	12.4 %
wrist_x_joint	30°	11.2 %	8.3 %	14.8 %

Table 15: NRMSE comparison across actuation modes. Klein (2023) values are Gazebo-vs-real-robot from Table 4.2. PD and Tendon values are Isaac Sim disc-approximation model. Both Isaac Sim models match or outperform the Gazebo reference.

Figure 21 shows representative step-response time traces for the elbow and wrist joints under PD and tendon control. The PD model settles rapidly with near-critical damping ($\zeta \approx 1.07$), while the tendon model exhibits a marginally slower rise due to the cable tension dynamics. Both models track the commanded step accurately and return to zero without residual offset.

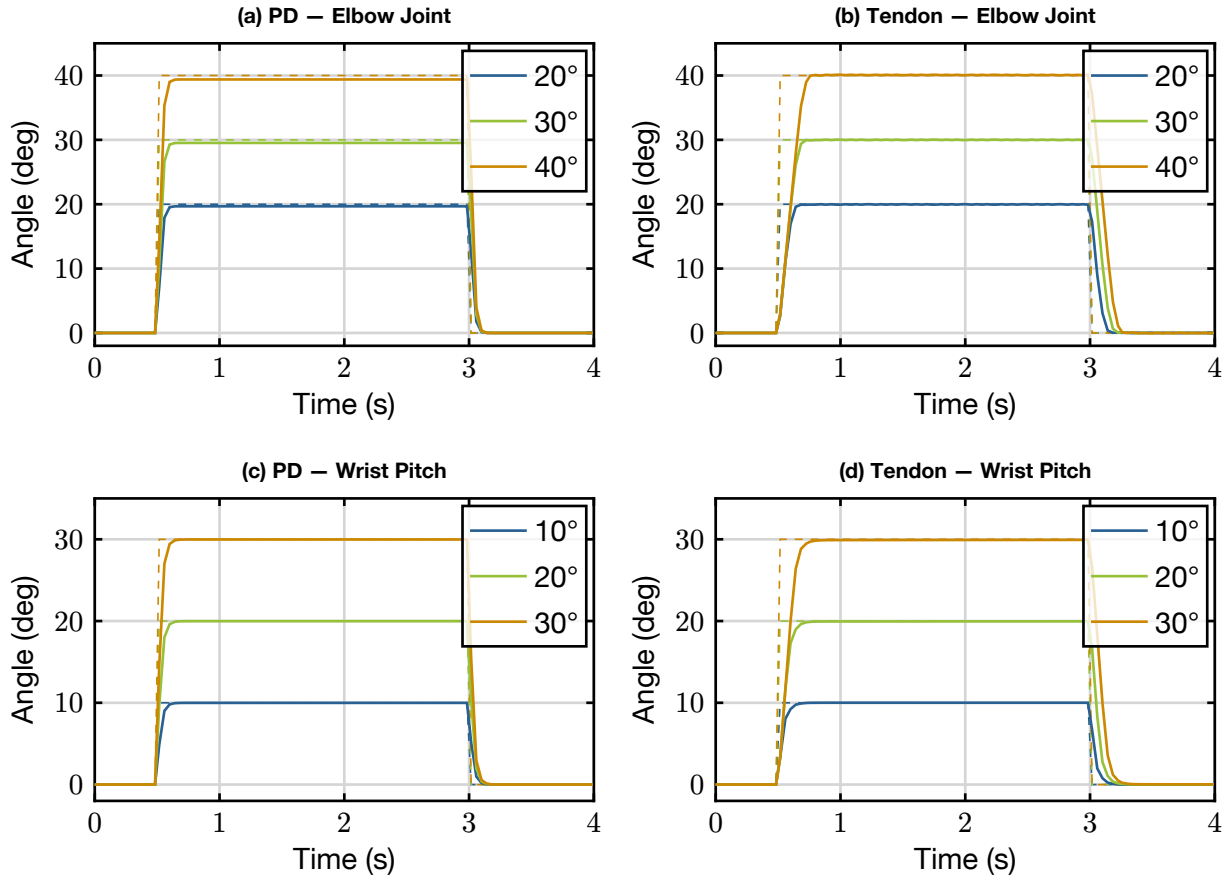


Figure 21: Step-response time traces for the elbow and wrist pitch joints under PD and tendon actuation. Dashed lines show the commanded setpoint; solid lines show the measured joint angle. **(a)** PD elbow ($K = 400$, $D = 20$); **(b)** tendon-driven elbow ($K_p = 50$, $K_i = 4$, $K_d = 2$); **(c)** PD wrist pitch; **(d)** tendon-driven wrist pitch ($K_p = 10$, $K_i = 1.5$, $K_d = 0.6$). All configurations settle accurately with no residual offset.

The physical-linkage tendon variant uses body forces on the four-bar mechanism instead of the single-DOF elbow approximation. Table 16 summarizes its performance, confirming that all joints settle within 300 ms across all amplitudes.

Joint	Amplitude	Rise (ms)	Settle (ms)	NRMSE
elbow_physical	20°	92	133	12.7 %
elbow_physical	30°	108	167	13.1 %
elbow_physical	40°	117	183	14.0 %
wrist_y_joint	10°	83	150	9.7 %
wrist_y_joint	20°	100	183	12.4 %
wrist_y_joint	30°	142	217	14.9 %
wrist_x_joint	10°	83	283	10.1 %
wrist_x_joint	20°	100	192	12.6 %
wrist_x_joint	30°	133	225	14.9 %

Table 16: Step-response metrics for the physical antiparallelogram tendon variant ($K_p = 75$, $K_t = 6$, $K_d = 3$ for elbow). All joints settle within 300 ms. NRMSE values are comparable to the disc-approximation variant, confirming correct four-bar linkage dynamics.

Figure 22 and Figure 23 provide cross-variant visual summaries of the NRMSE and settling time distributions.

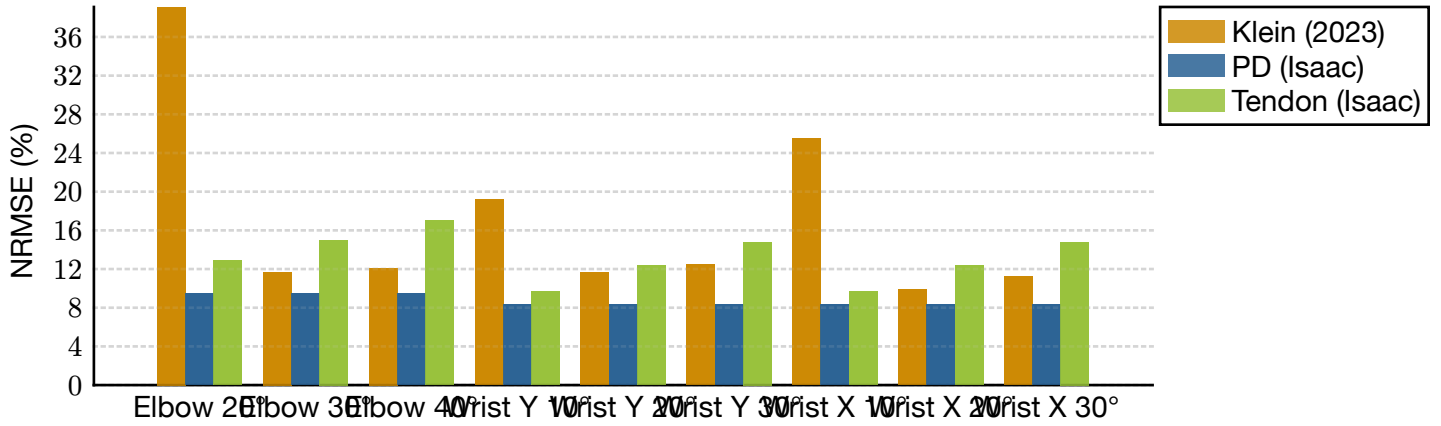


Figure 22: NRMSE comparison across all joints and actuation modes. Both Isaac Sim models (PD and tendon) consistently outperform or match the Klein (2023) Gazebo reference. The PD model achieves the lowest NRMSE due to direct joint-level actuation.

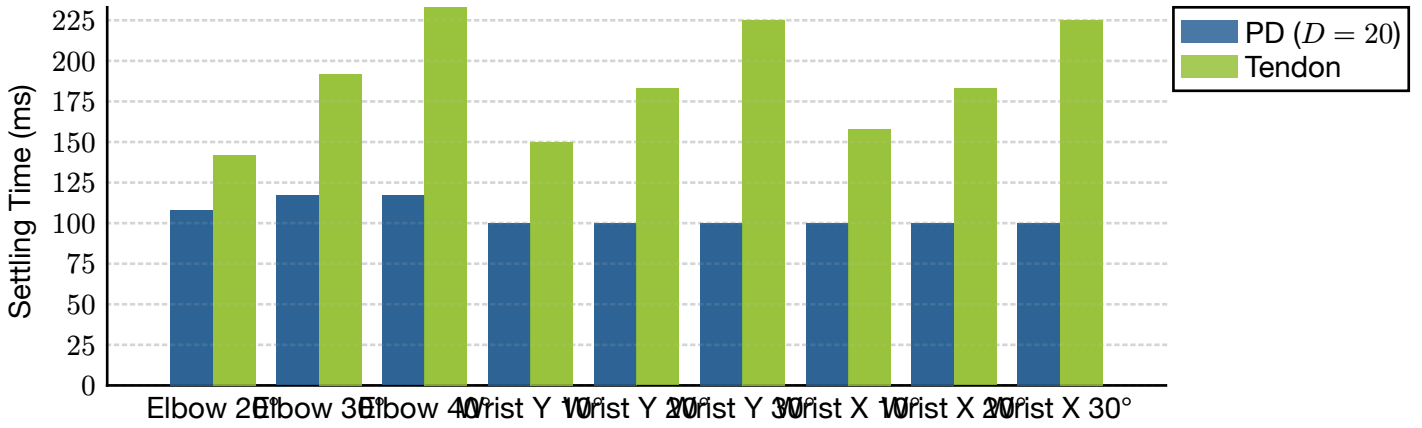


Figure 23: Settling time ($\pm 2\%$ band) across all joints and actuation modes. All configurations settle within 300 ms, meeting the design target of 0.10–0.30 s.

Damping sweep. Figure 24 illustrates the effect of the PD damping parameter on the elbow step-response. The sweep from $D = 120$ (heavily overdamped) to $D = 15$ (slightly underdamped) confirms $D = 20$ as the optimal trade-off between settling speed and overshoot, consistent with the analytical prediction ($D_{\text{crit}} \approx 18.7$).

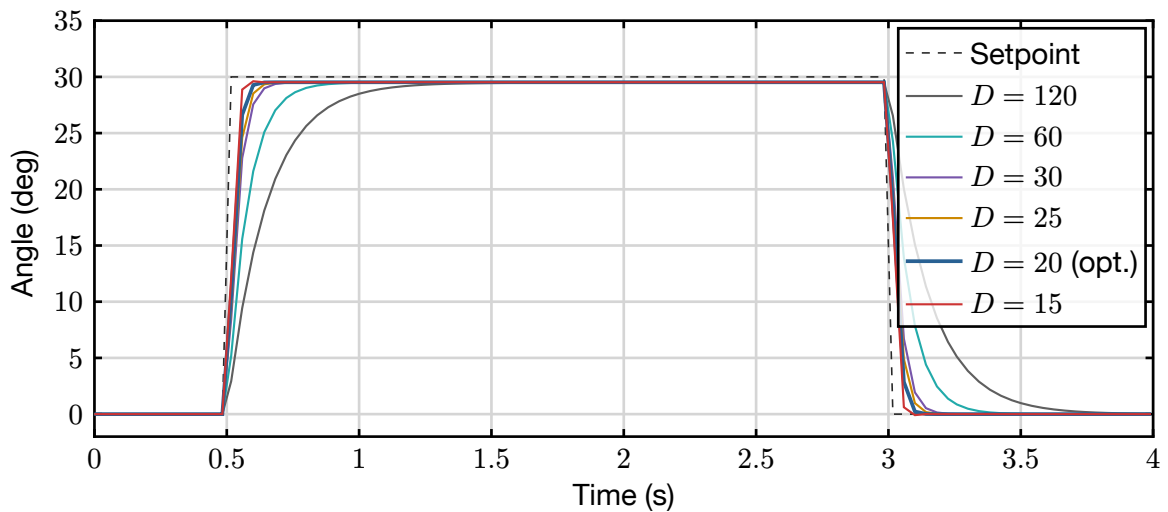


Figure 24: PD damping sweep for the elbow joint ($K = 400$ fixed, 30° step). Each curve corresponds to a different damping value. $D = 20$ (near-critically damped, $\zeta \approx 1.07$) achieves the fastest settling without significant overshoot. $D = 120$ (original) is heavily overdamped.

Base joint validation. The two prismatic base joints use PD control ($K = 8000$, $D = 800$) with gravitational feedforward for the Z axis. Table 17 summarizes the results: settling times are below 200 ms and steady-state errors are negligible ($\{<\}0.003$ mm).

Joint	Amplitude	Rise (ms)	Settle (ms)	NRMSE	SS Error
base_y_joint	50 mm	92	183	11.2 %	0.002 mm
base_y_joint	100 mm	92	183	11.2 %	0.002 mm
base_y_joint	200 mm	92	183	11.3 %	0.001 mm
base_z_joint	30 mm	100	183	11.2 %	0.001 mm
base_z_joint	60 mm	100	183	11.2 %	0.001 mm
base_z_joint	100 mm	100	183	11.3 %	0.001 mm

Table 17: Step-response metrics for the prismatic base joints ($K = 8000$, $D = 800$). Steady-state errors are negligible across all amplitudes, confirming accurate gravitational feedforward compensation on the Z axis.

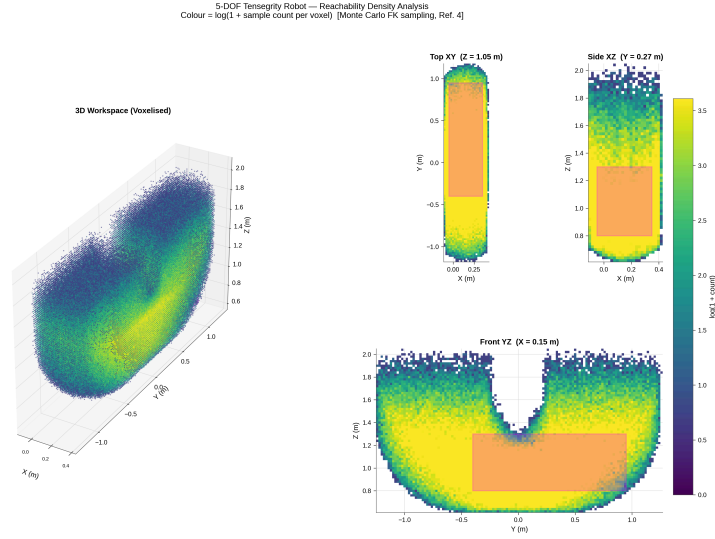
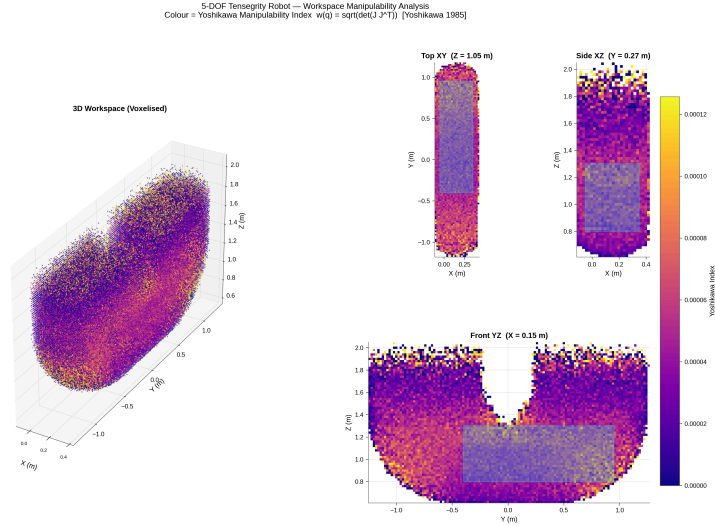
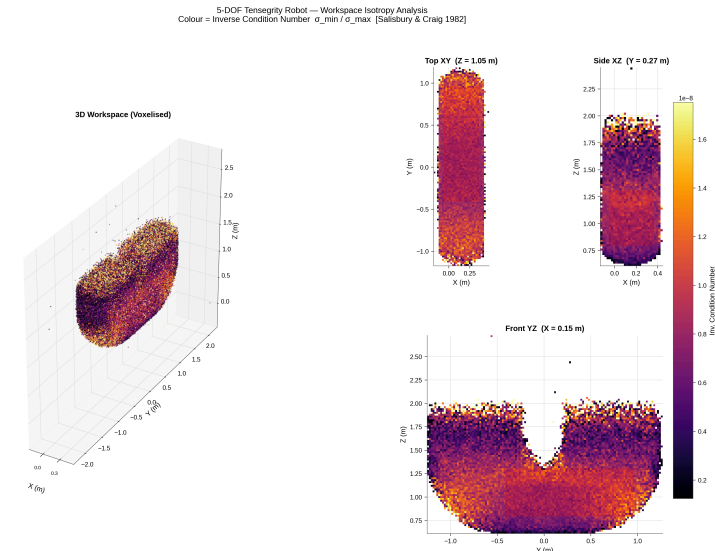
5.1.2 Workspace Analysis

The Monte Carlo workspace analysis used $N = 2000000$ FK samples with the disc-approximation arm variant, ceiling-mounted at 2.30 m, across 4,096 parallel environments. Table 18 summarizes the key statistics.

Metric	Value
Total FK samples	2000000
Samples inside desired workspace	701472 (35.1 %)
Desired workspace coverage	99.72 % (33904 / 34000 voxels)
Yoshikawa manipulability (mean, in WS)	5.2×10^{-5}
Yoshikawa manipulability (median, in WS)	1×10^{-6}
Yoshikawa manipulability (p95, in WS)	1.91×10^{-4}
Voxel size	$\Delta = 0.02$ m

Table 18: Monte Carlo workspace analysis statistics for the disc-approximation model. The desired workspace volume ($0.40 \times 1.35 \times 0.50 \text{ m}^3$, derived from the Klein (2023) task geometry) is covered to 99.7 %, confirming that the ceiling-mount height and base positioning provide adequate reach for all target tasks.

Figure 25 presents the three workspace quality metrics defined in Section 2.3: reachability density, Yoshikawa manipulability, and inverse condition number. Each subplot shows a 3D voxelised scatter plot alongside three orthogonal cross-section heatmaps through the center of the desired workspace.

(a) Reachability density**(b) Yoshikawa manipulability****(c) Inverse condition number****Figure 25:** Workspace quality metrics for the disc-approximation model (2M FK samples, 4096 parallel envs).

(a) Reachability density: high density near the workspace center indicates many joint configurations map to the task-relevant region. **(b)** Yoshikawa manipulability: highest near the center, decreasing toward boundaries. The low absolute values ($\mathcal{O}(10^{-5})$) reflect the rank-deficient 6×5 Jacobian rather than indicating poor dexterity within the robot's 5-dimensional motion capability. **(c)** Inverse condition number ($\kappa^{-1} = \sigma_{\min} / \sigma_{\max}$): near-zero values ($\mathcal{O}(10^{-8})$) throughout confirm the expected rank deficiency

The disc-approximation model achieves near-complete coverage (99.7 %) of the desired workspace. As discussed in Section 4.3.3 and Figure 16, this represents a conservative estimate: the physical antiparallelogram elbow can reach farther than the disc model at equivalent joint angles, so the true coverage of the physical robot is expected to be at least as high.

5.2 Reach Task Results

5.2.1 Training Convergence

Figure 26 compares the training convergence of the three tensegrity variants on the reach task. The PD and tendon variants, trained for 48k timesteps, converge rapidly to total rewards above $+0.75$, with the PD model reaching $+0.79$ and the tendon model $+0.76$. The physical tendon variant, trained for 150k timesteps with the dual-robot FK sampling architecture (Section 4), converges more slowly to -0.46 , reflecting the inherent difficulty of effort-based cable control through the four-bar linkage.

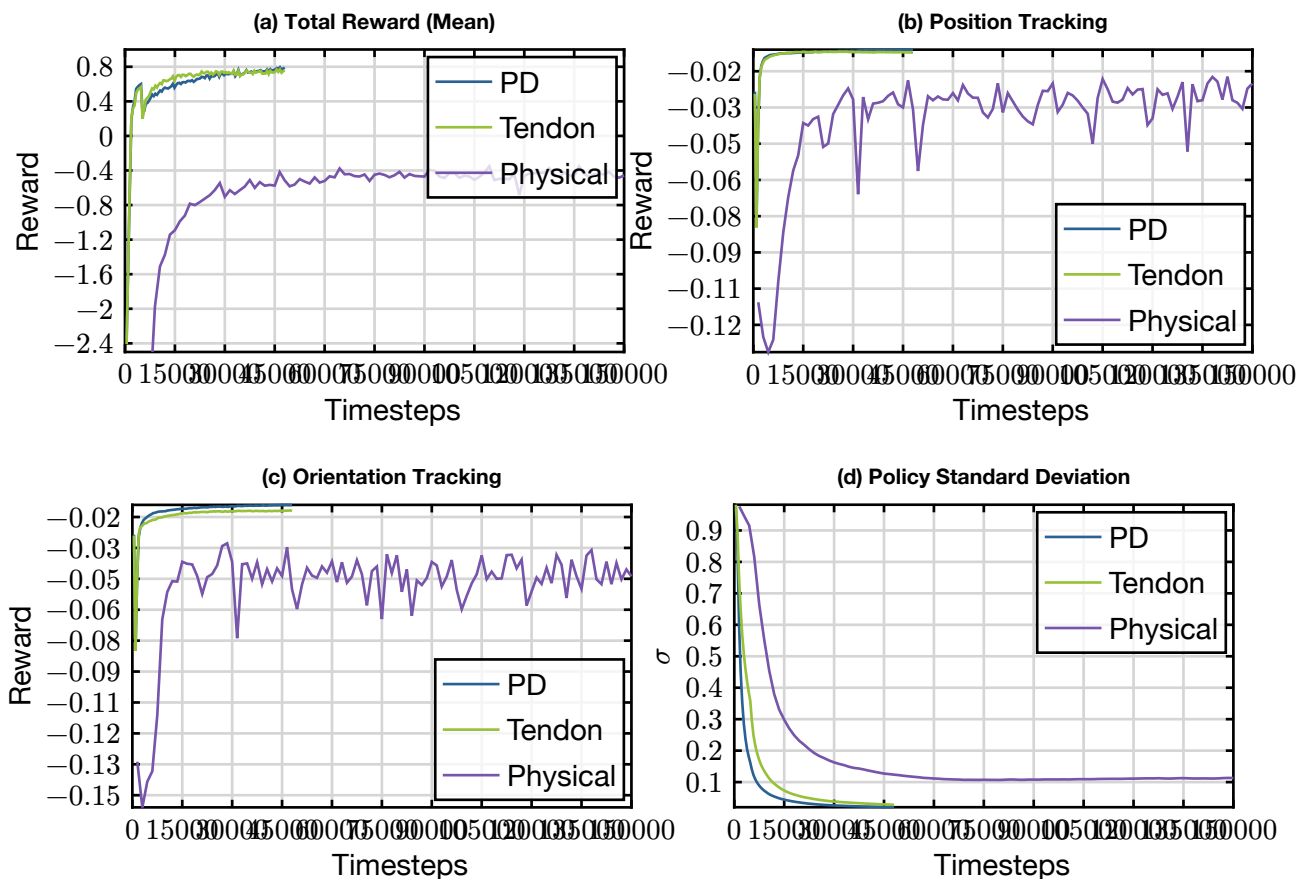


Figure 26: Reach task training convergence for the three tensegrity variants. **(a)** Mean total reward over training timesteps. **(b)** Position tracking reward component. **(c)** Orientation tracking reward component. **(d)** Policy standard deviation (σ), indicating exploration level. PD and tendon variants converge within 48k steps; the physical tendon variant requires 150k steps due to the effort-based control indirection.

5.2.2 Final Performance

Table 19 summarizes the final performance metrics from the reach task training runs. The PD and tendon variants achieve comparable performance, with position tracking errors below 1 cm. The physical tendon variant achieves 2 cm position error, limited by the cable tension control indirection inherent to the four-bar linkage mechanism.

Variant	Steps	Total Re-ward	Pos. Tracking	Orient. Tracking	Fine-Grained	Policy σ
Tensegrity PD	48k	+0.79	−0.006	−0.009	0.087	0.020
Tensegrity Tendon	48k	+0.76	−0.007	−0.012	0.085	0.028
Tensegrity Physical	150k	−0.46	−0.020	−0.044	0.041	0.113

Table 19: Reach task final performance metrics for the three tensegrity variants. Position and orientation tracking are per-step reward components (higher = better). Fine-grained measures sub-centimeter accuracy. The physical variant's larger policy σ indicates it has not yet fully converged.

5.2.3 Cross-Variant Comparison

Figure 27 provides a direct comparison of the final reach task metrics across all three variants. The PD and tendon-driven models perform almost identically on task metrics, while the physical tendon variant shows a clear performance gap, particularly in orientation tracking. This gap is attributable to the effort-based control through the antiparallelogram mechanism, which introduces a nonlinear mapping between actuator commands and joint motion.

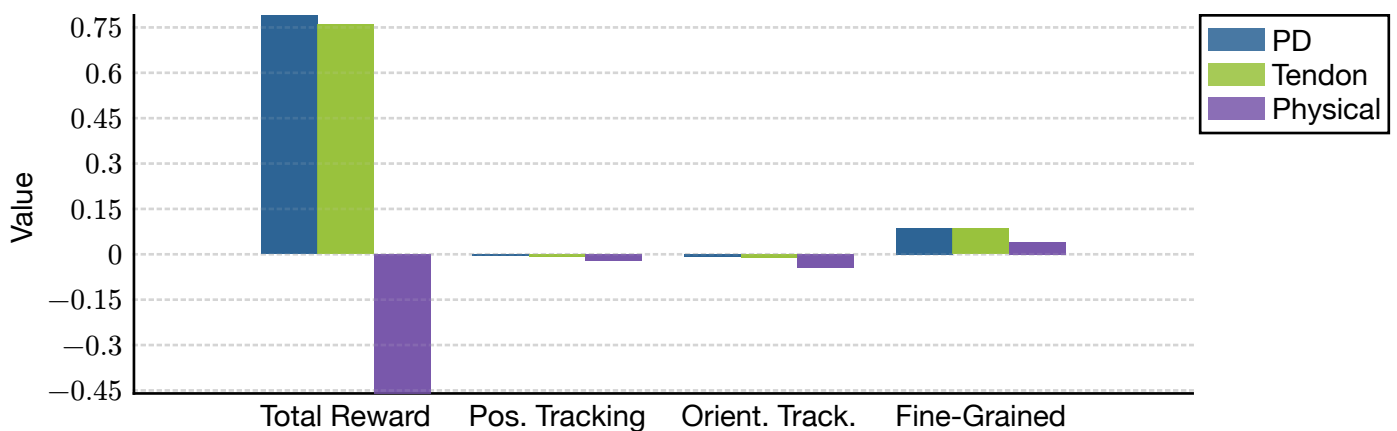


Figure 27: Cross-variant comparison of final reach task performance. The PD and tendon models achieve nearly identical scores, while the physical tendon variant shows reduced accuracy due to the effort-based cable actuation through the four-bar linkage.

5.3 Cube Place Results

5.3.1 Training Convergence

Figure 28 presents the training convergence for the cube place task across all three tensegrity variants. The PD and tendon variants both reach total rewards above 330 within 300k timesteps, demonstrating successful learning of the full grasp-lift-place pipeline. The physical tendon variant fails to learn the grasping sub-task, achieving a mean reward of only 58.9, consistent with its lower policy convergence on the reach task.

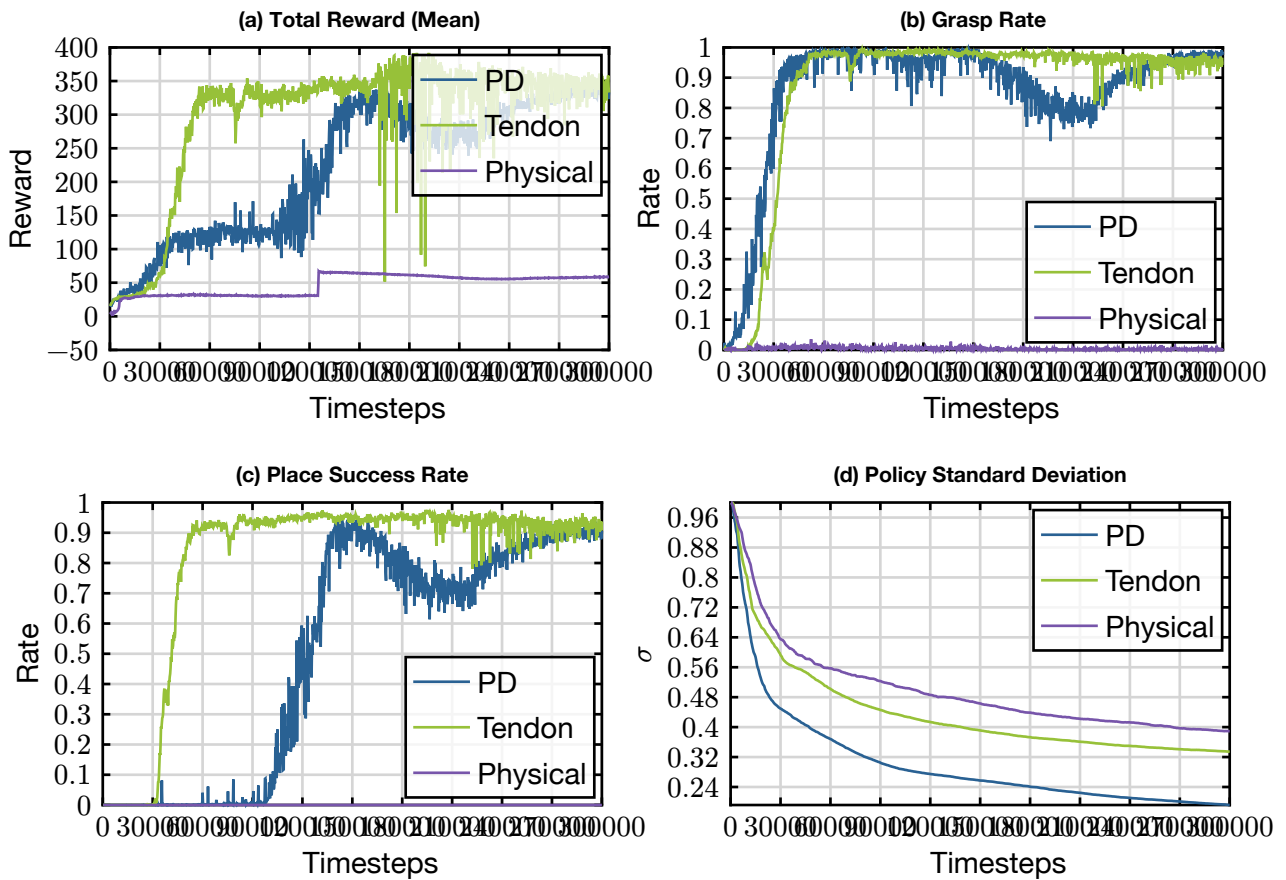


Figure 28: Cube place training convergence for the three tensegrity variants. **(a)** Mean total reward over training timesteps. **(b)** Grasp success rate. **(c)** Place success rate (cube placed in target drum). **(d)** Policy standard deviation. Both the PD and tendon variants converge to high task success within 300k steps; the physical tendon variant does not acquire the grasping skill.

5.3.2 Final Performance

Table 20 summarizes the cube place task evaluation metrics at the end of training. The PD and tendon variants achieve grasp rates above 94% and place success rates above 90%, while maintaining red cube safety above 95%. The physical tendon variant does not learn to grasp, consistent with its difficulty in precise position control.

Variant	Steps	Total Reward	Grasp Rate	Place Success	Red Safe
Tensegrity PD	300k	333.3	98.7%	90.6%	98.0%
Tensegrity Tendon	300k	332.9	94.7%	93.4%	95.2%
Tensegrity Physical	300k	58.9	0.5%	0.0%	97.8%

Table 20: Cube place final performance metrics. Grasp Rate: fraction of episodes achieving a stable grasp. Place Success: fraction of episodes where the green cube is placed in the target drum. Red Safe: fraction where the red distractor remains on the conveyor.

5.3.3 Curriculum Effect

Figure 29 illustrates the effect of the two curriculum transitions on training progression for the PD variant. At 100k timesteps, the red distractor cube is introduced, causing a brief dip in reward as the policy encounters a new obstacle. At 200k timesteps, action rate and joint velocity penalties ramp up, promoting smoother trajectories at a slight cost to peak reward. Both transitions demonstrate the curriculum design working as intended: the policy first acquires core skills, then learns robustness.

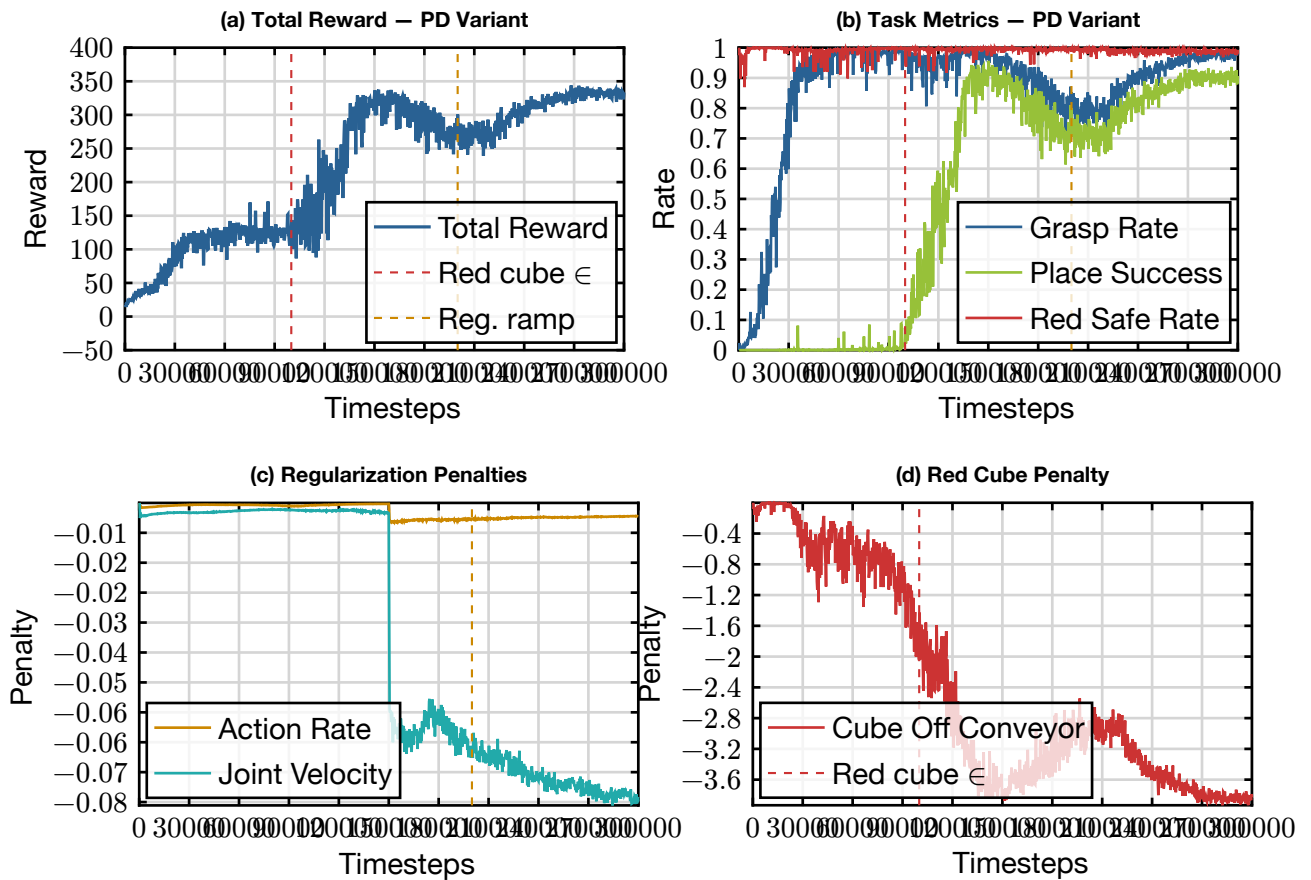


Figure 29: Effect of curriculum stages on the PD variant's cube place training. **(a)** Total reward with dashed lines marking the red cube introduction (100k) and regularization ramp (200k). **(b)** Task success rates showing brief transient dips at curriculum transitions. **(c)** Regularization penalty terms ramping up after 200k steps. **(d)** Red cube off-conveyor penalty activating after 100k steps.

5.4 Cube Sort Preliminary Results

The cube sort task represents an extension of the cube place pipeline to sequential multi-object sorting from a conveyor belt. Figure 30 shows preliminary training results using the PD variant only, as training is ongoing. The agent begins to learn approach and grasp behaviors within 77k timesteps, though reliable sorting has not yet been achieved, as indicated by the still-increasing reward trajectory.

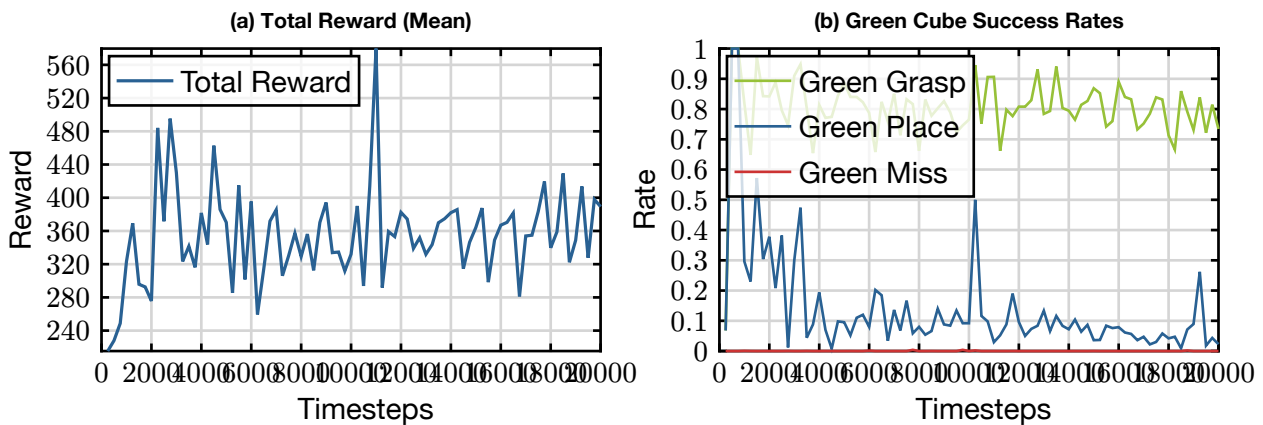


Figure 30: Preliminary cube sort training results using the PD tensegrity variant (77k timesteps, training ongoing). **(a)** Total reward showing an upward trend. **(b)** Green cube grasp, placement, and miss rates. The agent is beginning to acquire approach and grasp behaviors but has not yet achieved reliable sorting.

5.5 Summary of Key Results

The experimental evaluation yields three main findings:

1. **PD and tendon variants achieve comparable RL performance.** On the reach task, both variants converge within 48k timesteps to total rewards above $+0.75$, with position tracking errors below 1cm. On the cube place task, both achieve grasp rates above 94% and place success rates above 90%. The cable-mediated tendon transmission does not introduce a measurable performance penalty in the RL setting, suggesting that the disc-approximation tension mapping is sufficiently transparent to the policy.
2. **The physical tendon variant presents a significant control challenge.** With effort-based actuation through the four-bar antiparallelogram linkage, the physical variant converges more slowly and to lower final performance on both tasks. On reach, it achieves a total reward of -0.46 (compared to $+0.79$ for PD) with $3 \times$ the policy uncertainty. On cube place, it fails to acquire the grasping skill entirely. This result highlights the difficulty of RL with indirect force transmission and motivates future work on hierarchical control architectures.
3. **Curriculum learning enables robust multi-stage manipulation.** The staged introduction of the red distractor cube and regularization penalties in the cube place task produces

smooth curriculum transitions with only brief transient performance dips, validating the curriculum design described in Section 4.

6 Discussion

6.1 Evaluation of Outcomes

6.2 Comparison with Related Work

6.3 Limitations of the Study

6.4 Practical Implications

6.5 Recommendations and Lessons Learned

7 Conclusion

7.1 Conclusion

7.2 Outlook

Bibliography

- [1] ALLSHIRE, Arthur, MITTAL, Mayank, LODAYA, Varun, MAKОВIYCHUK, Viktor, MAKОВIICHUK, Denys, WIDMAIER, Felix, WÜTHRICH, Manuel, BAUER, Stefan, HANDA, Ankur and GARG, Animesh. Transferring Dexterous Manipulation from GPU Simulation to a Remote Real-World TriFinger. In : *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2022. p. 11802–11809. DOI [10.1109/IROS47612.2022.9981586](https://doi.org/10.1109/IROS47612.2022.9981586).
- [2] BICCHI, Antonio and TONIETTI, Giovanni. Fast and "Soft-Arm" Tactics: Dealing with the Safety-Performance Tradeoff in Robot Arms Design and Control. *IEEE Robotics & Automation Magazine*. 2004. Vol. 11, no. 2, p. 22–33. DOI [10.1109/MRA.2004.1310939](https://doi.org/10.1109/MRA.2004.1310939).
- [3] BOSSCHER, Paul, RIECHEL, Andrew T. and EBERT-UPHOFF, Imme. Wrench-Feasible Workspace Generation for Cable-Driven Robots. *IEEE Transactions on Robotics*. 2006. Vol. 22, no. 5, p. 890–902. DOI [10.1109/TRO.2006.878967](https://doi.org/10.1109/TRO.2006.878967).
- [4] BRUCKMANN, Tobias, POTT, Andreas and HILLER, Manfred. Calculating Force Distributions for Redundantly Actuated Tendon-Based Stewart Platforms. *Advances in Robot Kinematics*. 2008.
- [5] BRYANT, John and SANGWIN, Christopher. *How Round Is Your Circle? Where Engineering and Mathematics Meet*. Princeton University Press, 2008. ISBN 978-0-691-13118-4.
- [6] COUMANS, Erwin. Bullet Physics Simulation. In : *ACM SIGGRAPH 2015 Courses*. 2015. DOI [10.1145/2776880.2792704](https://doi.org/10.1145/2776880.2792704). Course material
- [7] CRAIG, John J. *Introduction to Robotics: Mechanics and Control*. 3rd. Pearson Prentice Hall, 2005.
- [8] DEVLIN, Sam and KUDENKO, Daniel. Dynamic Potential-Based Reward Shaping. In : *Proceedings of the 11th International Conference on Autonomous Agents and Multi-agent Systems (AAMAS)*. 2012. p. 433–440.
- [9] DHAKATE, Rohit, JANTOS, Thomas, ALLAK, Elvin, WEISS, Stephan and STEINBRENER, Jan. CaRoSaC: A Reinforcement Learning-Based Kinematic Control of Cable-Driven Parallel Robots by Addressing Cable Sag Through Simulation. *IEEE Robotics and Automation Letters*. 2025. DOI [10.1109/LRA.2025.3564225](https://doi.org/10.1109/LRA.2025.3564225).
- [10] DIJKSMAN, Evert A. On the Cognates of the Antiparallelogram. *ASME Journal of Engineering for Industry*. 1977. Vol. 99, no. 3. DOI [10.1115/1.3439273](https://doi.org/10.1115/1.3439273).
- [11] DONG, Hui, DU, Zhijiang and CHIRIKJIAN, Gregory S. Workspace density and inverse kinematics for planar serial revolute manipulators. *Mechanism and Machine Theory*. 2013. Vol. 70, p. 508–522. DOI [10.1016/j.mechmachtheory.2013.08.008](https://doi.org/10.1016/j.mechmachtheory.2013.08.008).
- [12] FASQUELLE, Benjamin, FURET, Matthieu, KHANNA, Priyanka, CHABLAT, Damien, CHEVALLEREAU, Christine and WENGER, Philippe. A Bio-Inspired 3-DOF Light-Weight Manipulator with Tensegrity X-Joints. In : *IEEE International Confer-*

- ence on Robotics and Automation (ICRA). 2020. p. 5054–5060. DOI [10.1109/ICRA40945.2020.9196589](https://doi.org/10.1109/ICRA40945.2020.9196589).
- [13] FASQUELLE, Benjamin, KHANNA, Priyanka, CHEVALLEREAU, Christine, CHABLAT, Damien, CREUSOT, Denis, JOLIVET, Sylvain, LEMOINE, Philippe and WENGER, Philippe. Identification and Control of a 3-X Cable-Driven Manipulator Inspired From the Bird's Neck. In :. 2022. p. 11005. DOI [10.1115/1.4051521](https://doi.org/10.1115/1.4051521).
- [14] FURET, Matthieu and WENGER, Philippe. Kinetostatic Analysis and Actuation Strategy of a Planar Tensegrity 2-X Manipulator. *ASME Journal of Mechanisms and Robotics*. 2019. Vol. 11, no. 6, p. 60904. DOI [10.1115/1.4044209](https://doi.org/10.1115/1.4044209).
- [15] GARRIDO CAMPOS, Julio, SILVA-MUÑIZ, Daniel, RIVERA-ANDRADE, Jorge and RIVEIRO, Eva. Integrated Simulation and Control Environment for the Development and Safe Start-up of Cable Driven Parallel Robots. *International Journal of Computer Integrated Manufacturing*. 2024. Vol. 38, no. 7. DOI [10.1080/0951192X.2024.2389640](https://doi.org/10.1080/0951192X.2024.2389640).
- [16] GOUTTEFARDE, Marc and GOSSELIN, Clément M. Analysis of the Wrench-Closure Workspace of Planar Parallel Cable-Driven Mechanisms. *ASME Journal of Mechanical Design*. 2006. Vol. 128, no. 1, p. 264–272.
- [17] GUIST, Simon, SCHNEIDER, Jonas, MA, Hongyi, CHEN, Lei, BERENZ, Vincent, MARTUS, Jan, OTT, Heiner, GRÜNINGER, Franziska, MUEHLEBACH, Michael, FIENE, Jonathan, SCHÖLKOPF, Bernhard and BÜCHLER, Dieter. Safe & Accurate at Speed with Tendons: A Robot Arm for Exploring Dynamic Motion. In : *Robotics: Science and Systems (RSS) XX*. 2024.
- [18] HADDADIN, Sami, ALBU-SCHÄFFER, Alin and HIRZINGER, Gerd. Requirements for Safe Robots: Measurements, Analysis and New Insights. *The International Journal of Robotics Research*. 2009. Vol. 28, no. 11–12, p. 1507–1527. DOI [10.1177/0278364909343970](https://doi.org/10.1177/0278364909343970).
- [19] HAMON, Arnaud, AOUSTIN, Yannick and CARO, Stéphane. Cross Four-Bar Linkage for the Knees of a Planar Bipedal Robot. In : *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2010. DOI [10.1109/IROS.2010.5686851](https://doi.org/10.1109/IROS.2010.5686851).
- [20] HANDA, Ankur, ALLSHIRE, Arthur, MAKОВIYCHUK, Viktor, PETRENKO, Aleksei, SINGH, Ritvik, LIU, Jingzhou, MAKОВIICHUK, Denys, VAN WYK, Karl, ZHURKEVICH, Alexander, SUNDARALINGAM, Balakumar, NARANG, Yashraj, LAFLECHE, Jean-Francois, FOX, Dieter and STATE, Gavriel. DeXtreme: Transfer of Agile In-hand Manipulation from Simulation to Reality. In : *IEEE International Conference on Robotics and Automation (ICRA)*. 2023. p. 5977–5984. DOI [10.1109/ICRA48891.2023.10161417](https://doi.org/10.1109/ICRA48891.2023.10161417).
- [21] THE HARDWARE_INTERFACE CONTRIBUTORS. Joint Kinematics for ros2_control. Online. 2026. [Accessed 3 March 2026]. Available from: https://docs.ros.org/en/rolling/p/hardware_interface/doc/joints_userdoc.htmlPackage documentation describing mimic joints and limitations of serial-chain URDF kinematics (accessed 2026-03-03).
- [22] HARTENBERG, Richard S. and DENAVIT, Jacques. *Kinematic Synthesis of Linkages*. McGraw-Hill, 1964.

- [23] THE ISAAC LAB PROJECT DEVELOPERS. Creating a Manager-Based RL Environment. Online. 2026. [Accessed 3 March 2026]. Available from: https://isaac-sim.github.io/IsaacLab/v1.0.0/source/tutorials/03_envs/create_manager_rl_env.html Isaac Lab Documentation, last updated Mar 03, 2026
- [24] THE ISAAC LAB PROJECT DEVELOPERS. Multi-GPU and Multi-Node Training. Online. 2026. [Accessed 3 March 2026]. Available from: https://isaac-sim.github.io/IsaacLab/main/source/features/multi_gpu.html
- [25] THE ISAAC LAB PROJECT DEVELOPERS. Importing a New Asset. Online. 2026. [Accessed 3 March 2026]. Available from: https://isaac-sim.github.io/IsaacLab/main/source/how-to/import_new_asset.html Isaac Lab documentation; last updated on Mar 03, 2026.
- [26] THE ISAAC LAB PROJECT DEVELOPERS. Task Design Workflows — Isaac Lab Documentation. Online. 2026. [Accessed 3 March 2026]. Available from: https://isaac-sim.github.io/IsaacLab/main/source/overview/core-concepts/task_workflows.html
- [27] JAKOBI, Nick, HUSBANDS, Phil and HARVEY, Inman. Noise and the Reality Gap: The Use of Simulation in Evolutionary Robotics. In : *Advances in Artificial Life (ECAL 1995)*. Springer, 1995. p. 704–720. Lecture Notes in Computer Science. DOI [10.1007/3-540-59496-5_337](https://doi.org/10.1007/3-540-59496-5_337).
- [28] KALASHNIKOV, Dmitry, IRPAN, Alex, PASTOR, Peter, IBARZ, Julian, HERZOG, Alexander, JANG, Eric, QUILLEN, Deirdre, HOLLY, Ethan, KALAKRISHNAN, Mrinal, VANHOUCKE, Vincent and LEVINE, Sergey. QT-Opt: Scalable Deep Reinforcement Learning for Vision-Based Robotic Manipulation. In : *Conference on Robot Learning (CoRL)*. 2018.
- [29] KATO, Takahisa, OKUMURA, Ichiro, KOSE, Hidekazu, TAKAGI, Kiyoshi and HATA, Nobuhiko. Tendon-driven continuum robot for neuroendoscopy: validation of extended kinematic mapping for hysteresis operation. *International Journal of Computer Assisted Radiology and Surgery*. 2016. Vol. 11, no. 4, p. 589–602. DOI [10.1007/s11548-015-1310-2](https://doi.org/10.1007/s11548-015-1310-2).
- [30] KLEIN, Max. *Arbeitsraumanalyse, Simulation und Bewegungsplanung eines seilgetriebenen robotischen Manipulators*. Masterarbeit. 2023.
- [31] KOBAYASHI, Hiroshi, HYODO, Kazuya and OGANE, Daisuke. On Tendon-Driven Robotic Mechanisms with Redundant Tendons. *International Journal of Robotics Research*. 1998. Vol. 17, no. 5, p. 561–571.
- [32] KOBER, Jens, BAGNELL, J. Andrew and PETERS, Jan. Reinforcement Learning in Robotics: A Survey. *The International Journal of Robotics Research*. 2013. Vol. 32, no. 11, p. 1238–1274. DOI [10.1177/0278364913495721](https://doi.org/10.1177/0278364913495721).
- [33] LESSARD, Soren, CASTRO, Dennis, ASPER, William, CHOPRA, Srishti Dali, BALTA-ADMONY, Leya B., TEODORESCU, Mircea, SUNSPIRAL, Vytas and AGOGINO, Adrian. A Bio-Inspired Tensegrity Manipulator with Multi-DOF, Structurally Compliant Joints. In : *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2016. p. 5515–5520. DOI [10.1109/IROS.2016.7759811](https://doi.org/10.1109/IROS.2016.7759811).

- [34] LILGE, Sven and BURGNER-KAHR, Jessica. How to Model Tendon-Driven Continuum Robots and Benchmark Modelling Performance. *Frontiers in Robotics and AI*. 2021. Vol. 7, p. 630245. DOI [10.3389/frobt.2020.630245](https://doi.org/10.3389/frobt.2020.630245).
- [35] MACKLIN, Miles, STOREY, Kier, LU, Michelle, TERDIMAN, Pierre, CHENTANEZ, Nuttapong, JESCHKE, Stefan and MÜLLER, Matthias. Small Steps in Physics Simulation. Online. 2019. [Accessed 3 March 2026]. Available from: <https://mmacklin.com/smallsteps.pdf>
- [36] MAKOVYCHUK, Viktor, WAWRZYNIAK, Lukasz, GUO, Yunrong, LU, Michelle, STOREY, Kyle, MACKLIN, Miles, HOELLER, David, RUDIN, Nikita, ALLSHIRE, Arthur, HANDA, Ankur, STATE, Gavriel, GIRDHAR, Ankur, KUMAR, Tushar, FINK, Jonathan, MEREL, Josh and TASSA, Yuval. Isaac Gym: High Performance GPU Based Physics Simulation for Robot Learning. In : *NeurIPS 2021 Datasets and Benchmarks Track (Round 2)*. Online. 2021. Available from: https://openreview.net/forum?id=fgFBtYgJQX_
- [37] THE MATHWORKS, INC. Generate reachable workspace of robot in environment (\texttt{generateRobotWorkspace}). Online. 2026. [Accessed 3 March 2026]. Available from: <https://www.mathworks.com/help/robotics/ref/generaterobotworkspace.html>
- [38] MAXON MOTOR AG. *EC60 flat BLDC motor, 150~W – Datasheet*. 2023. Article no. \ 625858
- [39] MAXON MOTOR AG. *EPOS4 70/15 Positioning Controller – Datasheet*. 2023.
- [40] MCCARTHY, J. Michael and SOH, Gim Song. *Geometric Design of Linkages*. 2. Springer, 2010.
- [41] MITTAL, Mayank, YU, Felix, ZHAO, Jitendra, CHENG, Yuanhao, DIMITROV, Dimitar, QIU, Louis, VELONAKI, Mari and ZHU, Yuke. Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments. *IEEE Robotics and Automation Letters*. 2023. Vol. 8, no. 6, p. 3740–3747. DOI [10.1109/LRA.2023.3270034](https://doi.org/10.1109/LRA.2023.3270034).
- [42] MIYASAKA, Muneaki, HAGHIGHIPANAH, Mohammad, LI, Yangming, MATHESON, Joseph, LEWIS, Andrew and HANNAFORD, Blake. Modeling Cable-Driven Robot With Hysteresis and Cable–Pulley Network Friction. *IEEE/ASME Transactions on Mechatronics*. 2020. Vol. 25, no. 2, p. 1095–1104. DOI [10.1109/TMECH.2020.2973428](https://doi.org/10.1109/TMECH.2020.2973428).
- [43] MURALIDHARAN, Vimallesh and WENGER, Philippe. Optimal Design and Comparative Study of Two Antagonistically Actuated Tensegrity Joints. *Mechanism and Machine Theory*. 2021. Vol. 159, p. 104249. DOI [10.1016/j.mechmachtheory.2021.104249](https://doi.org/10.1016/j.mechmachtheory.2021.104249).
- [44] MURALIDHARAN, Vimallesh, WENGER, Philippe and CHEVALLEREAU, Christine. Design Considerations and Workspace Computation of 2-X and 2-R Planar Cable-Driven Tensegrity-Inspired Manipulators. *Mechanism and Machine Theory*. 2024. Vol. 194, p. 105587. DOI [10.1016/j.mechmachtheory.2024.105587](https://doi.org/10.1016/j.mechmachtheory.2024.105587).
- [45] MURRAY, Richard M., LI, Zexiang and SASTRY, S. Shankar. *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994.
- [46] NARANG, Yashraj, STOREY, Kier, AKINOLA, Iretiayo, MACKLIN, Miles, REIST, Philipp, WAWRZYNIAK, Lukasz, GUO, Yunrong, MORAVANSZKY, Adam, STATE, Gavriel, LU,

- Michelle, HANDA, Ankur and FOX, Dieter. Factory: Fast Contact for Robotic Assembly. In : *Robotics: Science and Systems (RSS)*. 2022. DOI [10.15607/RSS.2022.XVIII.065](https://doi.org/10.15607/RSS.2022.XVIII.065).
- [47] NEMOTO, Takeru, WALTER, Jonas, BACHMANN, Christian, GERLICH, Matthias, REITELSHÖFER, Sebastian and FRANKE, Jörg. Highly Dynamic 2-DOF Cable-Driven Robotic Wrist Based on a Novel Topology. *IEEE Robotics and Automation Letters*. 2022. Vol. 7, no. 2, p. 5727–5734. DOI [10.1109/LRA.2022.3159816](https://doi.org/10.1109/LRA.2022.3159816).
- [48] NG, Andrew Y., HARADA, Daishi and RUSSELL, Stuart J. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. In : *Proceedings of the Sixteenth International Conference on Machine Learning (ICML 1999)*. Online. San Francisco, CA, USA : Morgan Kaufmann, 1999. p. 278–287. [Accessed 3 March 2026]. Available from: <https://www.cs.utexas.edu/~shivaram/readings/b2hd-NgHR1999.html>
- [49] NVIDIA. Rigid Body Dynamics — PhysX 5.1.1 Documentation. Online. 2022. [Accessed 3 March 2026]. Available from: <https://nvidia-omniverse.github.io/PhysX/physx/5.1.1/docs/RigidBodyDynamics.html>
- [50] NVIDIA. Rigid Body Collision — PhysX 5.1.0 Documentation. Online. 2022. [Accessed 3 March 2026]. Available from: <https://nvidia-omniverse.github.io/PhysX/physx/5.1.0/docs/RigidBodyCollision.html>
- [51] NVIDIA. PxMaterial — PhysX 5.1.0 API Reference. Online. 2022. [Accessed 3 March 2026]. Available from: https://nvidia-omniverse.github.io/PhysX/physx/5.1.0/_build/physx/latest/class_px_material.html
- [52] NVIDIA. Advanced Collision Detection — PhysX 5.1.2 Documentation. Online. 2022. [Accessed 3 March 2026]. Available from: <https://nvidia-omniverse.github.io/PhysX/physx/5.1.2/docs/AdvancedCollisionDetection.html>
- [53] NVIDIA. GPU Rigid Bodies. Online. 2022. [Accessed 3 March 2026]. Available from: <https://nvidia-omniverse.github.io/PhysX/physx/5.1.0/docs/GPURigidBody.html> PhysX 5.1.0 documentation, last updated Nov 08, 2022
- [54] NVIDIA. Welcome to PhysX — PhysX 5.1.3 Documentation. Online. 2023. [Accessed 3 March 2026]. Available from: <https://nvidia-omniverse.github.io/PhysX/physx/5.1.3/index.html>
- [55] NVIDIA. Articulations — PhysX 5.1.3 Documentation. Online. 2023. [Accessed 3 March 2026]. Available from: <https://nvidia-omniverse.github.io/PhysX/physx/5.1.3/docs/Articulations.html>
- [56] NVIDIA. Best Practices Guide — PhysX 5.4.1 Documentation. Online. 2024. [Accessed 3 March 2026]. Available from: <https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/docs/BestPractices.html>
- [57] NVIDIA. Simulation — PhysX 5.4.1 Documentation. Online. 2024. [Accessed 3 March 2026]. Available from: <https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/docs/Simulation.html>
- [58] NVIDIA. Geometry — PhysX 5.4.1 Documentation. Online. 2024. [Accessed 3 March 2026]. Available from: <https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/docs/Geometry.html>

- [59] NVIDIA. Tendon Visual Authoring — Omni Physics Documentation. Online. 2025. [Accessed 3 March 2026]. Available from: https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/extensions/ux/source/omni.usdphysics.ui/docs/dev_guide/tendon_authoring.html
- [60] NVIDIA. Isaac Sim: Extensions API. Online. 2025. [Accessed 3 March 2026]. Available from: <https://docs.isaacsim.omniverse.nvidia.com/5.1.0/py/index.html> Isaac Sim Documentation (v5.1.0), last updated Oct 17, 2025
- [61] NVIDIA. What Is Isaac Sim?. Online. 2026. [Accessed 3 March 2026]. Available from: <https://docs.isaacsim.omniverse.nvidia.com/5.1.0/index.html> Isaac Sim Documentation (v5.1.0), last updated Jan 28, 2026
- [62] NVIDIA. Omni PhysX — Overview. Online. 2026. [Accessed 3 March 2026]. Available from: https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/extensions/runtime/source/omni.physx/docs/index.html
- [63] NVIDIA. Physics Simulation Fundamentals. Online. 2026. [Accessed 3 March 2026]. Available from: https://docs.isaacsim.omniverse.nvidia.com/5.1.0/physics/simulation_fundamentals.html
- [64] NVIDIA. GPU Simulation (PhysX 5.4.1 Documentation). Online. 2026. [Accessed 3 March 2026]. Available from: <https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/docs/GPURigidBodies.html>
- [65] NVIDIA. Physx Joint Schema — Omni Physics Documentation. Online. 2026. [Accessed 3 March 2026]. Available from: https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/dev_guide/joints/physx_joint_schema.html Omni Physics documentation; last updated on Jan 20, 2026.
- [66] NVIDIA. Tutorial: Import URDF — Isaac Sim Documentation (5.1.0). Online. 2026. [Accessed 3 March 2026]. Available from: https://docs.isaacsim.omniverse.nvidia.com/5.1.0/importer_exporter/import_urdf.html Isaac Sim documentation; last updated on Jan 28, 2026.
- [67] NVIDIA. Known Physics Limitations — Omni Physics Documentation. Online. 2026. [Accessed 3 March 2026]. Available from: https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/dev_guide/guides/current_limitations.html Omni Physics documentation; last updated on Jan 20, 2026.
- [68] NVIDIA, MITTAL, Mayank, ROTH, Pascal, TIGUE, James, RICHARD, Antoine, ZHANG, Octi, DU, Peter, SERRANO-MUÑOZ, Antonio, YAO, Xinjie, ZURBRÜGG, René, RUDIN, Nikita, WAWRZYNIAK, Lukasz and OTHERS. Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning. Online. 2025. arXiv:2511.04831
- [69] OPEN ROBOTICS. URDF (Unified Robot Description Format) — ROS 2 Documentation (Rolling). Online. 2026. [Accessed 3 March 2026]. Available from: <https://docs.ros.org/en/rolling/Tutorials/Intermediate/URDF/URDF-Main.html> ROS 2 Rolling documentation (accessed 2026-03-03).
- [70] OPEN ROBOTICS. Adding Physical and Collision Properties to a URDF Model — ROS 2 Documentation (Foxy). Online. 2026. [Accessed 3 March 2026]. Available from: <https://docs.ros.org/en/foxy/Tutorials/Intermediate/URDF/Adding-Physical-and->

- [Collision-Properties-to-a-URDF-Model.html](#) IROS 2 Foxy (EOL) tutorial page (accessed 2026-03-03).
- [71] OR, Kanata, WU, Ke-Ya, NAKANO, Kosei, IKEDA, Masashi, ANDO, Mitsuna, KUNIYOSHI, Yasuo and NIYAMA, Ryuma. Curriculum-reinforcement learning on simulation platform of tendon-driven high-degree of freedom underactuated manipulator. *Frontiers in Robotics and AI*. 2023. Vol. 10, p. 1066518. DOI [10.3389/frobt.2023.1066518](#).
- [72] PENG, Xue Bin, ANDRYCHOWICZ, Marcin, ZAREMBA, Wojciech and ABBEEL, Pieter. Sim-to-Real Transfer of Robotic Control with Dynamics Randomization. In : *2018 IEEE International Conference on Robotics and Automation (ICRA)*. 2018. p. 3803–3810. DOI [10.1109/ICRA.2018.8460528](#).
- [73] PENG, Xue Bin, ANDRYCHOWICZ, Marcin, ZAREMBA, Wojciech and ABBEEL, Pieter. Sim-to-Real Transfer of Robotic Control with Dynamics Randomization. *arXiv preprint arXiv:1710.06537*. Online. 2018. Available from: <https://arxiv.org/abs/1710.06537>
- [74] PIXAR ANIMATION STUDIOS. UsdPhysics: USD Physics Schema. Online. 2021. [Accessed 3 March 2026]. Available from: https://openusd.org/docs/api/usd_physics_page_front.html OpenUSD API documentation; generated on 2021-07-15.
- [75] PIXAR ANIMATION STUDIOS. USD Frequently Asked Questions. Online. 2021. [Accessed 3 March 2026]. Available from: <https://openusd.org/release/usdfaq.html> OpenUSD documentation (release 26.03); © 2021 Pixar Animation Studios.
- [76] POPOV, Ivaylo, HEES, Nicolas, LILLICRAP, Timothy, HAFNER, Roland, BARTH-MARON, Gabriel, VECERIK, Matej, LAMPE, Thomas, TASSA, Yuval, EREZ, Tom and RIEDMILLER, Martin. Data-efficient Deep Reinforcement Learning for Dexterous Manipulation. *arXiv preprint arXiv:1704.03073*. 2017.
- [77] POTT, Andreas. *Cable-Driven Parallel Robots – Theory and Application*. Springer, 2018. Springer Tracts in Advanced Robotics.
- [78] POTT, Andreas, BRUCKMANN, Tobias and MIKELSONS, Lars. Closed-Form Force Distribution for Parallel Wire Robots. In : *Computational Kinematics*. Springer, 2009. p. 25–34. DOI [10.1007/978-3-642-01947-0_4](#).
- [79] PRATT, Gill A. and WILLIAMSON, Matthew M. Series Elastic Actuators. 1995. P. 399–406. DOI [10.1109/IROS.1995.525827](#).
- [80] PYBULLET CONTRIBUTORS. Bullet Real-Time Physics Simulation: PyBullet. Online. 2026. [Accessed 3 March 2026]. Available from: <https://pybullet.org/Project> website
- [81] RAO, Priyanka, PEYRON, Quentin, LILGE, Sven and BURGNER-KAHRS, Jessica. How to Model Tendon-Driven Continuum Robots and Benchmark Modelling Performance. *Frontiers in Robotics and AI*. 2021. Vol. 7, p. 630245. DOI [10.3389/frobt.2020.630245](#).
- [82] RASTEGAR, J. and FARDANESH, B. Manipulation workspace analysis using the Monte Carlo Method. *Mechanism and Machine Theory*. 1990. Vol. 25, no. 2, p. 233–239. DOI [10.1016/0094-114X\(90\)90124-3](#).

- [83] ROBOTIQ INC. 2F-85 and 2F-140 Adaptive Gripper Instruction Manual – Specifications. Online. 2019. Available from: <https://robotiq.com/products/adaptive-grippers>Product datasheet
- [84] SAITO, Nozomi and MORIMOTO, Jun. Characterization of Continuum Robot Arms Under Reinforcement Learning and Derived Improvements. *Frontiers in Robotics and AI*. 2022. Vol. 9, p. 895388. DOI [10.3389/frobt.2022.895388](https://doi.org/10.3389/frobt.2022.895388).
- [85] SALISBURY, J. Kenneth and CRAIG, John J. Articulated Hands: Force Control and Kinematic Issues. *The International Journal of Robotics Research*. 1982. Vol. 1, no. 1, p. 4–17. DOI [10.1177/027836498200100102](https://doi.org/10.1177/027836498200100102).
- [86] SALVATO, Erica, FENU, Gianfranco, MEDVET, Eric and PELLEGRINO, Felice Andrea. Crossing the Reality Gap: A Survey on Sim-to-Real Transferability of Robot Controllers in Reinforcement Learning. *IEEE Access*. 2021. Vol. 9, p. 153171–153187. DOI [10.1109/ACCESS.2021.3126658](https://doi.org/10.1109/ACCESS.2021.3126658).
- [87] SCHULMAN, John, MORITZ, Philipp, LEVINE, Sergey, JORDAN, Michael I. and ABBEEL, Pieter. High-Dimensional Continuous Control Using Generalized Advantage Estimation. *arXiv preprint arXiv:1506.02438*. Online. 2015. Available from: <https://arxiv.org/abs/1506.02438>
- [88] SCHULMAN, John, WOLSKI, Filip, DHARIWAL, Prafulla, RADFORD, Alec and KLIMOV, Oleg. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*. Online. 2017. [Accessed 3 March 2026]. Available from: <https://arxiv.org/abs/1707.06347>
- [89] SERRANO-MUÑOZ, Antonio, CHRYSOSTOMOU, Dimitrios, BØGH, Simon and ARANA-AREXOLALEIBA, Nestor. skrl: Modular and Configurable Library for Reinforcement Learning. *Journal of Machine Learning Research*. Online. 2023. Vol. 24, no. 254, p. 1–7. Available from: <https://jmlr.org/papers/v24/23-0112.html>
- [90] SHAH, Dylan S., BOOTH, Joran W., BAINES, Robert L., WANG, Kun, VESPIGNANI, Massimo, BEKRIS, Kostas and KRAMER-BOTTIGLIO, Rebecca. Tensegrity Robotics. *Soft Robotics*. 2022. Vol. 9, no. 4, p. 639–656. DOI [10.1089/soro.2020.0170](https://doi.org/10.1089/soro.2020.0170).
- [91] SHAHID, Asad Ali, TAKANISHI, Atsuo and KUO, Chung-Hsien. Curriculum-Reinforcement Learning on Simulation Platform of Tendon-Driven High-Degree of Freedom Underactuated Manipulator. *Frontiers in Robotics and AI*. 2023. Vol. 10, p. 1066518. DOI [10.3389/frobt.2023.1066518](https://doi.org/10.3389/frobt.2023.1066518).
- [92] SKELTON, Robert E. and OLIVEIRA, Mauricio C. de. *Tensegrity Systems*. Springer, 2009.
- [93] STACHEL, Hellmuth. Higher-Order Flexibility of a Bipartite Planar Framework. *Acta Technica Napocensis – Series: Applied Mathematics and Mechanics*. 2000. Vol. 43, p. 159–167.
- [94] SULTAN, Cornel. Tensegrity: 60 Years of Art, Science, and Engineering. *Advances in Applied Mechanics*. 2009. Vol. 43, p. 69–145. DOI [10.1016/S0065-2156\(09\)43002-3](https://doi.org/10.1016/S0065-2156(09)43002-3).
- [95] SUTTON, Richard S. and BARTO, Andrew G. *Reinforcement Learning: An Introduction*. Online. Second. The MIT Press, 2018. [Accessed 3 March 2026].

- ISBN 9780262039246. Available from: <https://mitpress.mit.edu/9780262039246/reinforcement-learning/>
- [96] TANG, Bingjie, LIN, Michael A., AKINOLA, Iretiayo, HANDA, Ankur, SUKHATME, Gaurav S., RAMOS, Fabio, FOX, Dieter and NARANG, Yashraj. IndustReal: Transferring Contact-Rich Assembly Tasks from Simulation to Reality. In : *Robotics: Science and Systems (RSS)*. 2023. DOI [10.15607/RSS.2023.XIX.069](https://doi.org/10.15607/RSS.2023.XIX.069).
- [97] TOBIN, Josh, FONG, Rachel, RAY, Alex, SCHNEIDER, Jonas, ZAREMBA, Wojciech and ABBEEL, Pieter. Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. In : *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Online. 2017. DOI [10.1109/IROS.2017.8202133](https://doi.org/10.1109/IROS.2017.8202133).
- [98] TODOROV, Emanuel, EREZ, Tom and TASSA, Yuval. MuJoCo: A Physics Engine for Model-Based Control. In : *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2012. p. 5026–5033. DOI [10.1109/IROS.2012.6386109](https://doi.org/10.1109/IROS.2012.6386109).
- [99] TORO ICARTE, Rodrigo, KLASSEN, Toryn Q., VALENZANO, Richard and MCILRAITH, Sheila A. Reward Machines: Exploiting Reward Function Structure in Reinforcement Learning. *Journal of Artificial Intelligence Research*. 2022. Vol. 73, p. 173–208.
- [100] TSAI, Lung-Wen. *Robot Analysis: The Mechanics of Serial and Parallel Manipulators*. Wiley, 1999.
- [101] TSAI, Lung-Wen and LEE, Jae-June. Kinematic Analysis of Tendon-Driven Robotic Mechanisms Using Graph Theory. *ASME Journal of Mechanisms, Transmissions, and Automation in Design*. 1989. Vol. 111, no. 1, p. 59–65.
- [102] UICKER, John J., PENNOCK, Gordon R. and SHIGLEY, Joseph E. *Theory of Machines and Mechanisms*. 4. Oxford University Press, 2011.
- [103] VANDERBORGHT, Bram, ALBU-SCHÄFFER, Alin, BICCHI, Antonio, BURDET, Etienne, CALDWELL, Darwin G., CARLONI, Raffaella, CATALANO, Manuel, EIBERGER, Oliver and OTHERS. Variable Impedance Actuators: A Review. *Robotics and Autonomous Systems*. 2013. Vol. 61, no. 12, p. 1601–1614. DOI [10.1016/j.robot.2013.06.009](https://doi.org/10.1016/j.robot.2013.06.009).
- [104] VERHOEVEN, Richard. *Analysis of the Workspace of Tendon-based Stewart Platforms*. Doctoral dissertation. 2004.
- [105] WALTER, Jonas, ROTHFISCHER, Lisa, STIERSTORFER, Robin, NEMOTO, Takara, FRANKE, Jörg and REITELSHÖFER, Sebastian. Tensegrity-Inspired Joint Can Protect from Impacts by Isolating. In : *Cable-Driven Parallel Robots – Proceedings of the 6th International Conference on Cable-Driven Parallel Robots (CableCon 2023)*. Springer, 2023. p. 344–355. Mechanisms and Machine Science. DOI [10.1007/978-3-031-32322-5_28](https://doi.org/10.1007/978-3-031-32322-5_28).
- [106] YOON, Dongwon, KANG, Lev, MANZOOR, Shehryar and CHOI, Young-Bong. The Improved DLR Wrist: Design and Analysis of 2-Degrees-of-Freedom Rotational Mechanism Using Spatial Antiparallelogram Linkages. *ASME Journal of Mechanical Design*. 2021. Vol. 143, no. 5, p. 53303. DOI [10.1115/1.4048719](https://doi.org/10.1115/1.4048719).

- [107] YOSHIKAWA, Tsuneo. Manipulability of Robotic Mechanisms. *The International Journal of Robotics Research*. 1985. Vol. 4, no. 2, p. 3–9. DOI [10.1177/027836498500400201](https://doi.org/10.1177/027836498500400201).
- [108] ZHU, Yuke, WONG, Josiah, MANDLEKAR, Ajay, MARTÍN-MARTÍN, Roberto, JOSHI, Abhishek, NASIRIANY, Soroush and ZHU, Yifeng. robosuite: A Modular Simulation Framework and Benchmark for Robot Learning. *arXiv preprint arXiv:2009.12293*. 2020.

A Appendix: Simulation and Control Parameters

A.1 Joint Drive Parameters

Table 21 lists the actuator drive parameters for the tensegrity manipulator's joints.

Joint Group	Stiffness	Damping	Effort Limit	Vel. Limit
base_y	8 000	800	300 N	5.0 m/s
base_z	8 000	800	200 N	5.0 m/s
elbow	400	20	40 N·m	1.0 rad/s
wrist (both)	400	20	10 N·m	0.5 rad/s

Table 21: Implicit actuator PD drive parameters for the tensegrity manipulator.

A.2 Step-Response PID Parameters

Table 22 lists the PID parameters used in the step-response validation tests (Section 4.5). Gains are scaled $\approx 133 \times$ from Klein's originals to match Isaac Sim's effective rotational inertia.

Joint	K_p	K_i	K_d	Saturation (N)
elbow_joint	40	4	3	400
wrist_y_joint	10	2	0.3	600
wrist_x_joint	10	2	0.3	600

Table 22: PID gains and torque saturation for step-response testing.

The PID controllers include feedforward gravity compensation: $\tau_g = m_e \cdot g \cdot l_c \cdot \sin(\theta)$.

B Appendix: PPO Hyperparameters

Table 23 compares the full PPO configurations for the reach and cube place tasks.

Parameter	Reach	Cube Place
Seed	42	42
Network architecture	[64, 64] ELU	[256, 128, 64] ELU
Shared policy/value	Yes	Yes
Rollout horizon	24 steps	48 steps
Learning epochs	5	5
Mini-batches	4	8
Learning rate	10^{-3}	10^{-4}
LR scheduler	KL-adaptive	KL-adaptive
KL threshold	0.01	0.004
Discount γ	0.99	0.99
GAE λ	0.95	0.95
Clip ratio ϵ	0.2	0.2
Value clip	0.2	0.2
Entropy scale	0.01	0.01
Value loss scale	1.0	1.0
Grad norm clip	1.0	1.0
Initial log-std range	$[-20, 2]$	$[-2.5, 2]$
State preprocessor	RunningStandardScaler	RunningStandardScaler
Total timesteps	48 000	500 000
num_envs	4 096	4 096

Table 23: Full PPO hyperparameter comparison between reach and cube place tasks.

C Appendix: Cube Place Reward Terms

Table 24 lists all reward terms used in the cube place task, including regularization and metric-only terms.

Term	Weight	Description	Gate
<i>Task Rewards</i>			
reaching_object	+2.0	Fingertip-to-cube distance shaping	—
grasping	+5.0	Closure $\times$ proximity	—
lifting_object	+3.0	Binary: above belt, near gripper	—
height_bonus	+5.0	$\frac{\min(\Delta z, 0.30)}{0.30}$	—
object_velocity_penalty	+5.0	Penalize excessive cube motion	—
goal_tracking	+40.0	$1 - \tanh\left(\frac{d_{xy}}{1.0}\right)$	was_grasped
goal_tracking_fine	+10.0	$1 - \tanh\left(\frac{d_{xy}}{0.20}\right)$	was_grasped
release	+25.0	$(1 - \text{cl.}) \times \text{in}_{xy} \times \text{above}$	was_grasped
green_in_target	+100.0	Cube center inside drum volume	was_grasped
red_in_target	−12.0	Red cube in drum (penalty)	red active
<i>Regularization</i>			
object_stay_bonus	+3.0	Bonus for cube remaining on belt	—
action_rate	$-10^{-4} \rightarrow -2 \times 10^{-3}$	L2 action delta (curriculum)	—
joint_vel	$-10^{-4} \rightarrow -2 \times 10^{-3}$	L2 joint velocity (curriculum)	—
fingertip_orientation	+0.5	Fingertip alignment shaping	—
belt_contact	−10.0	Finger tips below belt surface	—
joint_torque	−0.05	L2 torque penalty	—
cube_off_conveyor	−5.0	Cube knocked off belt	—
<i>Metric-Only (Small Weight)</i>			
cube_height_info	+0.01	Cube height above belt	—
gripper_state_info	+0.01	Gripper closure tracking	—
collision_penalty_info	−0.01	Self-collision monitoring	—

Table 24: Complete cube place reward specification. Curriculum terms ramp weights at 200k training steps.

D Appendix: Reach Task — Complete Training Metrics

This appendix provides complete training curves for the reach task across all three tensegrity variants. All plots are cut at 48k timesteps (PD variant training duration).

D.1 Reward Overview

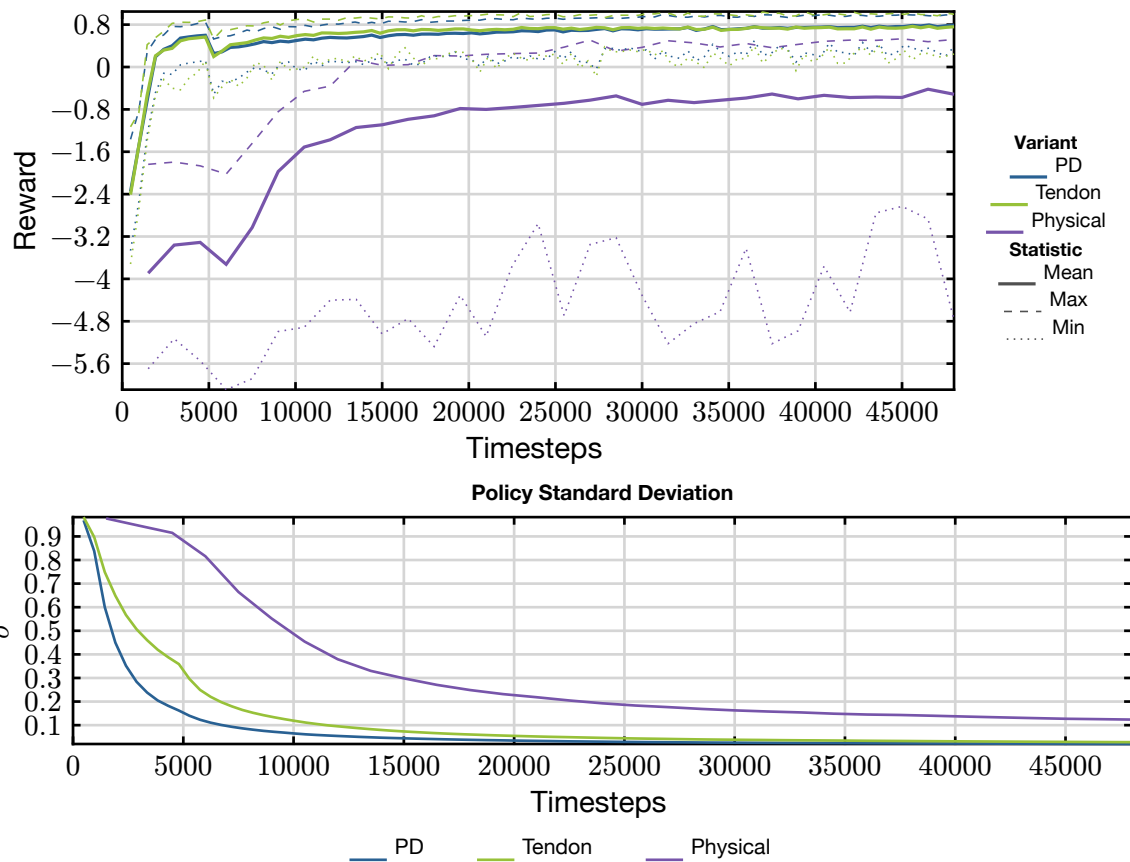


Figure 31: Reach task reward overview. Top: total reward mean (solid), max (dashed), and min (dotted) for all three variants. Bottom: policy standard deviation (σ).

D.2 Task Reward Components

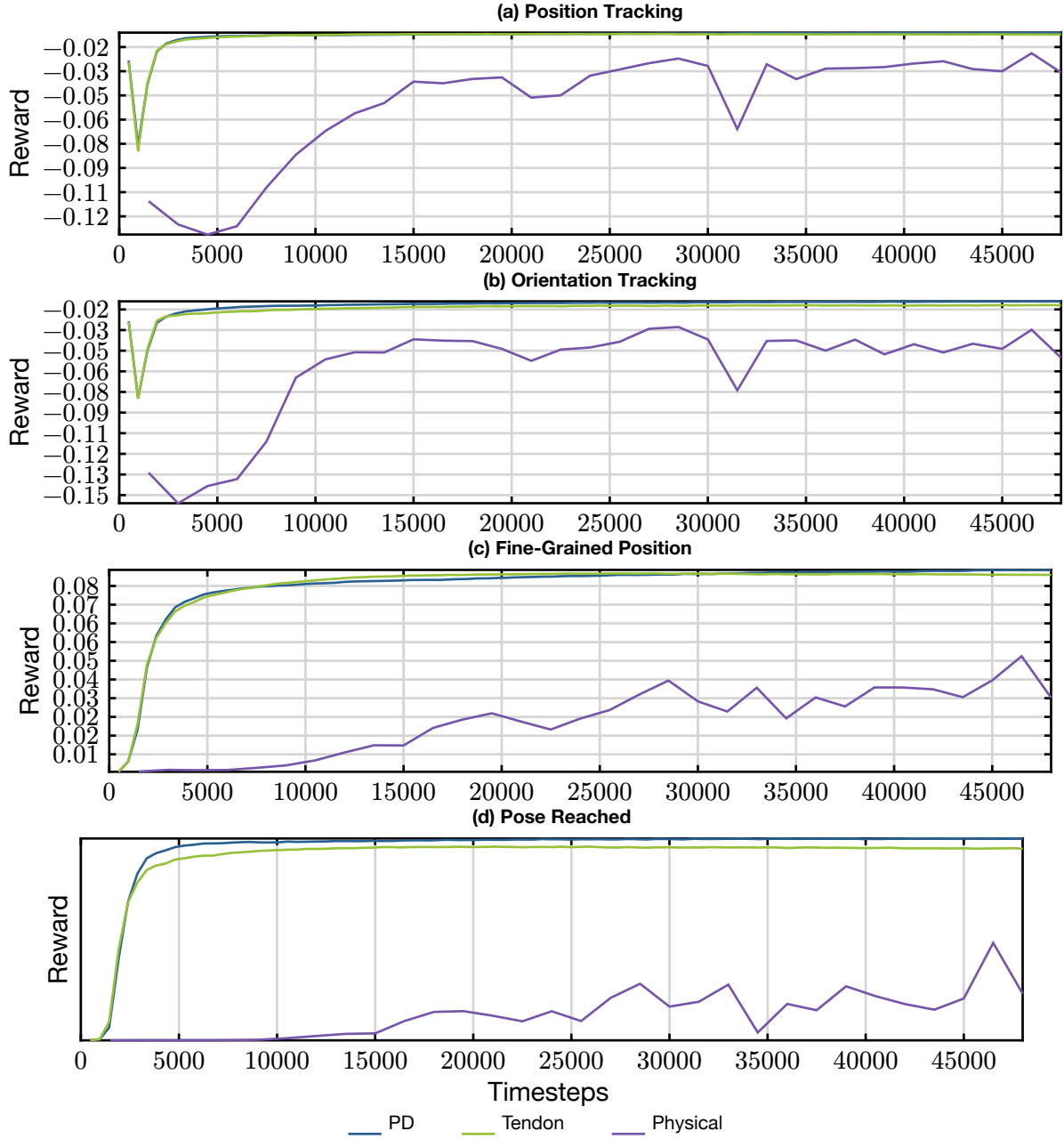


Figure 32: Reach task reward components. **(a)** End-effector position tracking ($1 - \tanh(d_{\text{pos}})$). **(b)** Orientation tracking ($1 - \tanh(d_{\text{orient}})$). **(c)** Fine-grained position tracking (tight tolerance). **(d)** Pose reached bonus.

D.3 Regularization and Training Diagnostics

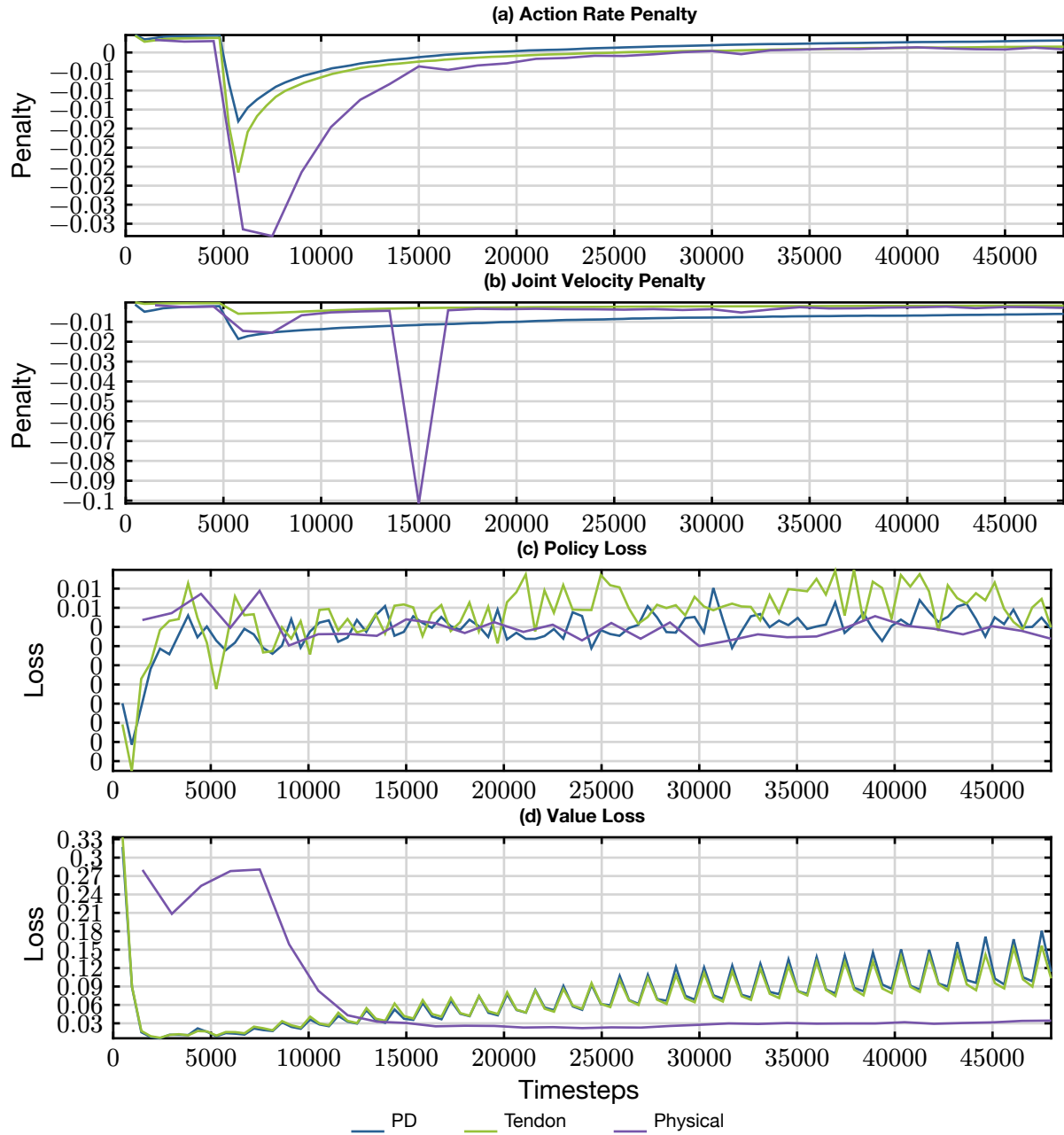


Figure 33: Reach task regularization and training diagnostics. **(a)** Action rate penalty (L2 action delta). **(b)** Joint velocity penalty. **(c)** Policy loss. **(d)** Value function loss.

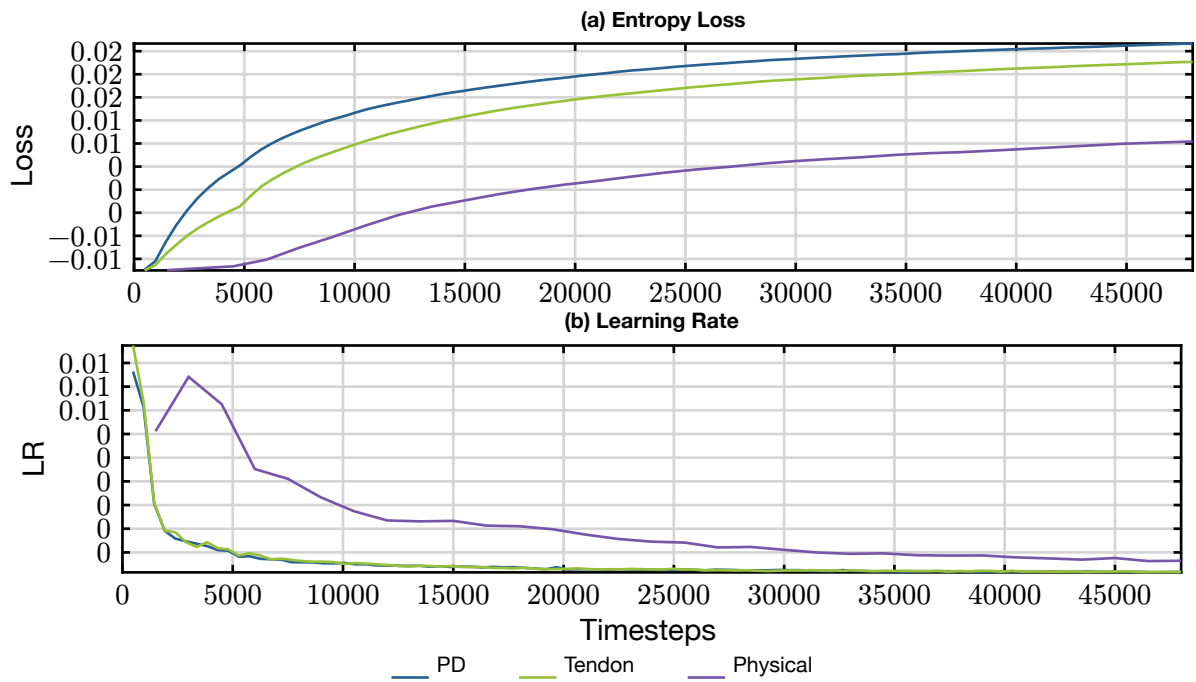


Figure 34: Reach task additional diagnostics. **(a)** Entropy loss (exploration incentive). **(b)** KL-adaptive learning rate schedule.

E Appendix: Cube Place Task — Complete Training Metrics

This appendix provides complete training curves for the cube place task across all three tensegrity variants. All plots are cut at 300k timesteps (PD variant training duration).

E.1 Reward Overview

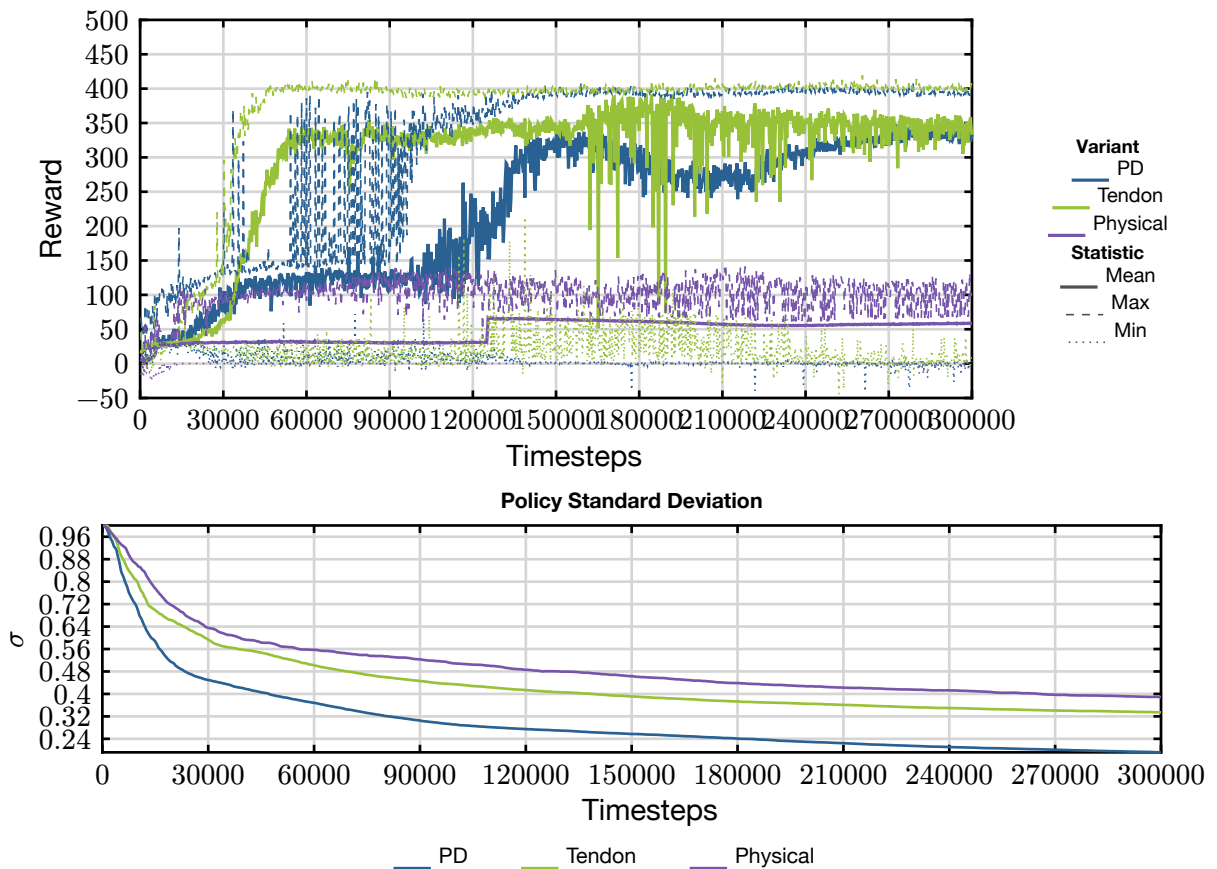


Figure 35: Cube place reward overview. Top: total reward mean (solid), max (dashed), and min (dotted) for all three variants. Bottom: policy standard deviation (σ).

E.2 Task Metrics

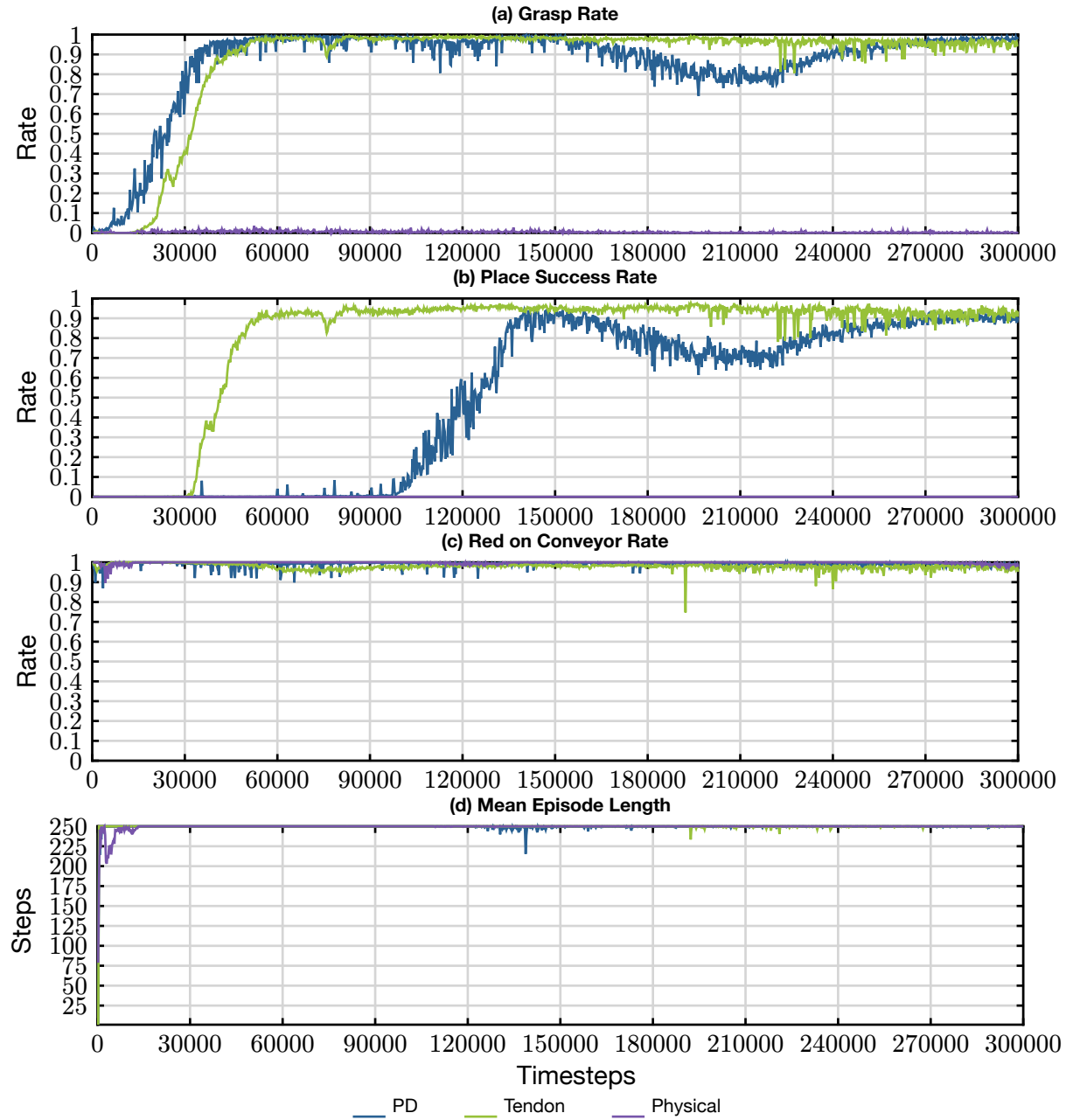


Figure 36: Cube place task metrics. **(a)** Grasp success rate. **(b)** Place success rate. **(c)** Red cube safety rate (remaining on conveyor). **(d)** Mean episode length.

E.3 Task Reward Components – Manipulation Pipeline

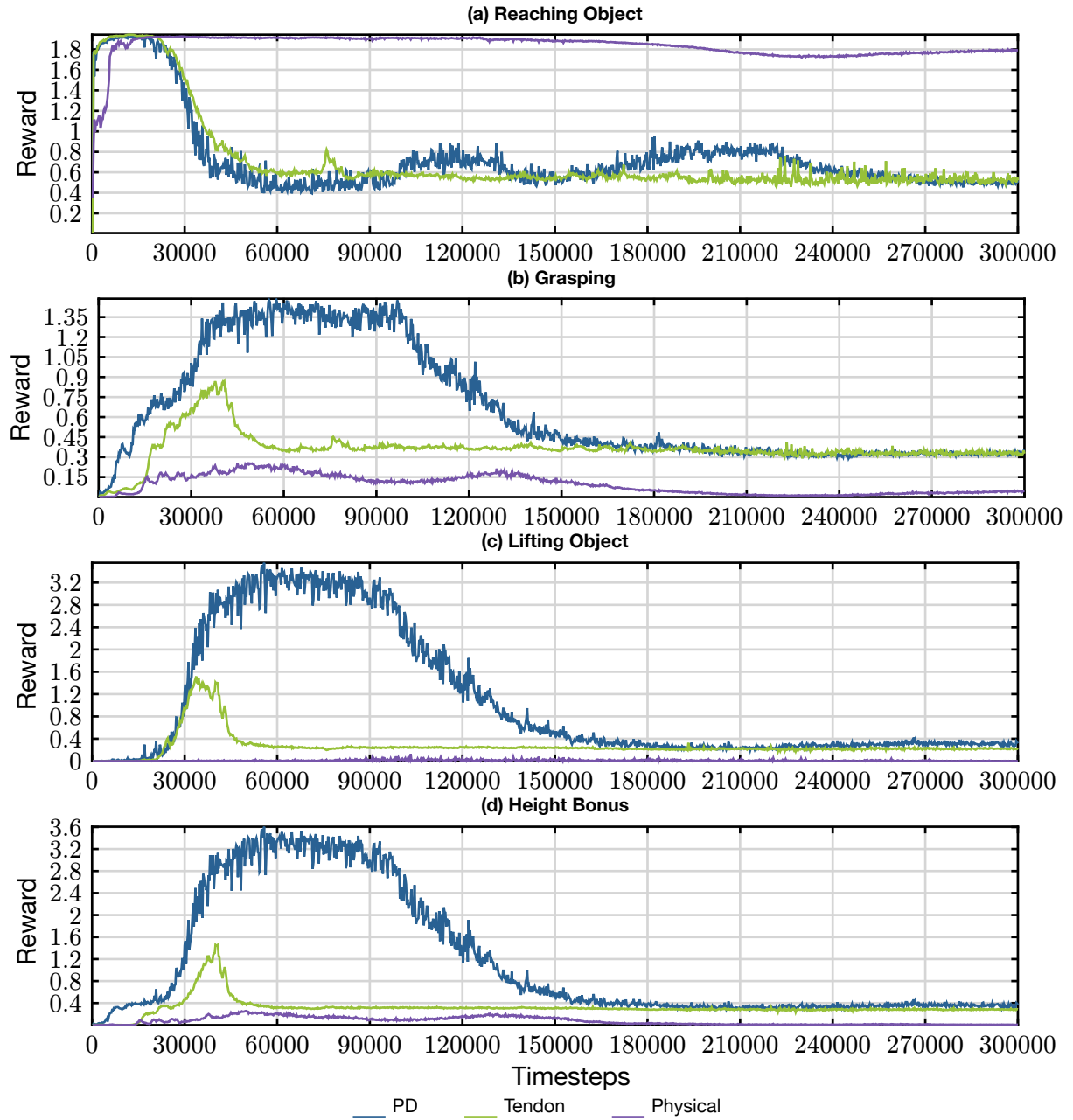


Figure 37: Cube place manipulation pipeline rewards. **(a)** Distance shaping toward the cube. **(b)** Grasp reward (closure $\times$ proximity). **(c)** Lifting binary reward. **(d)** Height bonus (proportional to lift distance).

E.4 Task Reward Components — Placement and Target

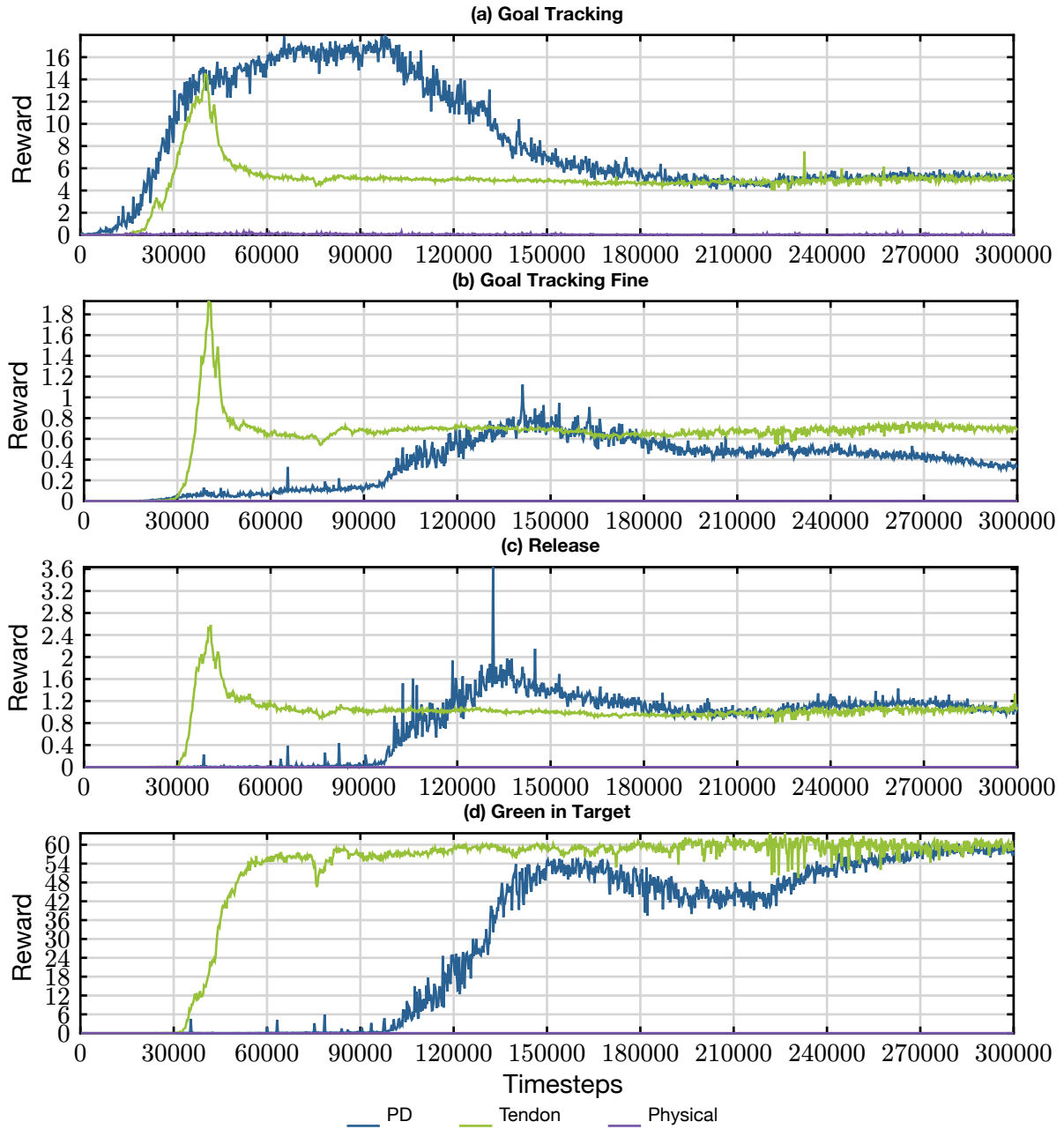


Figure 38: Cube place placement rewards. **(a)** Goal tracking shaping ($1 - \tanh\left(\frac{d_{xy}}{1.0}\right)$), gated on grasp. **(b)** Fine goal tracking ($1 - \tanh\left(\frac{d_{xy}}{0.20}\right)$). **(c)** Release reward. **(d)** Green cube in target drum bonus.

E.5 Regularization and Training Diagnostics

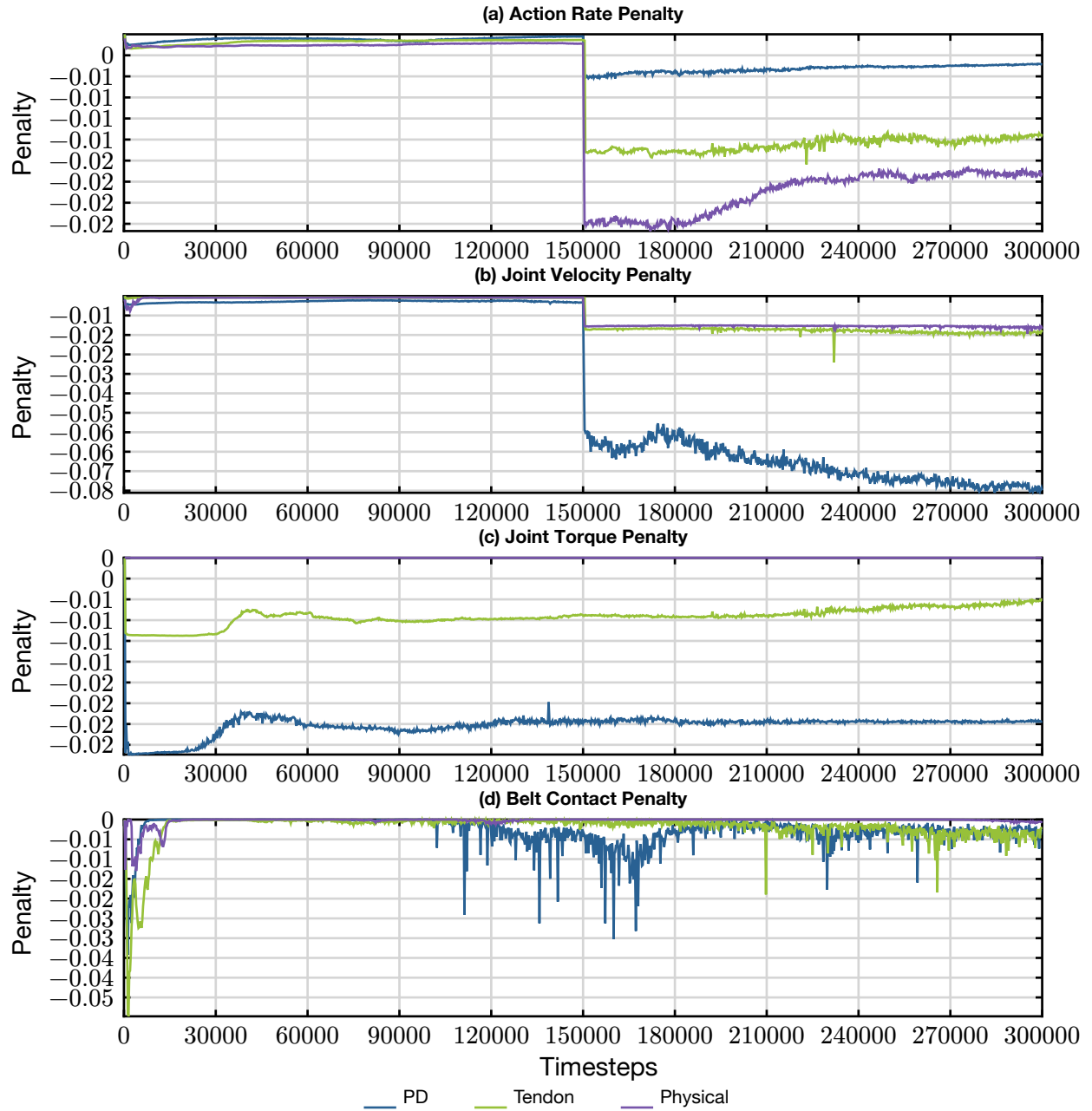


Figure 39: Cube place regularization terms. **(a)** Action rate penalty (ramps at 200k). **(b)** Joint velocity penalty (ramps at 200k). **(c)** Joint torque penalty. **(d)** Belt contact penalty.

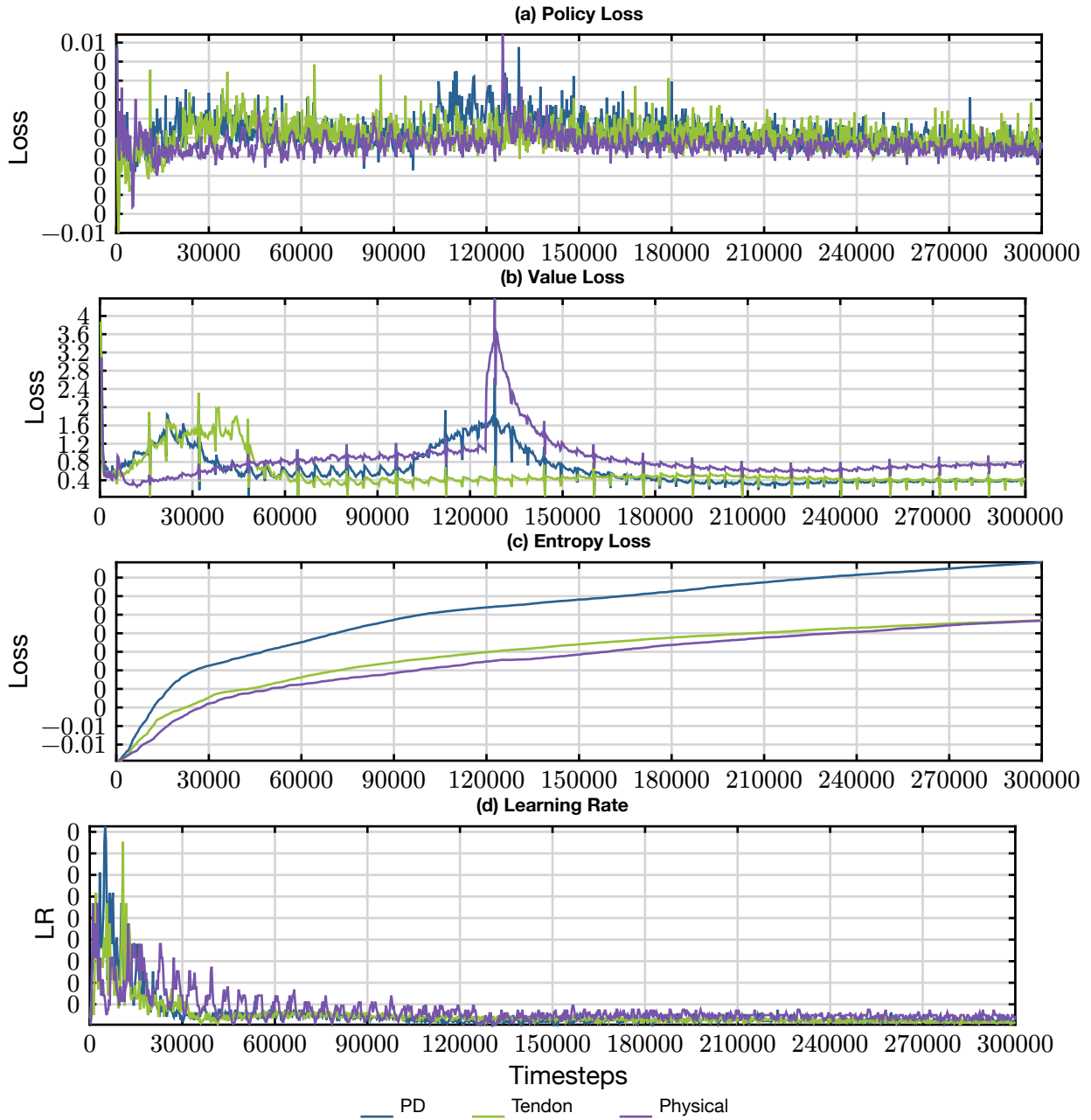


Figure 40: Cube place training diagnostics. **(a)** Policy loss. **(b)** Value function loss. **(c)** Entropy loss. **(d)** KL-adaptive learning rate schedule.

F Appendix: Cube Sort Task — Preliminary Training Metrics

This appendix documents the preliminary cube sort training metrics using the PD tensegrity variant (20k timesteps, training ongoing). Only the PD variant has been trained on this task.

F.1 Reward Overview

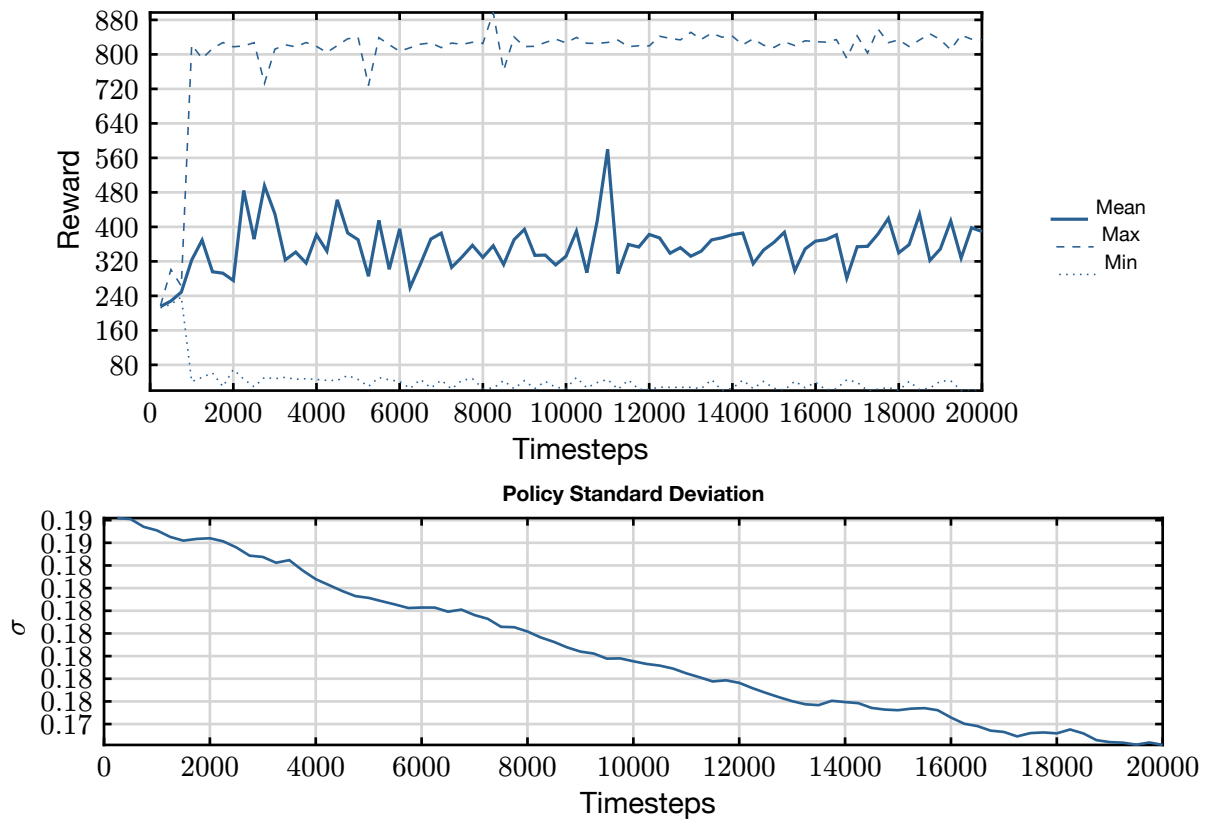


Figure 41: Cube sort reward overview (PD variant). Top: total reward mean (solid), max (dashed), and min (dotted). Bottom: policy standard deviation (σ).

F.2 Task Metrics

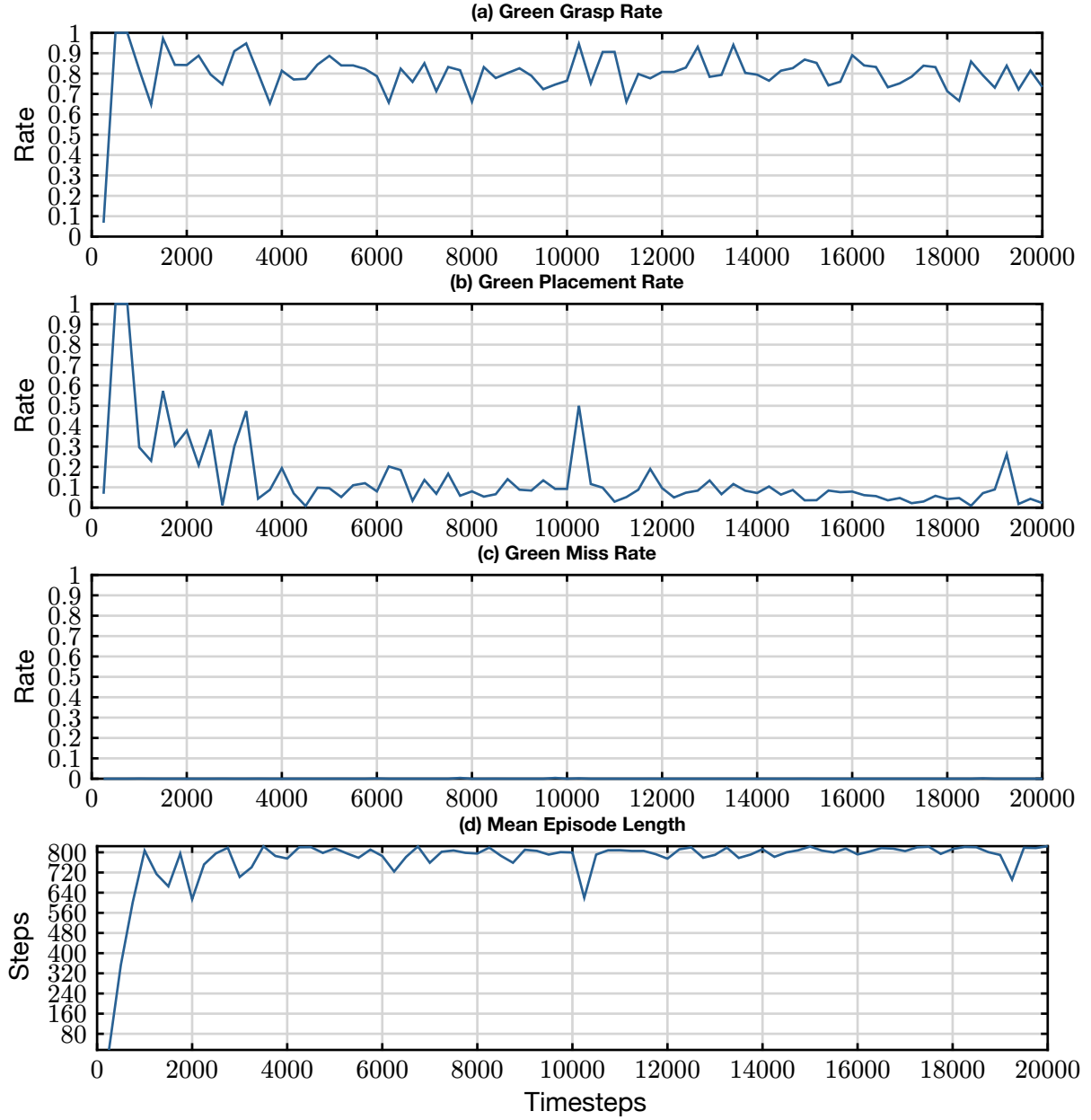


Figure 42: Cube sort task metrics (PD variant). **(a)** Green cube grasp rate. **(b)** Green cube placement rate. **(c)** Green cube miss rate. **(d)** Mean episode length.

F.3 Reward Components

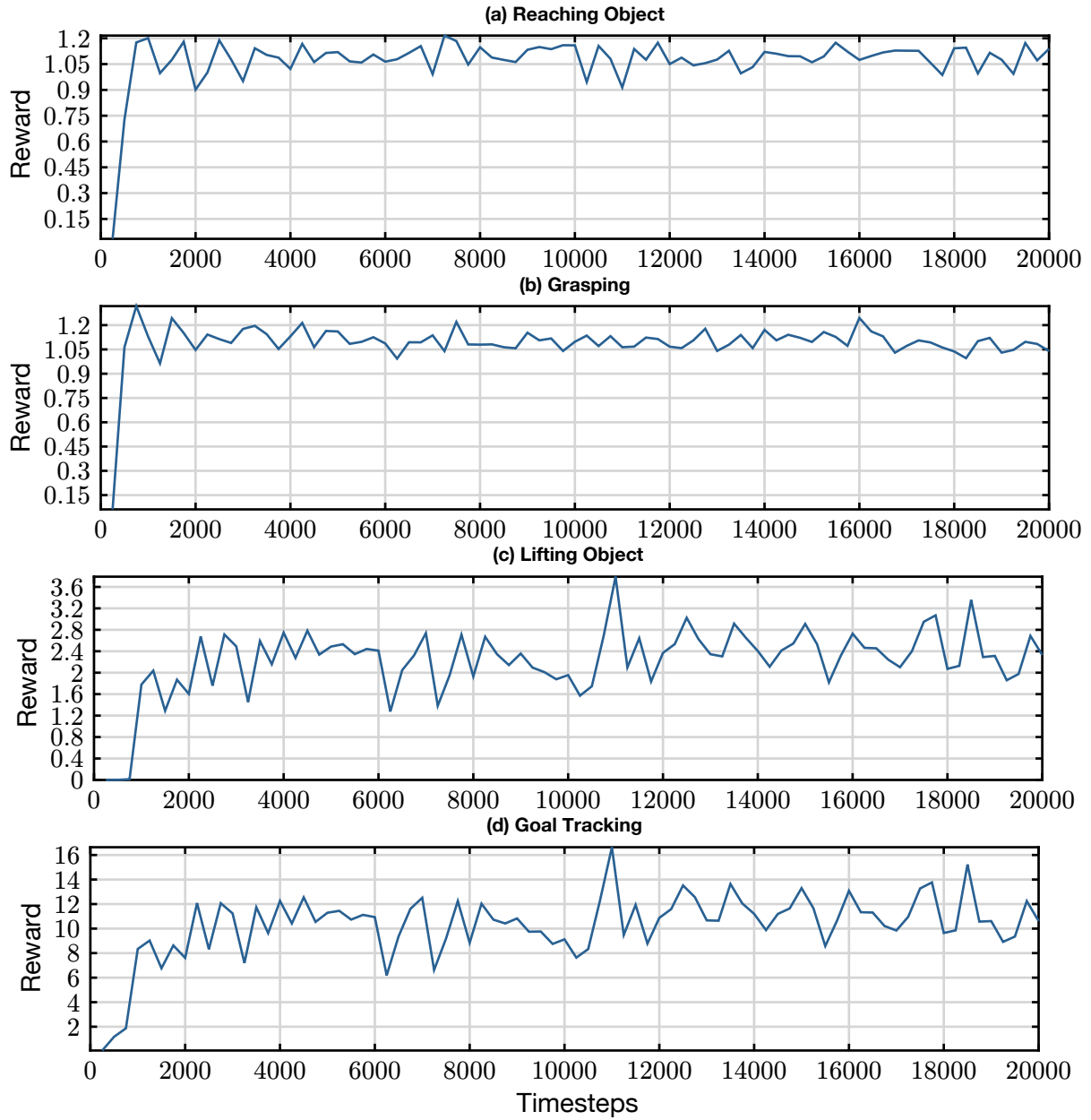


Figure 43: Cube sort reward components (PD variant). **(a)** Approach shaping. **(b)** Grasp reward. **(c)** Lifting reward. **(d)** Goal tracking shaping.

F.4 Training Diagnostics

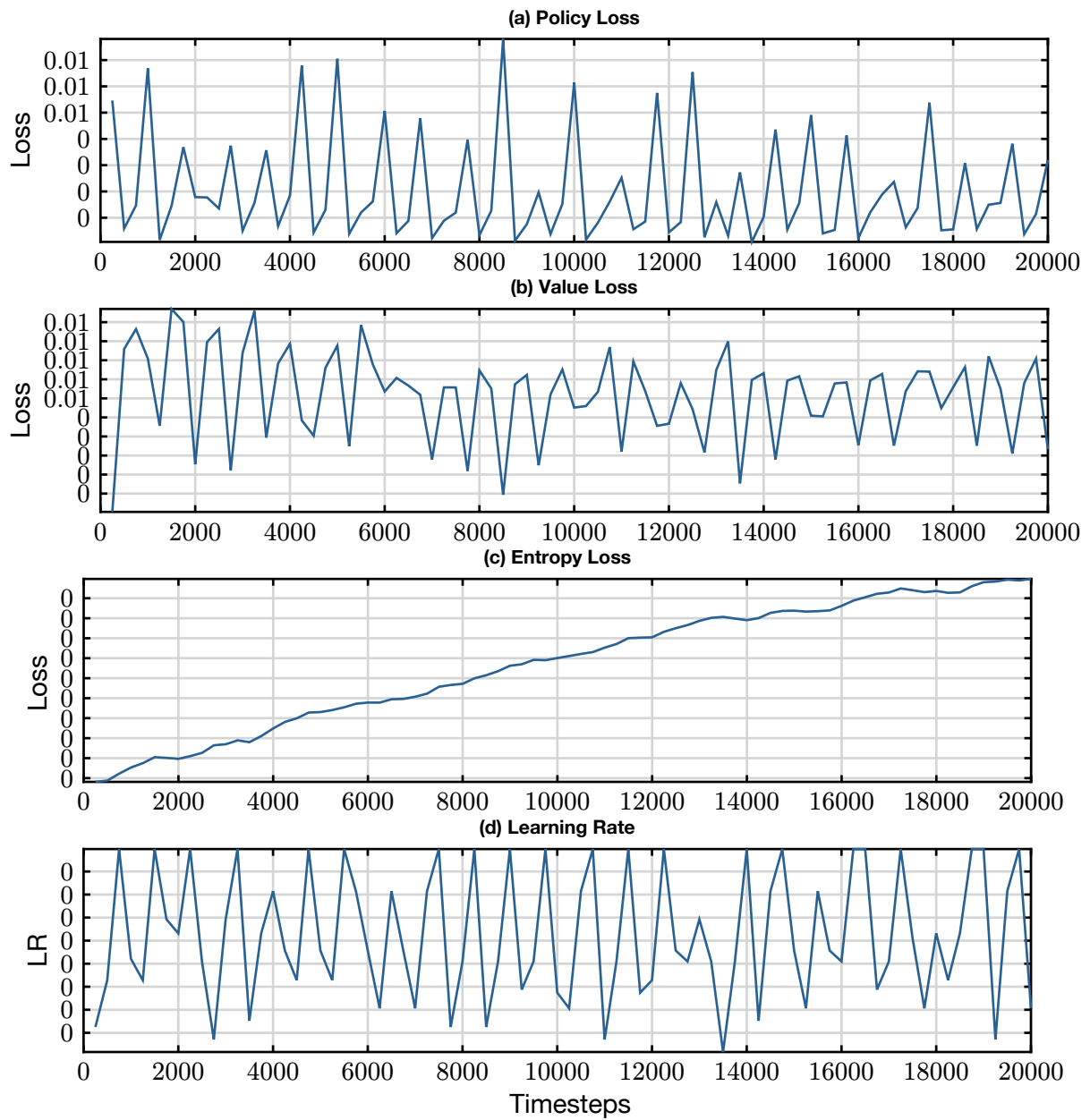


Figure 44: Cube sort training diagnostics (PD variant). **(a)** Policy loss. **(b)** Value function loss. **(c)** Entropy loss. **(d)** KL-adaptive learning rate schedule.

G Appendix: Training Monitor

To observe training progress in real time, a custom monitoring tool was developed as part of this work. Reinforcement learning experiments in Isaac Lab using the SKRL library produce TensorBoard event files containing all logged scalar metrics. While TensorBoard itself provides a web-based visualization, it requires a graphical browser and does not present task-specific summaries such as training speed, estimated time of arrival, or reward decomposition in a consolidated view. Since the training workstation is accessed exclusively over SSH, a terminal-native solution was needed.

The monitor is implemented as a Python package (`tools.monitor`) with four modules. The **data layer** (`core.py`) handles run discovery and TensorBoard parsing. It recursively scans the log directories for `run_status.json` files that the training scripts write at startup, and uses the `tbparse` library to read scalar events into a Pandas DataFrame. All metric tag names are auto-discovered on every refresh cycle — no metric names are hardcoded. Tags are categorized by their prefix (e.g.

`Episode_Reward/`, `Loss/`, `Metrics/`) so the dashboard can group them semantically. The module also computes training speed in steps per second and estimates the remaining time from wall-clock timestamps.

The **terminal frontend** (`terminal.py`) builds a Rich Live dashboard rendered directly in the terminal. It uses `plotext` to draw inline line charts of reward curves and policy diagnostics using Unicode block characters. Three switchable views are provided — **Overview**, **Reward Charts**, and **Policy** — navigable with the number keys 1–3. A background thread reads raw keypresses via `termios` to allow view switching without interrupting the Rich render loop. Figure 45 shows the overview with tabular reward terms, learning metrics, and custom task metrics. Figure 46 shows the reward chart view with total reward (mean/max/min) and the top reward term decomposition. Figure 47 shows the policy diagnostics view with standard deviation, losses, and learning rate.

The **web frontend** (`web.py`) provides an alternative browser-based dashboard using Streamlit with interactive Plotly charts. It offers the same information but with zoom, pan, and hover capabilities. For security, the web server binds only to localhost; access from other machines is only supported via SSH port forwarding. Figure 48 shows the web dashboard.

The monitor auto-refreshes at a configurable interval (default 10 seconds) and supports command-line flags for run selection, inclusion of completed runs, and custom log root directories. A convenience shell script wraps the invocation with the required `conda run --no-capture-output` flags to prevent output buffering.

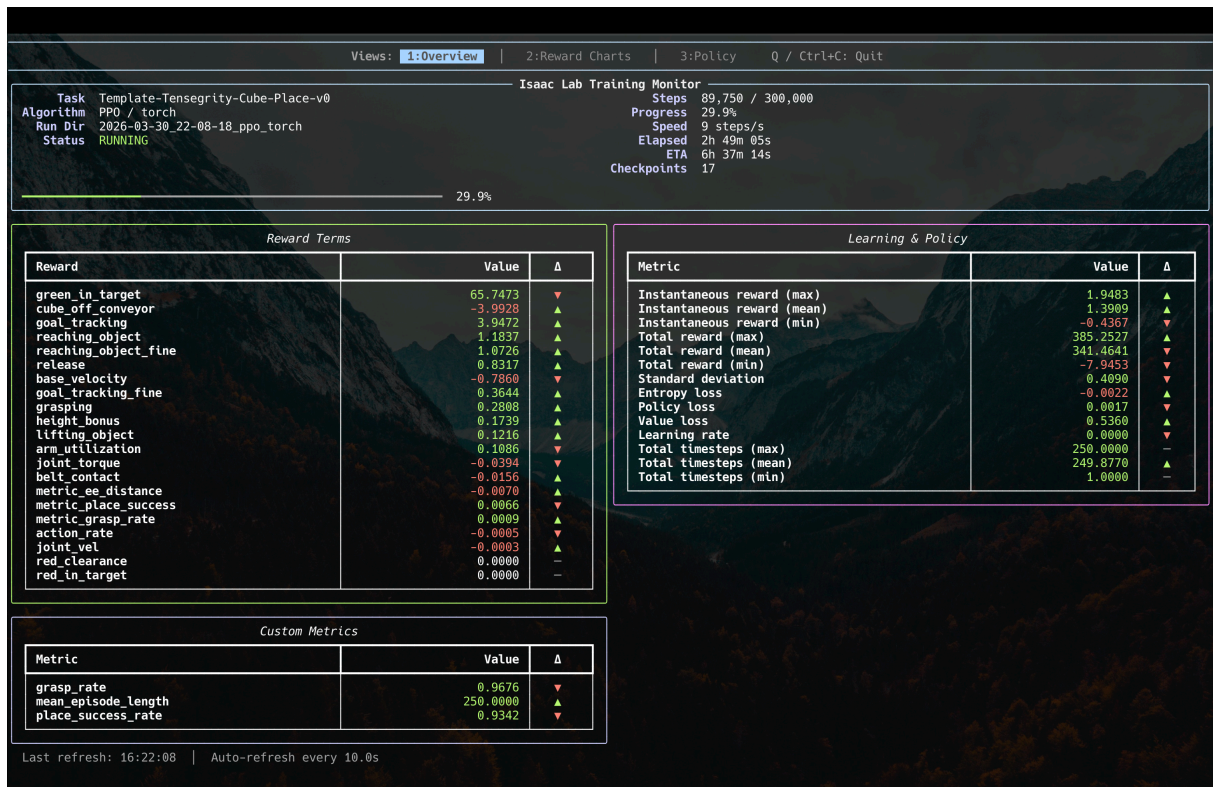


Figure 45: Terminal monitor overview: header with task name, progress bar, and ETA; tables of all reward terms with trend indicators, learning and policy statistics, and custom task metrics (grasp rate, success rate).

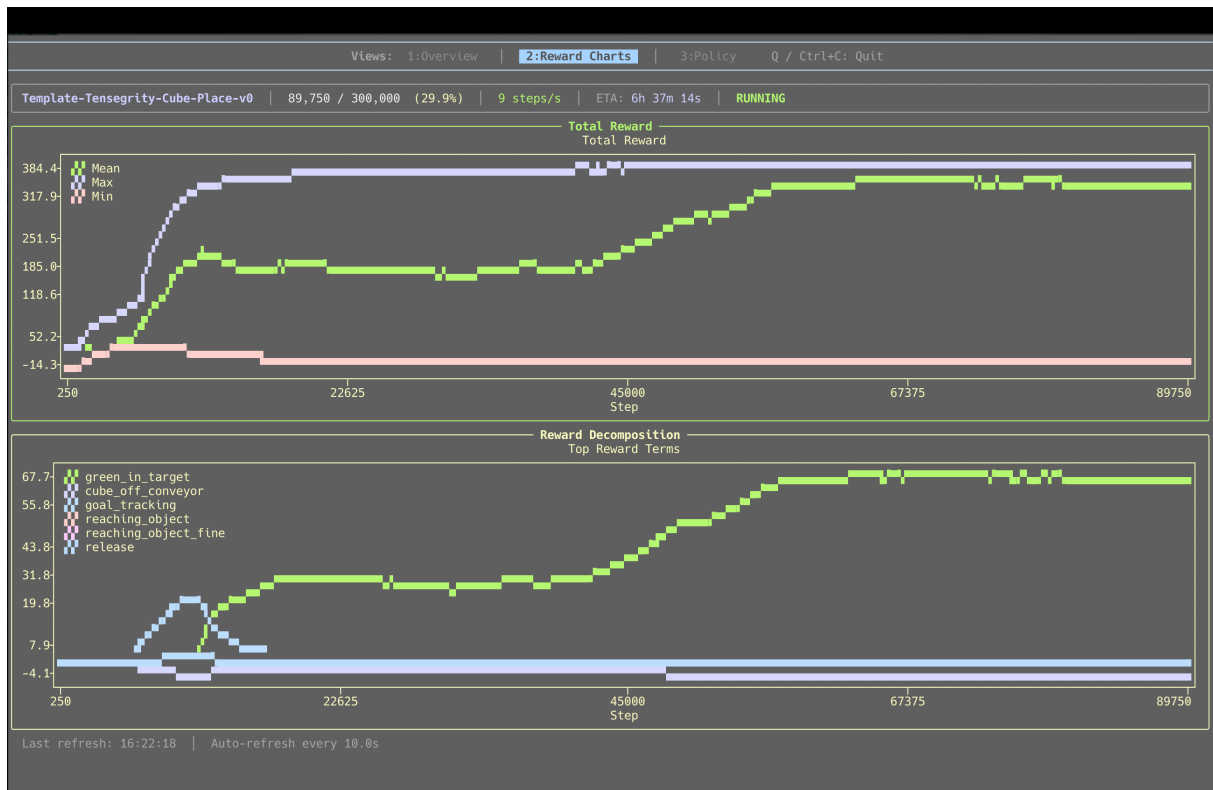


Figure 46: Terminal monitor reward charts: total reward with mean, max, and min envelopes (top) and decomposition of the top individual reward terms over training steps (bottom). Charts are rendered inline using plotext Unicode block characters.

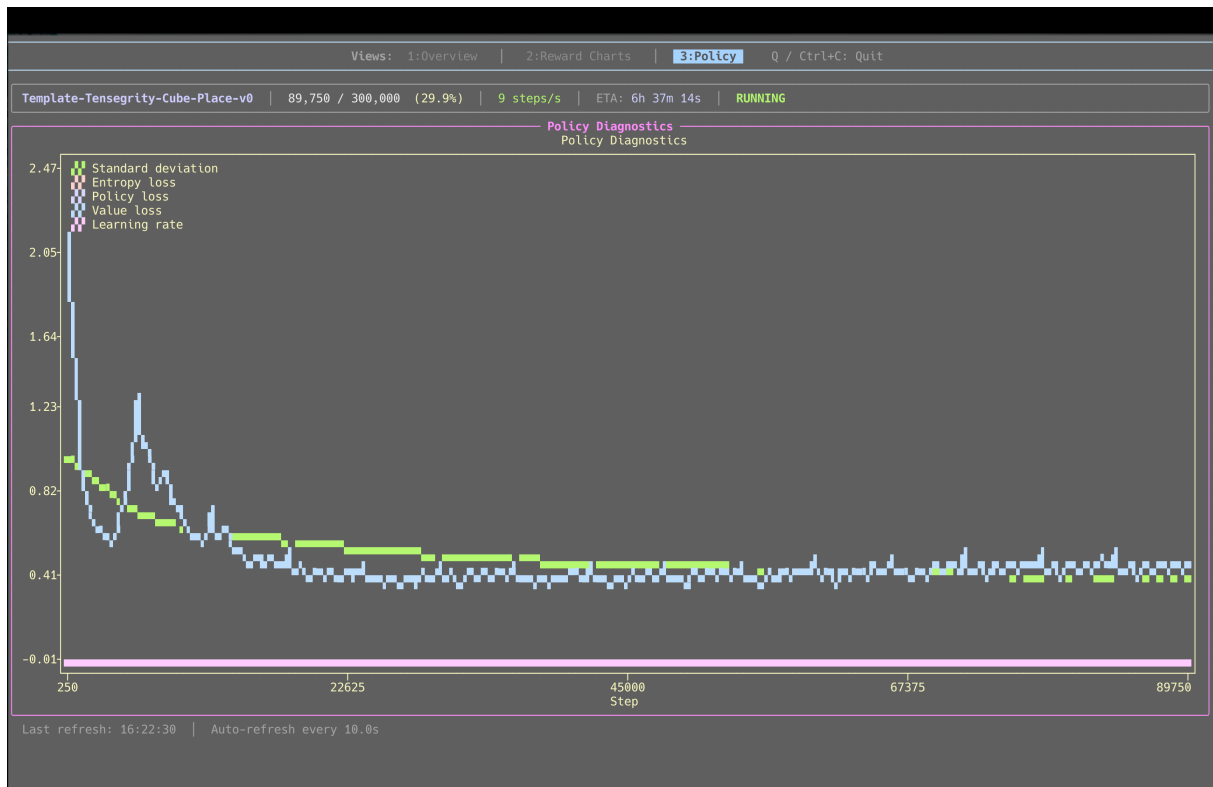


Figure 47: Terminal monitor policy diagnostics: policy standard deviation, entropy loss, policy loss, value loss, and learning rate over training steps.

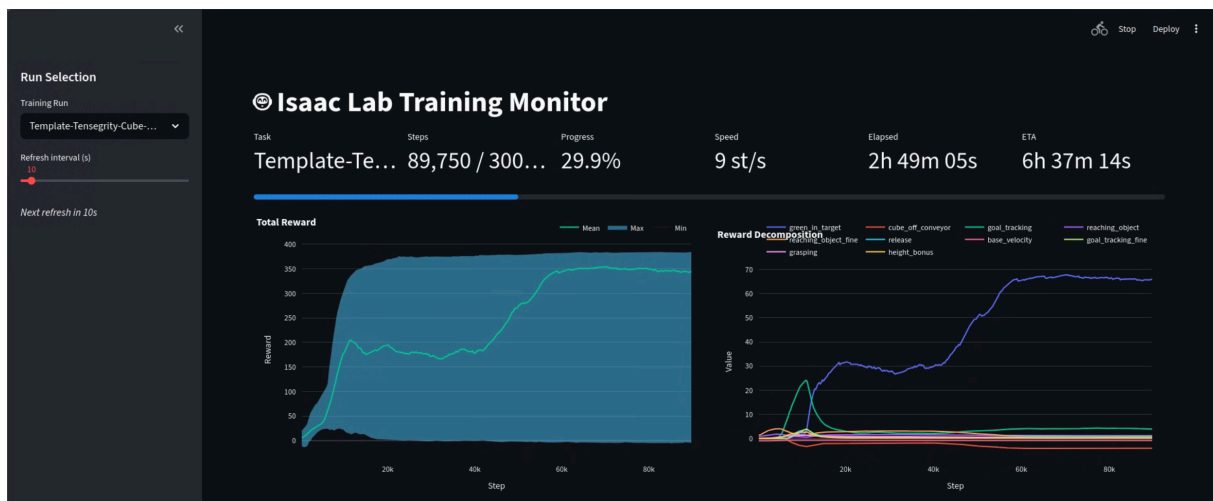


Figure 48: Streamlit web dashboard with interactive Plotly charts. The sidebar provides run selection and refresh interval controls. Metric cards show steps, progress, speed, elapsed time, and ETA. The main area displays total reward with a filled min/max envelope and reward decomposition with a toggleable legend.